UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE & ALGORITHM

SEARCHING TECHNIQUES 2

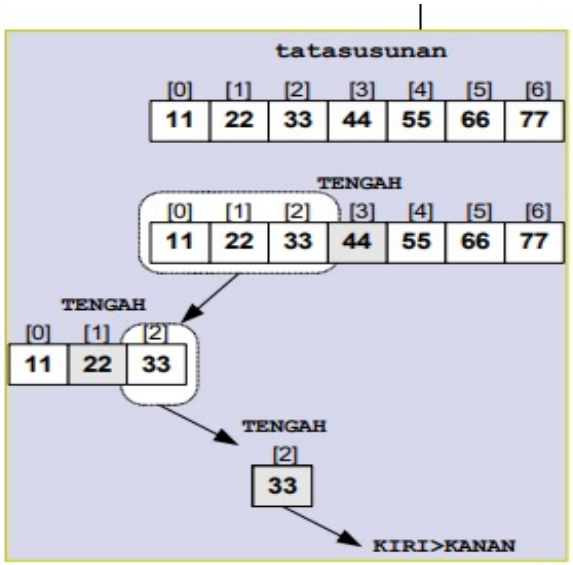
Nor Bahiah Hj Ahmad & Dayang Norhayati A.Jawawi
School of Computing

Binary Search

- The drawbacks of sequential search algorithm is having to traverse the entire list, **O(n)**
- Sorting the list minimize the cost of traversing the whole data set, but we can improve the searching efficiency by using the Binary Search algorithm
- Consider a list in ascending sorted order. For a sequential search, searching is from the beginning until an item is found or the end is reached. Binary search improve the algorithm by removing as much of the data set as possible so that the item is found more quickly.
- Search process is started at the middle of the list, then the algorithm determine which half the item is in (because the list is sorted).
  - It divides the working range in half with a single test. By repeating the procedure, the result is an efficient search algorithm, **($\log_2$ n).**

Binary Search

- The algorithm starts by comparing the search key with the element at the middle of the list
  - If the value matches, it will be return to the calling function with (index = MIDDLE)
  - If the search key is smaller than the middle element, search will be focused on the elements between the first element to the element before the middle element (MIDDLE -1)
  - If the search key is not found, the element at the middle of the first element and the MIDDLE –1 element will be compared with the search key
  - If the value is bigger, search will only be focused on the elements between the second MIDDLE element to the first MIDDLE element.
- Search is repeated until the searched key is found or the last element in the subset is traversed.

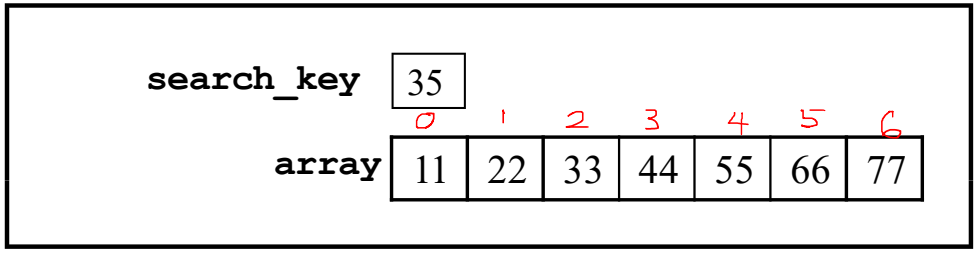


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Binary Search Function

```
int binary_search( int  search_key, int array_size,
                       const int array [] )
{   bool found = false;
    int index = -1 //-1 means record not found
    int MIDDLE,
    LEFT = 0,
    RIGHT = array_size-1;
    while (( LEFT<= RIGHT ) && (!found))
    {    MIDDLE = (LEFT + RIGHT )/ 2;    // Get middle index
         if ( array[MIDDLE] == search_key)
         {     index = MIDDLE;
               found = true;
         }
        else if (array[MIDDLE] > search_key)
              RIGHT= MIDDLE- 1;    // search is focused
                                   // on the left side of list
        else
              LEFT= MIDDLE+ 1      // search is focused
                                 // on the right side of the list
       } //end while
     return index;
}//end function
```

Binary Search on a Sorted List



- Search starts by obtaining the **MIDDLE**index of the array
- assume:

  `search_key` = 35
- **`MIDDLE`**= (0 + 6) / 2

  = 3 { First **`MIDDLE`** index}

Binary Search on a Sorted List..cont

```
else if (array[MIDDLE] > search_key)
    RIGHT= MIDDLE- 1;   // search is focused
                        // on the left side of list
else
    LEFT= MIDDLE+ 1     // search is focused
                        // on the right side of the list
```

search_key 35

array	11	22	33	44	55	66	77

y 35 is compared with the element at the fourth
e array, which is `array[3]` with the value **44**.

_key < MIDDLE value, therefore search will be
d on the elements between the first index and the
dex only (index 1 to **MIDDLE-1**)

- rocess to obtain **MIDDLE** index is repeated

  MIDDLE = (0 + 2) / 2

  = 1 { second **MIDDLE** index}

- **Search_key 35** is compared with the element at the second index, which is `array[1]` with the value **22**
  - **search_key > MIDDLE** value, therefore search will be focused only on the elements between the second **MIDDLE** index to the first **MIDDLE** index.

    MIDDLE = (2 + 2) / 2

    = 2 {**MIDDLE** index}

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

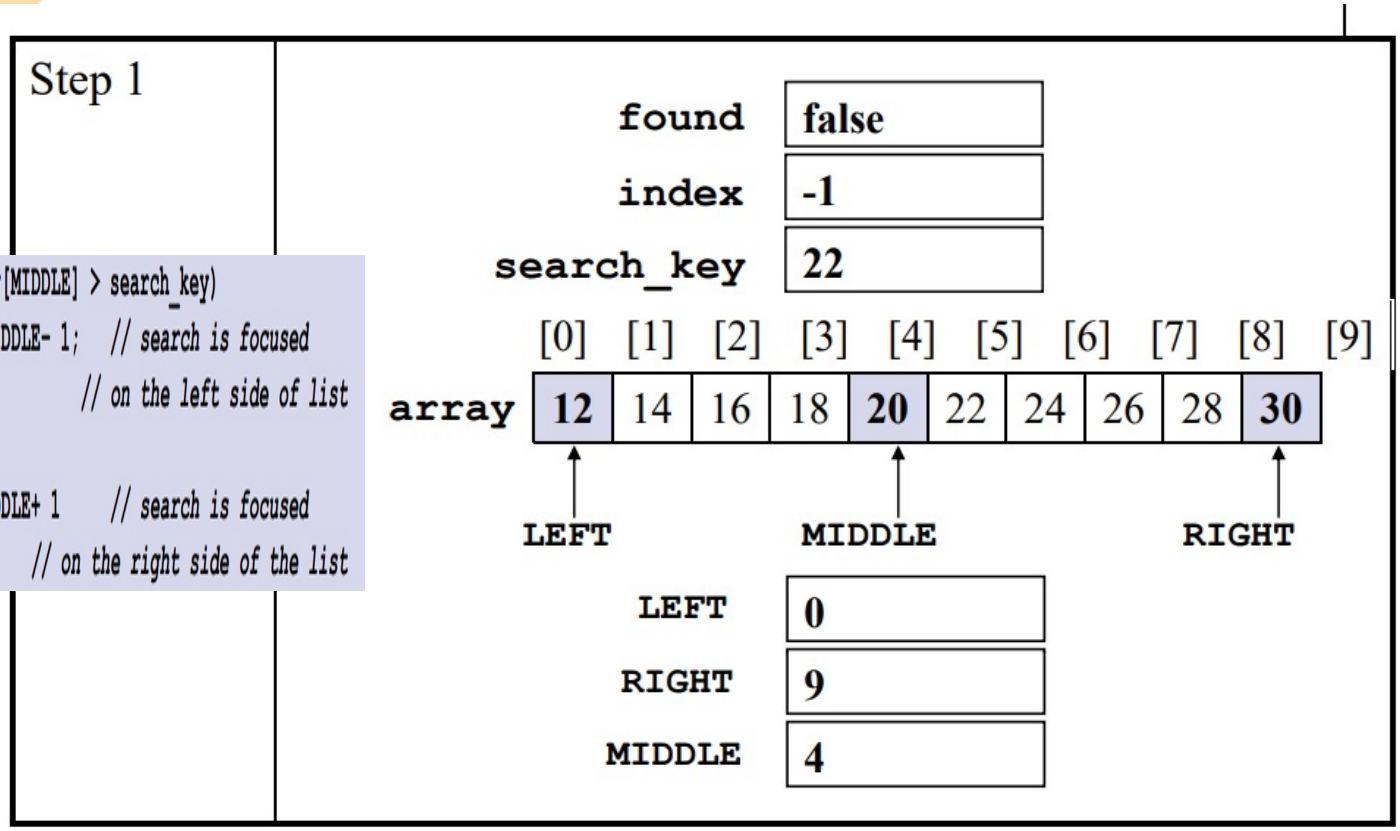
cont...



- Element at the third index, array[2] with the value 33 is not equal to the value of the search key.
- Search process has reached the last element of the traversed subset, therefore search is terminated and assumed fail.
- To search from the list sorted in descending, change operator " **>** " to operator " **<** " to the following statement :

```
else if (array [ MIDDLE ] > search_key)
        RIGHT = MIDDLE - 1;
```

Steps to Execute Binary Search



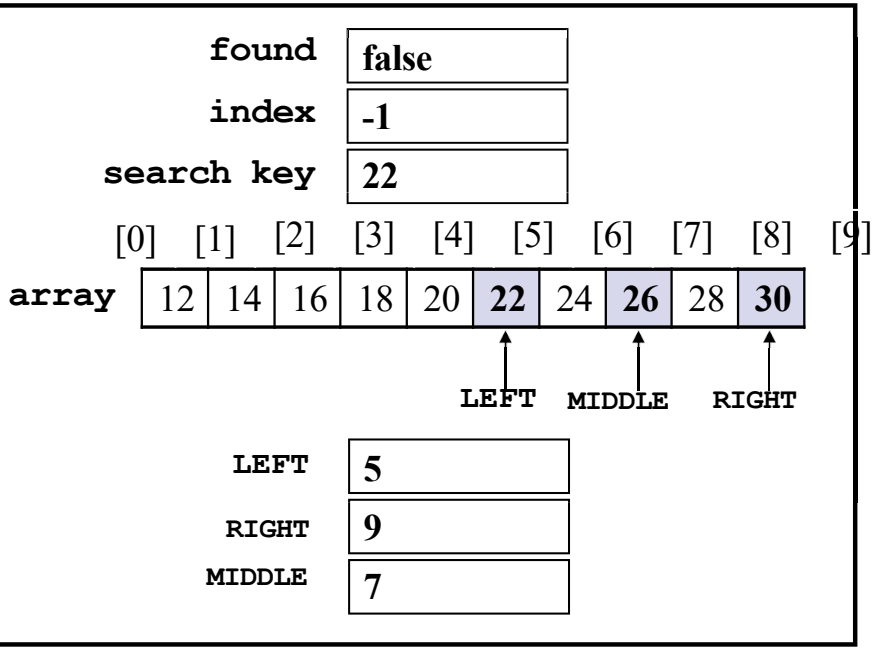

```
else if (array[MIDDLE] > search_key)
    RIGHT= MIDDLE- 1;   // search is focused
                        // on the left side of list
else
    LEFT= MIDDLE+ 1     // search is focused
                        // on the right side of the list
```

Assume: `search_key` = 22, `array_size` = 10

cont...

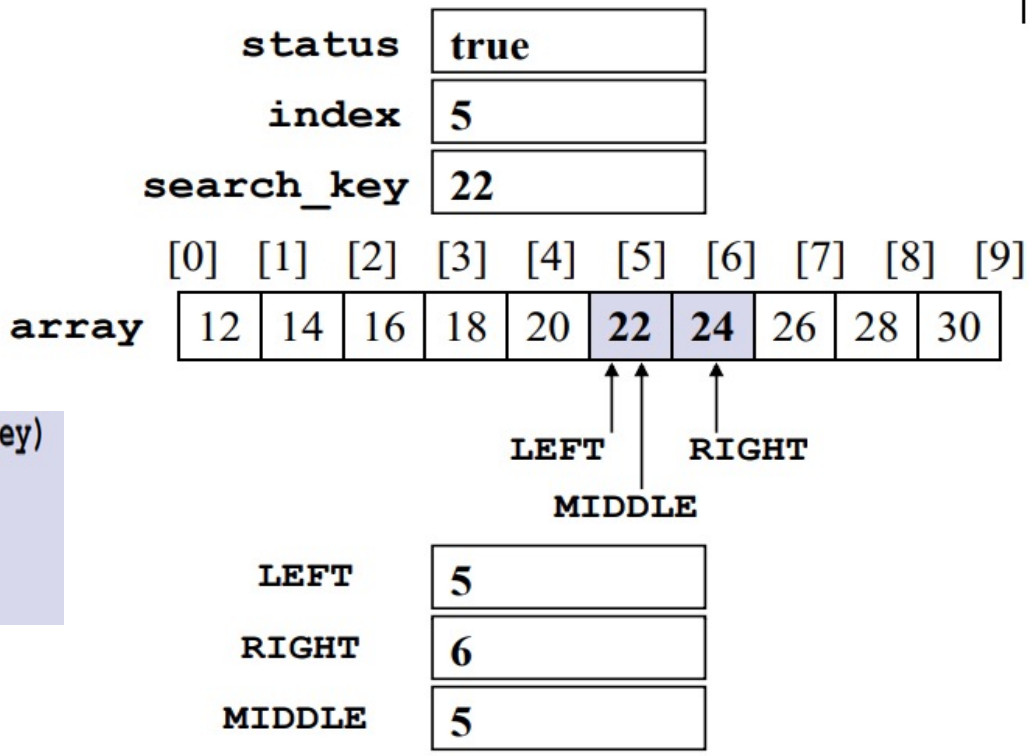
Step 2

```
else if (array[MIDDLE] > search_key)
    RIGHT= MIDDLE- 1;   // search is focused
                        // on the left side of list
else
    LEFT= MIDDLE+ 1     // search is focused
                        // on the right side of the list
```



cont…

Step 3




```
if ( array[MIDDLE] == search_key)
{      index = MIDDLE;
       found = true;
}
```

Binary Search Analysis

- Binary Search starts searching by comparing element in the middle. Thus the searching process start at n/2 for a list size = n.
- If the middle value does not matches with the search key, then the searching area will be reduced to the left or right sublist only. This will reduce the searching area to ½ *n*. From half of the list, the second middle value will be identified. Again, if the middle value does not matches with the search key, the searching area will be reduced to the left or right sub list only. The searching area will reduce ½ (½ *n*).
- The process of looking for middle point and reducing the searching area to the left or right sublist will be repeated until the middle value is equal to the middle value (search key is found) or the last value in sublist has been traverse.
- If the repetition occur *k* times, then at iteration *k* , the searching area is reduced to $(½)^k n$.

Binary Search Analysis

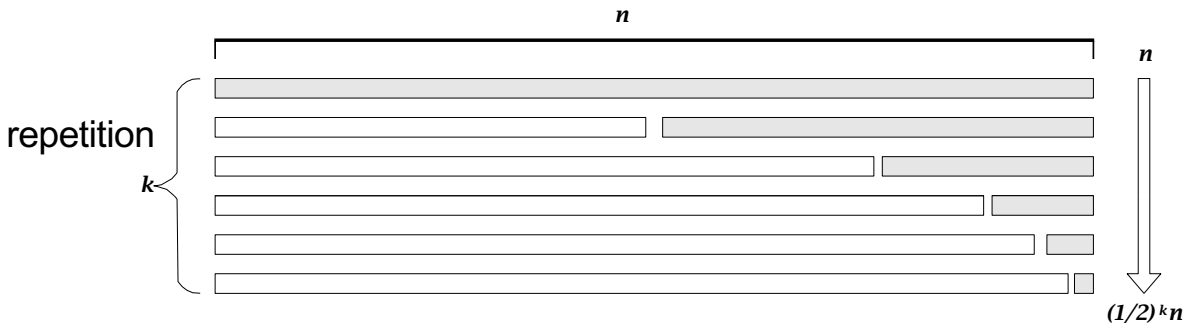


Figure above shows the reducing size for binary search area.

At iteration k for array size = n , searching area will be reduced from n to $(½)^k n$.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Binary Search Analysis

- Best case for binary search happen if the search key is found in the middle array. O(1).
- Worse case for binary search happen if the search key is not in the list or the searching size equal to 1.
- The searching time for worse case is $O(log_2 n)$.

Conclusion

- Searching is a process to locate an element in a list and return the index of the searched element.
- Basic searching techniques : sequential search and binary search.
- Sequential search can be implemented on sorted and unsorted list, while binary search can be implemented only on sorted list.
- Sequential search on sorted data is more efficient than sequential search on unsorted data.
- Binary search is more efficient than sequential search.
- Basic searching techniques explained in this class are only suitable for small sets of data. Hashing and indexing are suitable for searching large sets of data.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE & ALGORITHM

SEARCHING TECHNIQUES 1

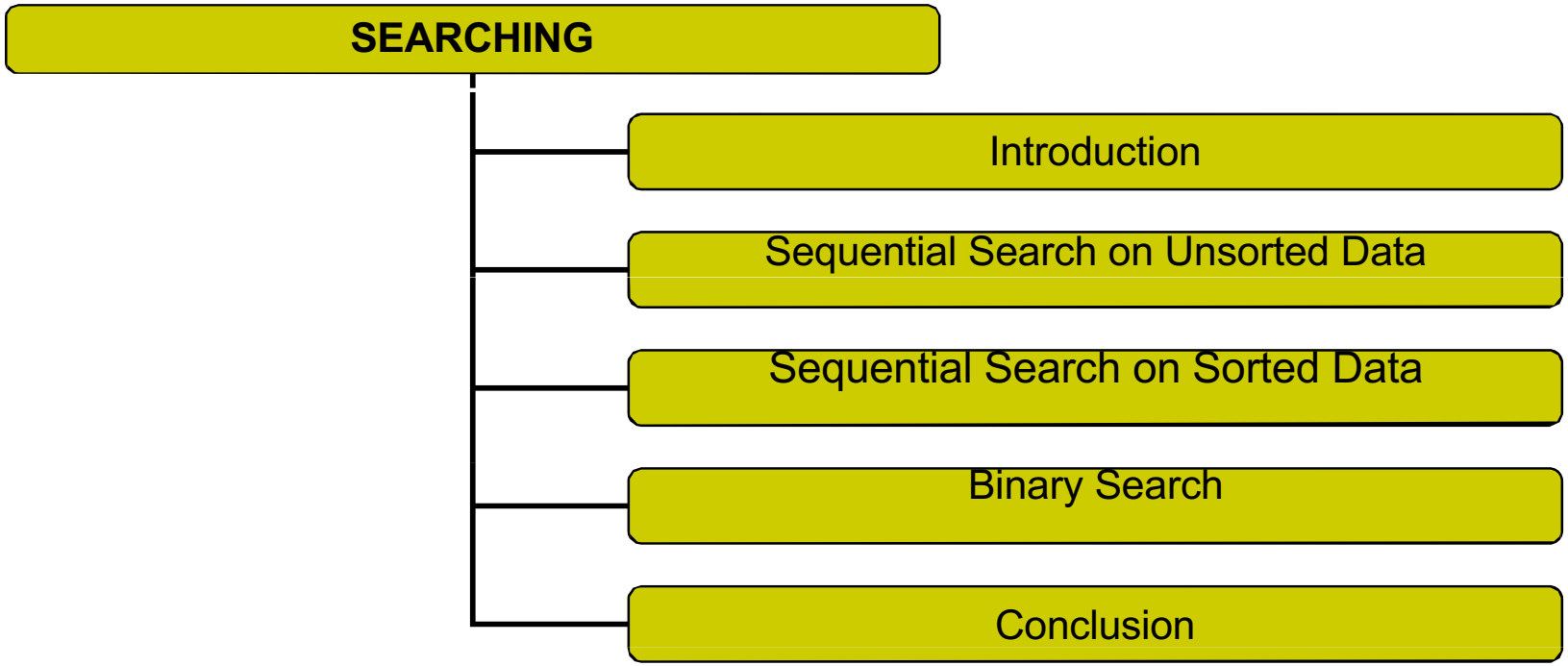
Nor Bahiah Hj Ahmad & Dayang Norhayati A.Jawawi
School of Computing

Objective

At the end of the lecture, students are expected to be able to :

- Understand the searching technique concept and the purpose of searching operation.
- Understand the implementation of basic searching algorithm;
  - Sequential search.
    - Sequential search on unsorted data.
    - Sequential search on sorted data.
  - Binary Search.
- Able to analyze the efficiency of the searching technique.
- Able to implement searching technique in problem solving

Outline


SEARCHING
Introduction
Sequential Search on Unsorted Data
Sequential Search on Sorted Data
Binary Search
Conclusion

Introduction

Searching Definition

- Clifford A. Shaffer[1997] define searching as a process to determine whether an element is a member of a certain data set.
- The process of finding the location of an element with a specific value (key) within a collection of elements
- The process can also be seen as an attempt to search for a certain record in a file
  - Each record contains data field and key field
  - Key field is a group of characters or numbers used as an identifier for each record
  - Searching can done based on the key field.

Example: Employee Record

index	empID	Empl_IC	EmpName	Post
[0]	1111	701111-11-1234	Ahmad Faiz Azhar	Programmer
[1]	**122**	**800202-02-2323**	**Mohd. Azim Mohd. Razi**	**Clerk**
[2]	211	811003-03-3134	Nurina Raidah Abdul Aziz	System Analyst

Searching can be done based on certain field:

empID, or Empl_IC, or EmpName

To search empID = 122, give us the record value at index 1.

To search empID = 211, give us the record value at index 2

5

Introduction

- Among Popular Searching Technique
  - Sequential search *
  - Binary search *
  - **Binary search Tree** **
  - Indexing.

- Similar with sorting, Searching can also be implemented in two cases, internal and external search.

* topic discuss in this chapter

** topic discussed in later chapter

Introduction

- External search – only implemented if searching is done on a very large size of data. Half of the data need to be processed in RAM while half of the data is in the secondary storage.
- Internal search – searching technique that is implemented on a small size of data. All data can be load into RAM while the searching process is conducted.
- This session will only discuss several searching techniques on the data stored in an array.

Basic Sequential Search

- Basic sequential search usually is implemented to search item from unsorted list/ array.
- The technique can be implemented on a small size of list.
- This is because the efficiency of sequential search is low compared to other searching techniques.
- In a sequential search, every element in the array will be examine sequentially, starting from the first element.
- The process will be repeated until the last element of the array or until the searched data is found.

Basic Sequential Search

- Used for searching that involves records stored in the main memory (RAM)
- The simplest search algorithm, but is also the slowest.
- Searching strategy:
  - Examines each element in the array one by one (sequentially) and compares its value with the one being looked for – the search key.
  - Search is successful if the search key matches with the value being compared in the array. Searching process is terminated.
  - Else,if no matches is found, the search process is continued to the last element of the array.
  - Search is failed if there is no matches found from the array.

Basic Sequential Search Function

```
int SequenceSearch(int search_key,
  int array[ ],int   array_size )
{   int p;
    int index =-1;
    //-1 means record is not found
    for ( p = 0; p < array_size; p++ )
    {  if ( search_key == array[p] )
        { indeks = p;
          // assign current array index
          break; //terminate searching
        }
     } //end for
  return index;
  // return location of value
} //end function
```

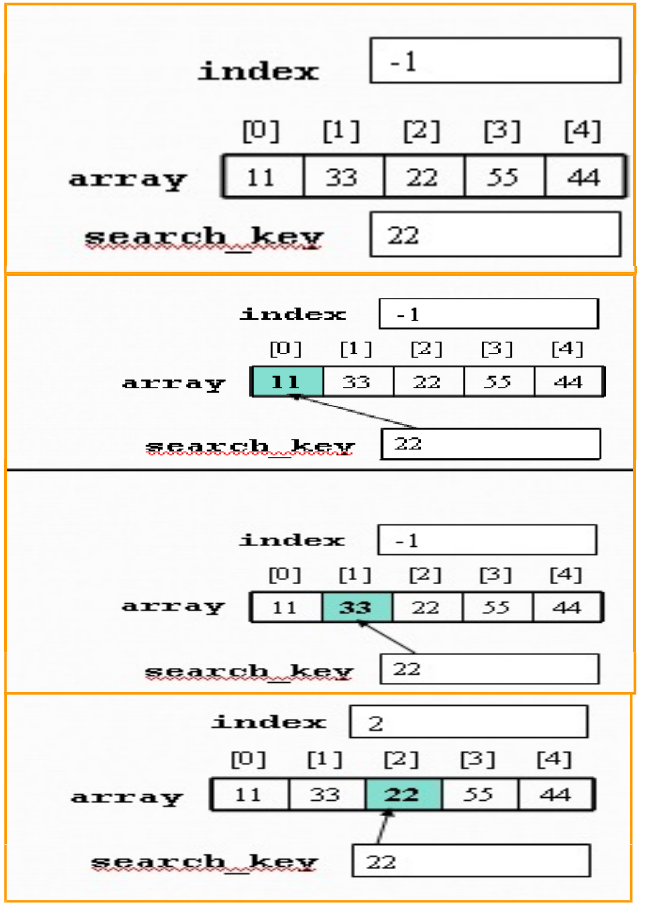
Every element in the array will be examined until the search key is found

or

until the search process has reached the last element of the array.

Basic Sequential Search implementation – Search key = 22

```
int SequenceSearch(int search_key,
  int array[ ],int array_size )
{ int p;
  int index =-1;
  //-1 means record is not found
  for ( p = 0; p < array size; p++ )
  {
    if ( search_key == array[p] )
    { indeks = p;
      // assign current array index
       break; //terminate searching
     }// end if
   } //end for
  return index;
  // return location of value
} //end function
```

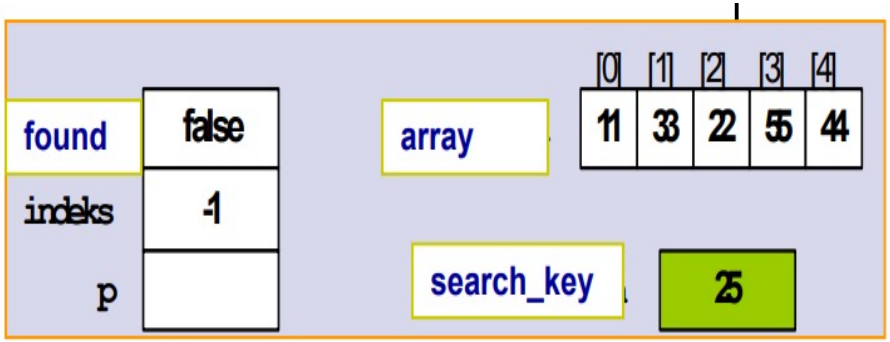


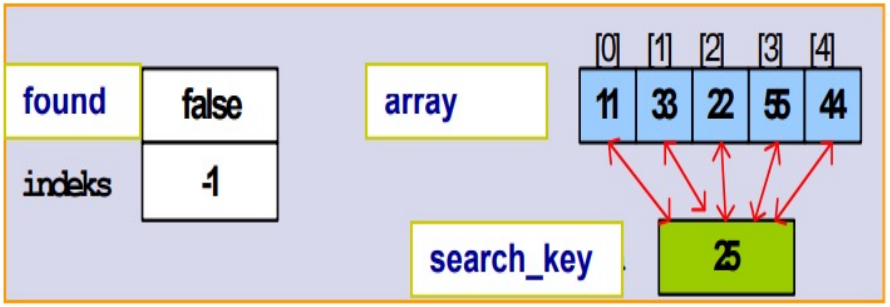
Search for key 22 is successful

p=0,1,2

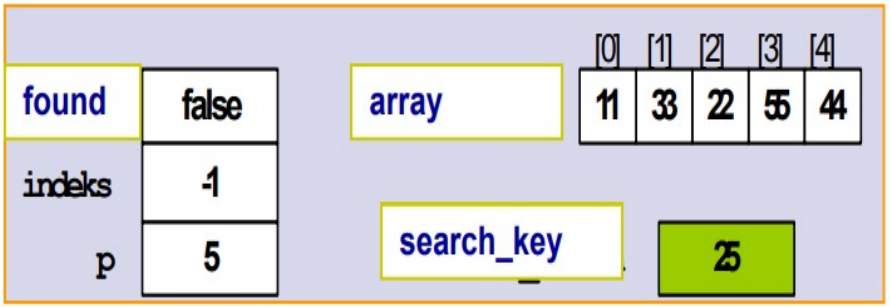
Basic Sequential Search

```
int SequenceSearch(int search_key,
  int array[ ],int array_size )
{ int p;
  int index =-1;
  //-1 means record is not found
  for ( p = 0; p < array_size; p++ )
  {
    if ( search_key == array[p] )
    { indeks = p;
      // assign current array index
       break; //terminate searching
     }// end if
   } //end for
  return index;
  // return location of value
} //end function
```





p=0,1,2,3,4 => search key is not matches



Searching is unsuccessful

Sequential Search Analysis

- Searching time for sequential search is O(n).
- If the searched key is located at the end of the list or the key is not found, then the loop will be repeated based on the number of element in the list, O(n).
- If the list can be found at index 0, then searching time is, O(1).

Improvement of Basic Search Technique

- The efficiency of basic search technique can be improved by searching on a sorted list.
- For searching on ascending list, the search key will be compared one by one until the searched key is found, or
  - until the searched key value is smaller than the item compared in the list. If this situation happen the searching process stopped.

  => This will minimize the searching process.

Sequential Search on Sorted Data

```
int SortedSeqSearch( int search_key,   int array[ ],
                                         int array_size)
{   int p;
    int index = -1;     //-1 means record not found
    for (p = 0; p < array_size; p++)
    {   if (search key < array [p] )
              break;
          // loop terminats when the value of
          // search key is smaller than the current array
          // element
        else if(search_key== array[p]) // key is found
        {    index = p;
             // assign current index
              break;
       } // end else-if
    }//end for
    return index;
    // return the value of index
} //end function
```

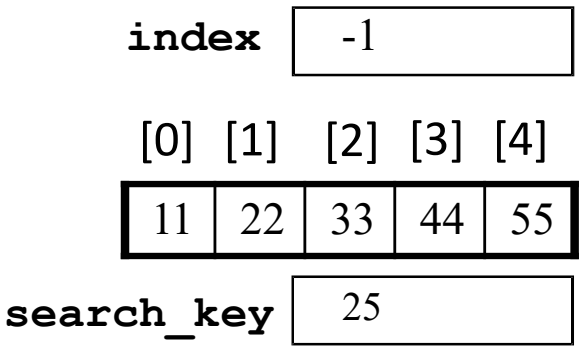


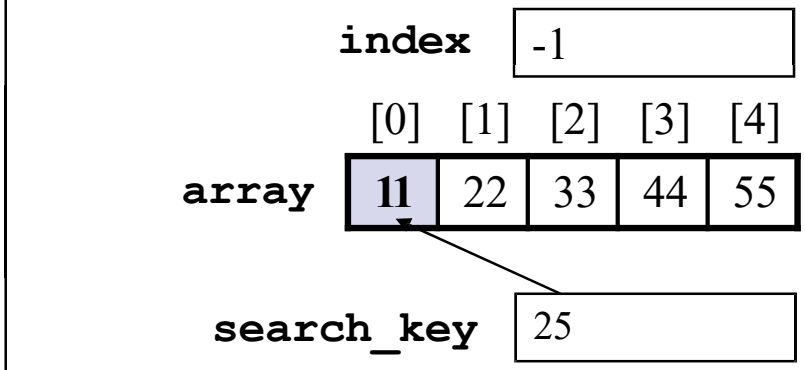
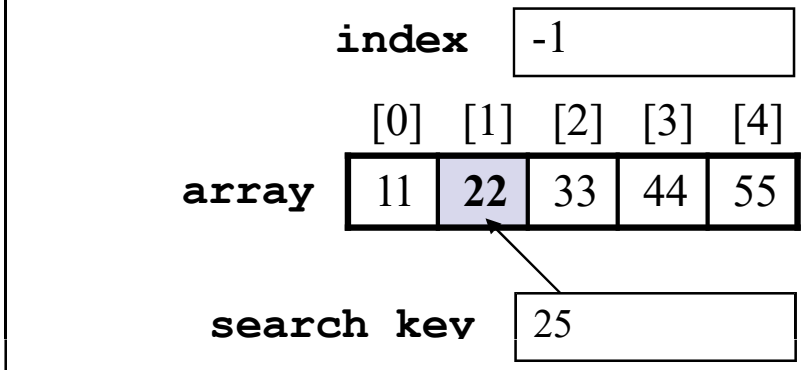
Steps to Execute Sequential Search Function on a Sorted List

Assume:

search_key = 25

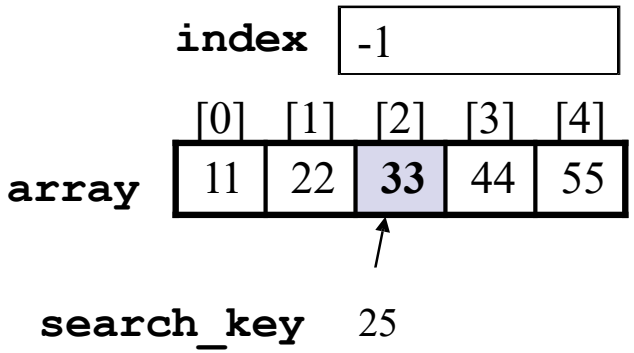
array size = 5

Step 1		Initial value for variable **index** and **array** elements

Step 2		p = 0 **search_key** is compared with the first element in the **array**
Step 3		p = 1 **search_key** is compared with the second element in the **array**

cont...

Step 4



p=2

- **`search_key`** is compared with the third element in the array
- the value of **`search_key`** is smaller than the current element of array
- loop repetition is terminated using "**`break`**" statement

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

CHAPTER 8

STACK

Nor Bahiah Hj Ahmad & Dayang Norhayati A.Jawawi
School of Computing

Objectives

At the end of the lesson students are expected to be able to:

- Understand stack concepts and the structure.
- Understand operations that can be done on stack.
- Understand and know how to implement stack using array and linked list.

Introduction to Stack

- Stack is a collection of items which is organized in a sequential manner.
- Example: stack of books or stack of plates.
- All additions and deletions are restricted at one end, called top.
- LAST IN FIRST OUT (LIFO) data structure.





Example of Stack

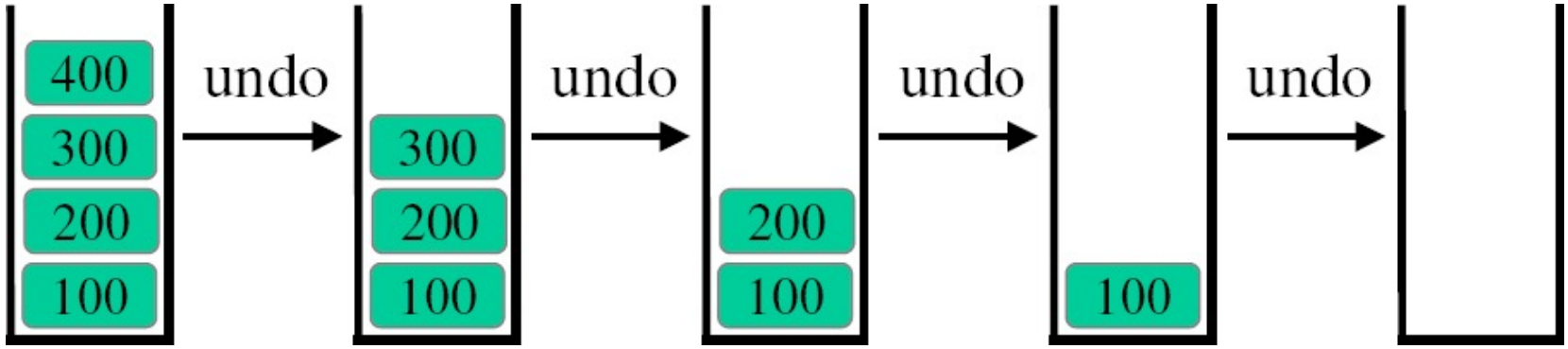
- Long and narrow driveway
  - BMW comes in first, Lexus follows, Benz enters in last.
  - When the cars come out, Benz comes out first, then Lexus follows, and BMW comes out last.
- Reverse a string

  Example: TOP , Reverse: POT
- Brackets balancing in compiler
- Page-visited history in a Web browser
- Undo operation in many editor tools
- Check nesting structure

More Example: *undo* operation

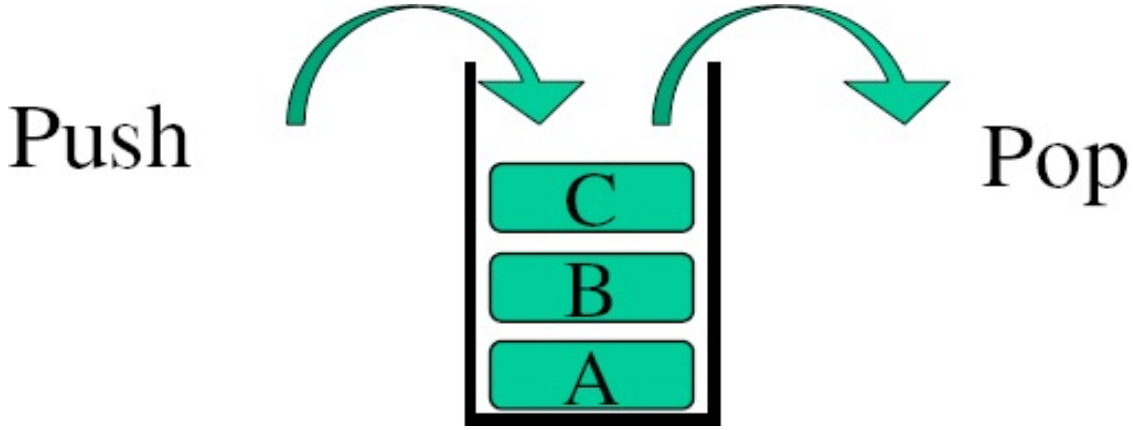
- In an Excel file, input your data in row
  - 100, 200, 300, 400
- Find something wrong, use undo, return to previous state

Operation lists

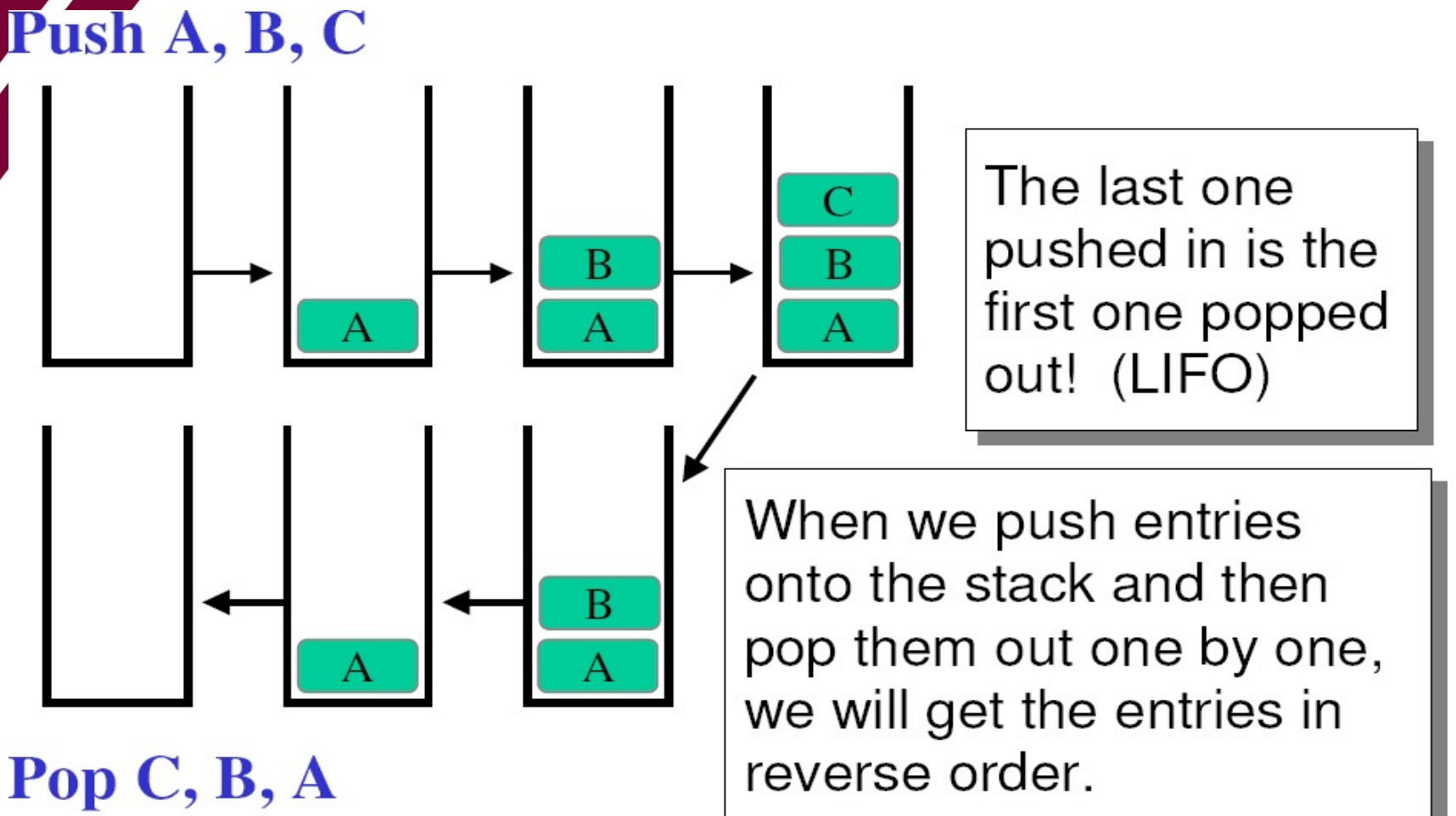


What Is Stack?

- Stack is an abstract data type
- Adding an entry on the top is called push
- Deleting an entry from the top is called pop
- A stack is open at one end only(the top). You can push entry onto the top, or pop the top entry out of the stack.



Last-in First-out (LIFO)



The last one pushed in is the first one popped out! (LIFO)

When we push entries onto the stack and then pop them out one by one, we will get the entries in reverse order.

Stack implementation

- Stack is an abstract data structure
- Item can be Integer, Double, String, and also can be any data type, such as Employee, Student…
- How to implement a general stack for all those types?
- We can implement stack using array or linked list.

Implementation for Stack

Array

- **Size of stack is fixed during declaration**
- Item can be pushed if there is some space available, need isFull( ) operations.
- Need a variable called, top to keep track the top of a stack.
- Stack is empty when the value of Top is –1.

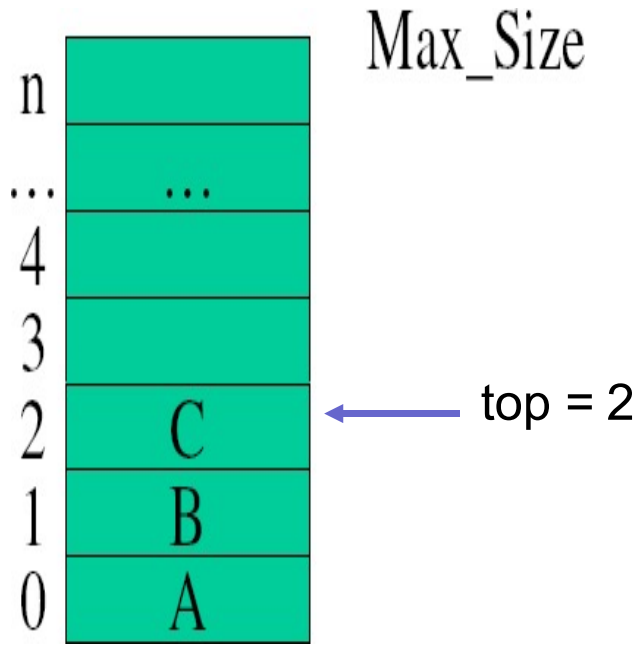
Linked List

- **Size of stack is flexible**. Item can be pushed and popped dynamically.
- Need a pointer, called top to point to top of stack.

Array Implementation of Stack

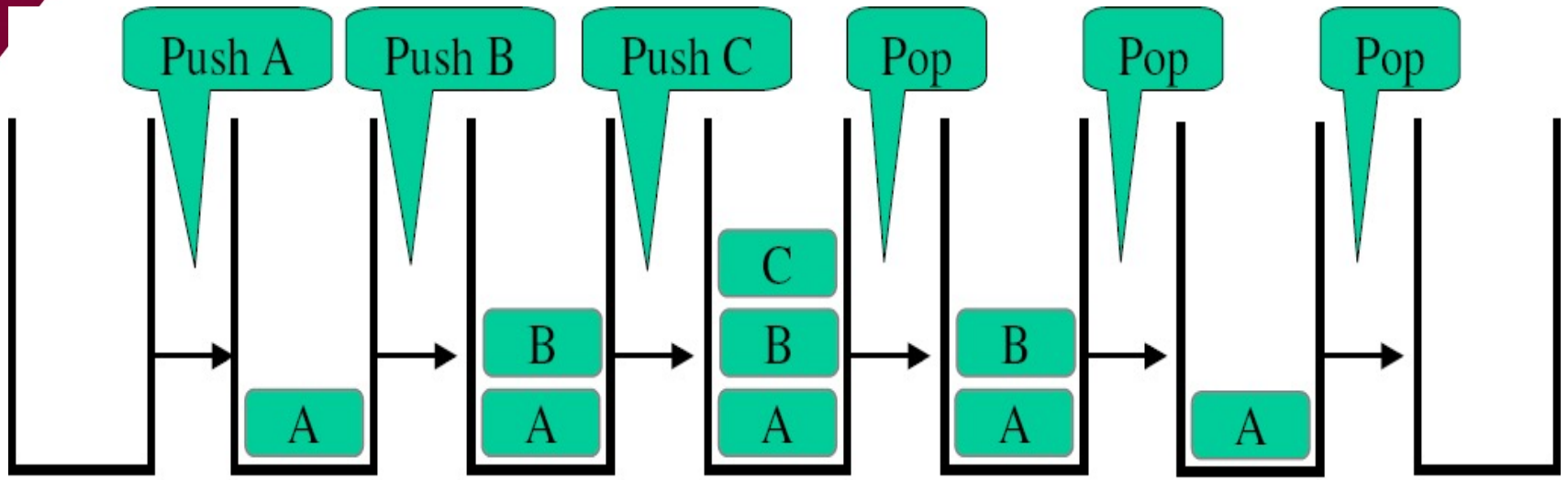
Stack Operations:

- createStack()
- push(item)
- pop( )
- isEmpty( )
- isFull( )
- stackTop()



"Stack can be visualized as array, BUT the operations can be done on top stack only. "

Push() and pop() operations



Top:

-1 0 1 2 1 0 -1

stack empty

stack empty

Array Implementation of Stack

3 things to be considered for stack with array

1. **Stack Empty :** when top is -1.
2. **Push operations:** To insert item into stack

   2 statements must be used

```
top = top + 1;
stack[top] = newitem;
```

3. **pop operations:** To delete item from stack.

   2 statements should be used

```
Item = stack[top]; or stackTop();
top = top – 1;
```

- `Item = stack[top];` statement is needed if we want to check the value to be popped.

Array Implementation of Stack

Stack declaration:

```
const int size = 100;
class stack
{
  private : // data declaration
    int top ;
    char data[size] ;
  public : // function declaration
  void createStack();
  void push(char) ;     // insert operation
  void pop() ;      // delete operation
  char stackTop() ; // get top value
   bool isFull() ; // check if stack is Full
   bool isEmpty(); // check if stack is empty
} ;
```

Array Implementation of Stack

- We need two data attributes for stack:
  1. **Data :** to store item in the stack, in this example data will store char value
  2. **top** : as index for top of stack, integer type
- Size of the array that store component of stack is 100. In this case, stack can store up to 100 char value.
- Declaration of stack instance:

  stack aStack;

Array Implementation of Stack

`createStack()` operation

- Stack will be created by initializing top to -1.
- `createStack()` implementation:

```
void stack:: createStack();
{
        top = -1;
}
```

- Top is **–1 :- means that there is no item being pushed into stack yet.**

Array Implementation of Stack

`isFull()` Operation

- This operation is needed ONLY for implementation of stack using array.
- In an array, size of the array is fixed and to create new item in the array will depend on the space available.
- This operation is needed before any push operation can be implemented on a stack.
- `bool isFull()` implementation

```
bool stack::isFull()
{
     return (top == size-1 );
}
```

- Since the size of the array is 100,
  - `bool isFull()` will return ***true***, If top is 99 (100 – 1).
  - `bool isFull()` will return *false*, if there is some space available, top is less than 100.

Array Implementation of Stack

`bool isEmpty()` operation

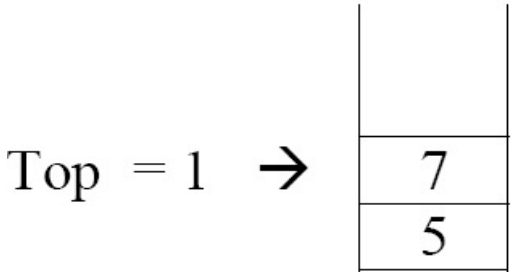
- This operation will check whether the array for stack is empty.
- This operation is needed before ANY pop operation can be done. If the stack is empty, then pop operation cannot be done.
- **`bool isEmpty()`** will return *true* if top –1 and return *false* if top is not equal to -1, showing that the stack has element in it.
- **`bool isEmpty()`** implementation :

```
bool stack::isEmpty()
{
      return ( top == -1 );
}
```

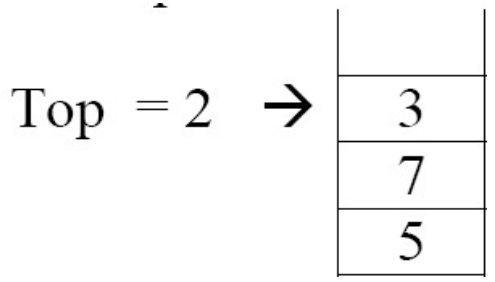
Array Implementation of Stack

`push(newItem)` operation : Insert item onto stack

- **push()** operation will insert an item at the top of stack. This operation can be done only if there is space availbale in the array
- Before any item can be inserted into a stack, isFull() operation must be called first.
- Insertion operation involve the following steps:
  - Top will be increased by 1.
    top = top + 1;
  - New item will be inserted at the top
    data[Top] = newItem;



before push()



after push()

Array Implementation of Stack

```
void stack::push(char newitem)
{
  if (isFull()) // check whether stack is full
     cout << "Sorry,Cannot push item.
              Stack is now full!"<< endl;
  else
  {  top = top + 1       // Top point to next index
     data[top] = newitem; //assign new item at top
   }//end else
}//end push()
```

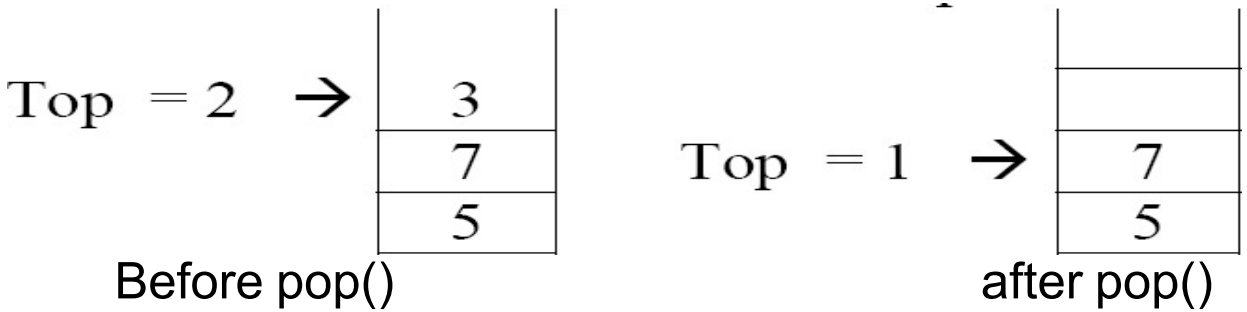
Top will be increased first before item is inserted in order to avoid inserting item at the current top value.

Array Implementation of Stack

`pop()` Operation

- This operation will delete an item at top of scak.
- Function `isEmpty()` will be called first in orderto ensure that there is item in a stack to be deleted.
- `pop()` operation will decrease the value of top by 1:

```
top = top - 1;
```



Before pop()

after pop()

Array Implementation of Stack

```
void stack::pop()
{
   char item;
   if ( isEmpty() )
        cout << "Sorry, Cannot pop item.
                        Stack is empty!" << endl;
   else
   { //display value at top to be deleted
        cout << "Popped value :" << data[top];
        top = top - 1;
         // top will hold to new index
   }// end if
}//end pop
```

Array Implementation of Stack

`stackTop()` operation : to get value at the top

```
char stackTop()
{ //function to get top value
   if (isEmpty())
      cout <<"Sorry, stack is empty!"<< endl;
   else
      return data[top];
} // end stackTop
```

Linked List Implementation of Stack

- Stack implemented using linked list – number of elements in stack is not restricted to any size.
- Dynamic memory creation, memory will be assigned to stack when a new node is pushed into stack, and memory will be released when an element being popped from the stack.
- Stack using linked list implementation can be empty or contains a series of nodes.
- Each node in a stack must contain at least 2 attributes:
- i) **data** – to store information in the stack.
- ii) pointer next (store address of the next node in the stack

Linked List Implementation of Stack

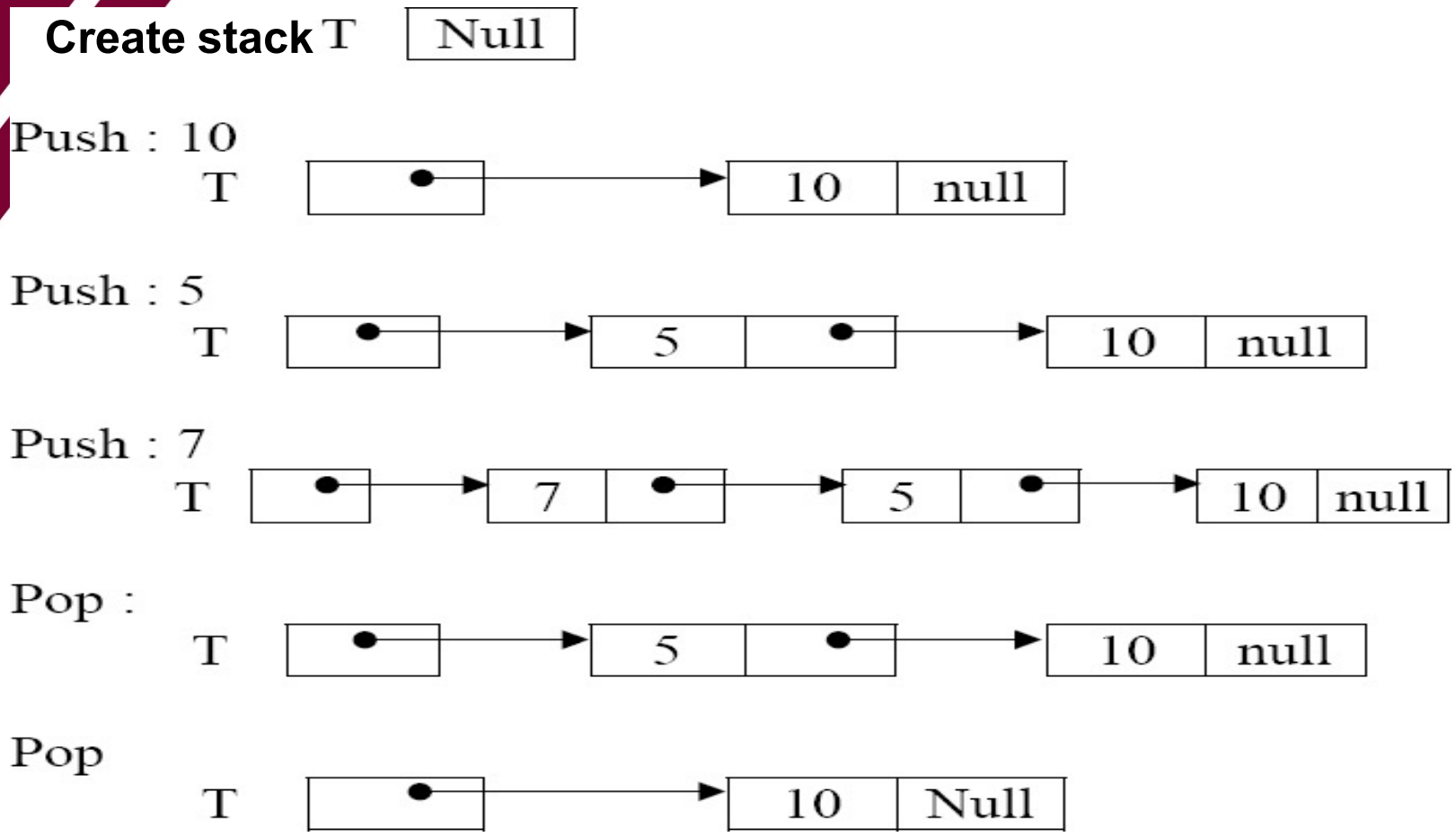
Basic operations for a stack implemented using linked list:

- **`createStack()`** – initialize top
- **`push(char)`** – insert item onto stack
- **`pop()`** – delete item from stack
- **`isEmpty()`** – check whether a stack is empty.
- **`stackTop()`** – get item at top

`isFull()` operation is not needed since elements can be inserted into stack without limitation to the stack size.

- Push and pop operations can only be done at the top ~ similar to add and delete in front of the linked list.

Linked List Implementation of Stack: push() and pop() operations



Linked List Implementation of Stack

```
class nodeStack
{
   int data;
   nodeStack *next;
};
class stack
{
   private: // pengisytiharan ahli data
      nodeStack *top;
   public : // pengisytiharan ahli fungsi
      void createStack(); // set Top to NULL
      void push(int) ; // insert item into stack
      void pop() ; // delete item from stack
      int stackTop() ; // get content at top stack
      bool isEmpty(); // check whether stack is empty
};
```

Create Stack and isEmpty()

- Creating a stack will initialize top to NULL - showing that currently, there is no node in the stack.

```
void stack::createStack()
{
    top = NULL;
}
```

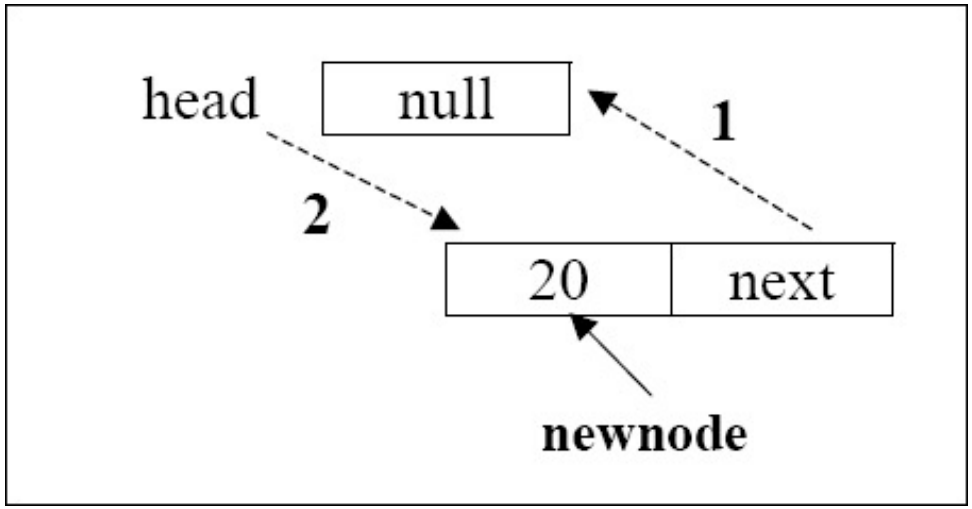
- Is Empty() stack will return true if stack is empty, top is NULL.

```
bool stack::isEmpty()
{
 return (top == NULL);
}
```

`push()` operations

- 2 conditions for inserting element in stack:
  - Insert to empty stack.
  - Insert item to non empty stack : stack with value.

`push()` to empty stack



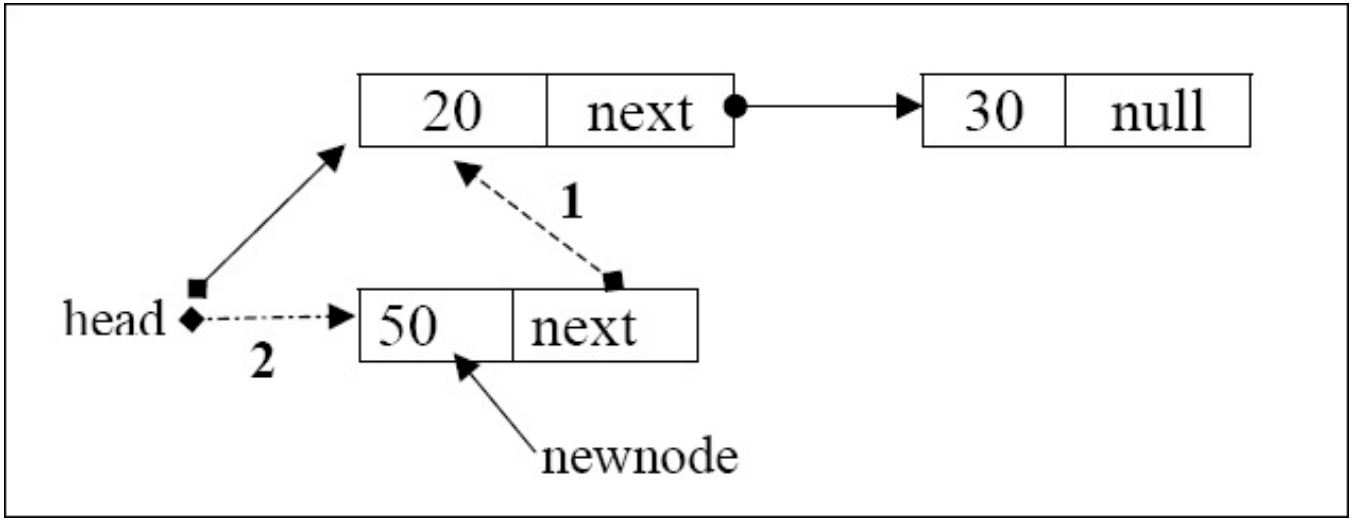
In this situation the new node being inserted, will become the first item in stack.

STEP 1 : newnode-> next = head;

STEP 2 : head = newnode;

`push()` to non-empty stack

This operation is similar to inserting element in front of a linked list. The next value for the new element will point to the top of stack and head will point to the new element.



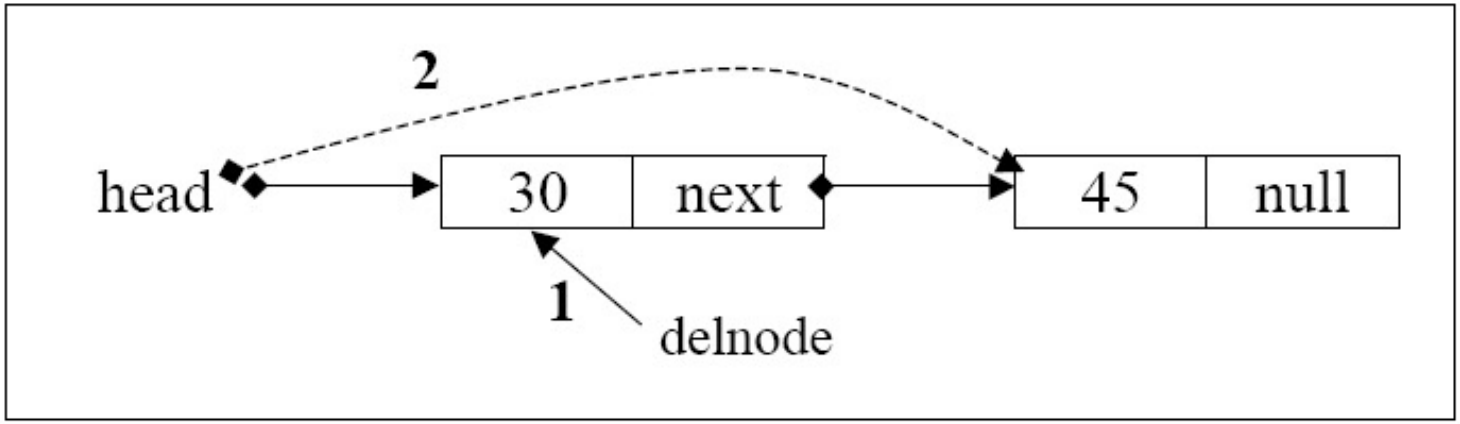
STEP 1 : newnode-> next = head;
STEP 2 : head = newnode;

Push() operations

```
void stack::push(int newitem)
{ // create newnode
  nodeStack *newnode;
  newnode = new (nodeStack);
  if( newnode == NULL)
     cout << "Cannot allocate memory…" << endl;
  else // add to empty stack, or to front stack
  { newnode->data = newitem;
    newnode->next = head;
    head = newnode;
  }// end if
} //end push operation
```

Delete item from stack (pop)

- Pop operation can only be done to non-empty stack. Before pop() operation can be done, operation must be called in order to check whether the stack is empty or there is item in the stack. If **`isEmpty()`** function return true, pop() operation cannot be done.
- During pop() operation, an external pointer is needed to point to the delete node. In the figure below, **`delnode`** is the pointer variable to point to the node that is going to be deleted.



STEP 1 : delnode = head;
STEP 2 : head = delnode -> next; or **head = head->next;**
STEP 3 : delete(delnode);

Pop() operation

```
void stack::pop()
{  nodeStack *delnode;
   if (isEmpty())
     cout <<"Sorry, Cannot pop item from
          stack.Stack is still empty!" <<endl;
   else
   { delnode =head;
     cout << "Item to be popped from stack
                 is: " << stackTop()<<endl;
      head = delnode->next;
      delete(delnode);
    }// end else
} // end pop
```

Check item at top stack

```
int stack::stackTop()
{   if (isEmpty())
       cout <<"Sorry,Stack is still empty!"
            <<endl;
    else
        return head->data;
} // end check item at top
```

End of Stack Implementation

What we have learned so far….

- Stack is a LIFO data structure
- Can be implemented using array and link list
- Structure of a stack using array and link list
- Basic Operation for a stack
  - `createStack(),push(),pop()`
  - `stackTop(),isEmpty(),isFull()`



Next…..Stack Application

Stack Application Examples

- Check whether parantheses are balanced (open and closed parantheses are properly paired)
- Evaluate Algebraic expressions.
- Creating simple Calculator
- Backtracking (example. Find the way out when lost in a place)

Example1: Parantheses Balance

- Stack can be used to recognize a balanced parentheses.
- Examples of balanced parentheses.

  (a+b), (a/b+c), a/((b-c)*d)

  Open and closed parentheses are properly paired.
- Examples of not balance parentheses.

  ((a+b)*2 and m*(n+(k/2)))

  Open and closed parentheses are not properly paired.

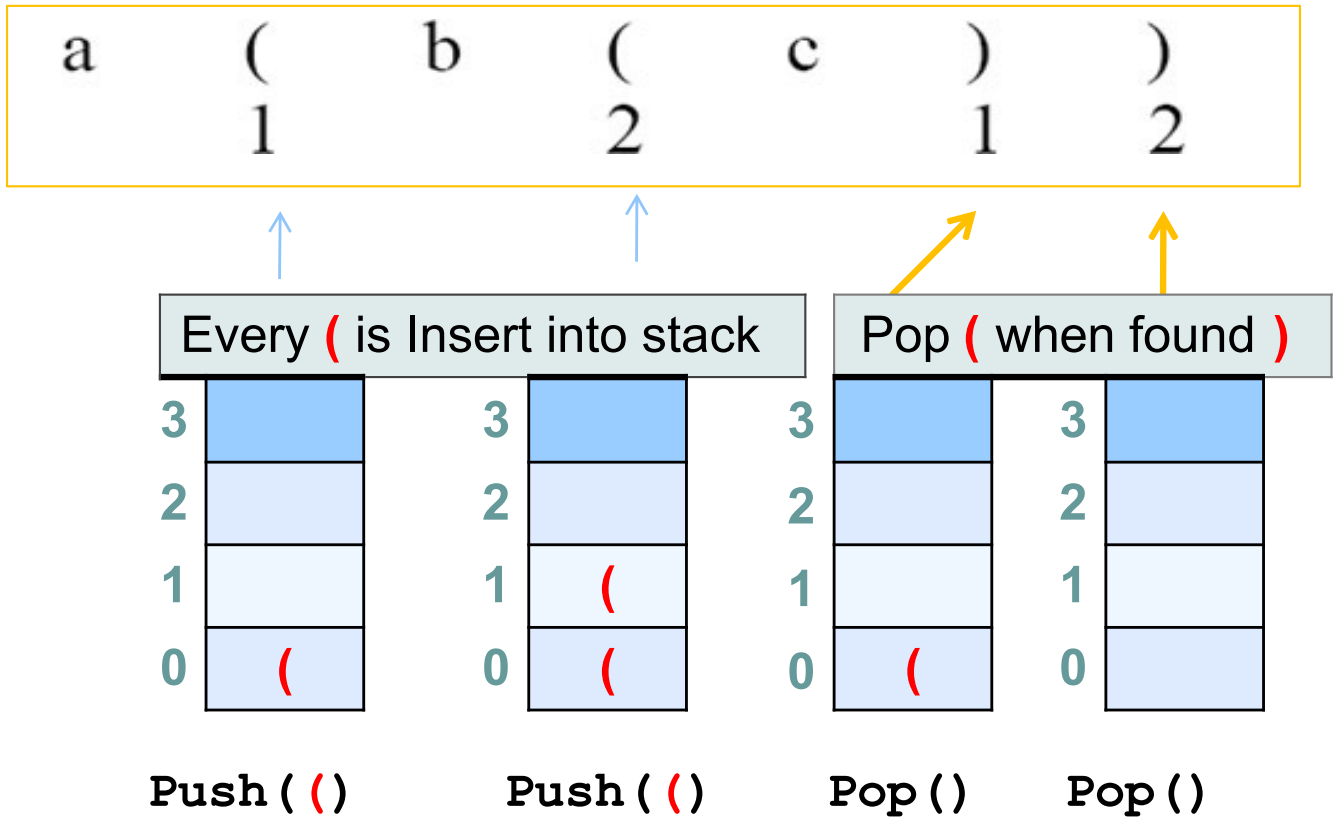
Check for Balanced Parantheses Algorithm

```
create (stack);
continue = true;
while ( not end of input string) && continue
{     ch = getch( );
      if ch = '(' || ch = ')'
        { if ch = '('
             Push(stack, '(');
          else if IsEmpty(stack)
            continue = false;
          else
             Pop(s);
         } // end if
} // end while
if ( end of input && isEmpty(stack);
     cout << "Balanced.." << end1;
else
     cout << "Not Balanced.. " << endl;
```

Check for Balanced Parantheses Algorithm

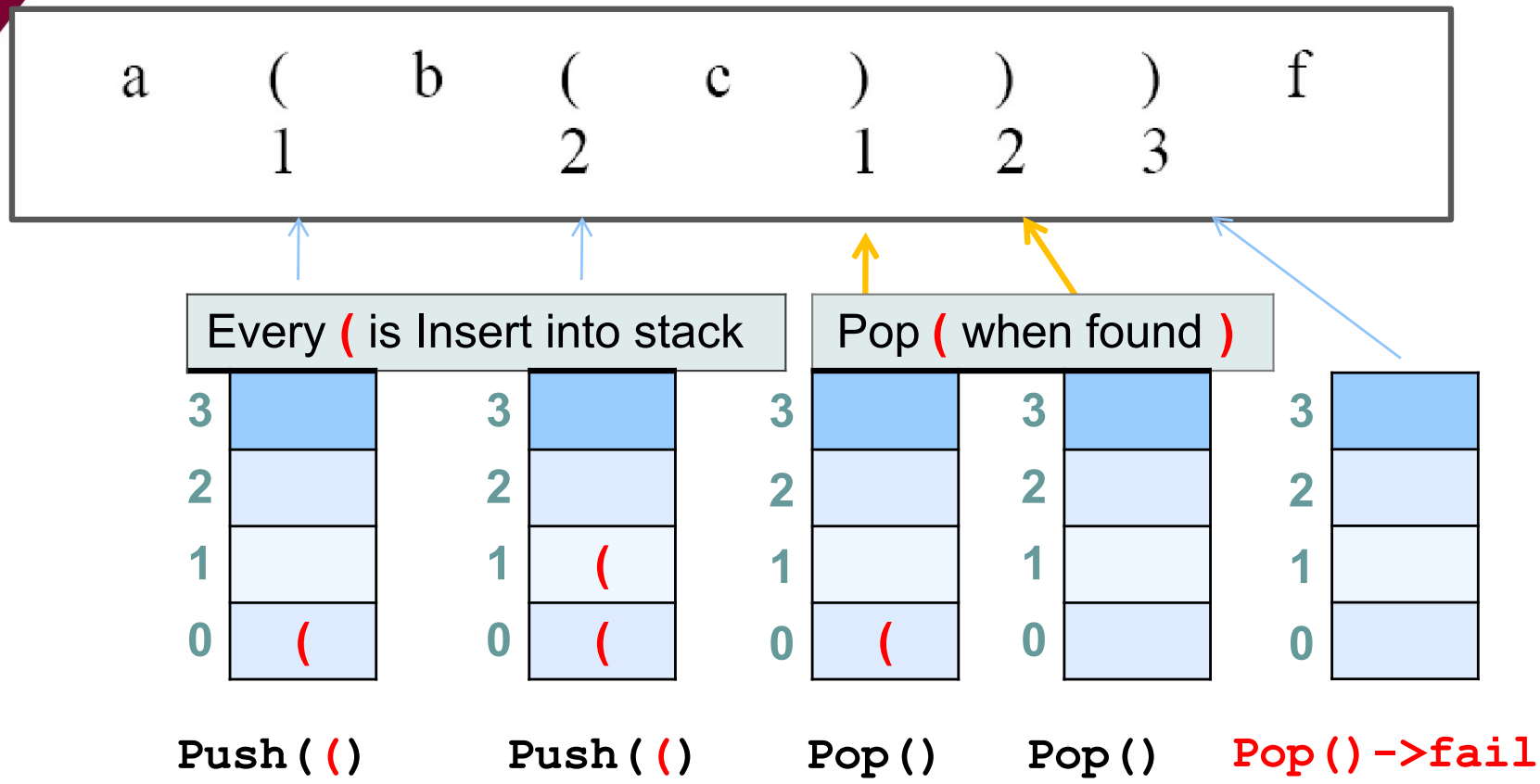
- Every '(' read from a string will be pushed into stack.
- The open parentheses '(' will be popped from a stack whenever the closed parentheses ')' is read from string.
- An expression have balanced parentheses if :
  - Each time a ")" is encountered it matches a previously encountered "(".
  - When reaching the end of the string, every "(" is matched and stack is finally empty.
- An expression does NOT have balanced parentheses if :
  - When there is still ')' in input string, the stack is already empty.
  - When end of string is reached, there is still '(' in stack.

Example for Balance Parantheses



Expression **a(b(c))** have balance parentheses since when end of string is found the stack is empty.

Example for ImBalanced Parantheses



Expression **a(b(c))) f** does not have balance parentheses => the third) encountered does not has its match, the stack is empty.

Algebraic expression

- One of the compiler's task is to evaluate algebraic expression.
- Example of assignment statement:

  y = x + z * (w / x + z * (7 + 6))

- Compiler must determine whether the right expression is a syntactically legal algebraic expression before evaluation can be done on the expression.
- 3 algebraic expressions are :

  Infix, prefix and postfix

Infix Expression

- The algebraic expression commonly used is infix.
- The term infix indicates that every binary operators appears between its operands.
- Example

A	+	B
operan	operator	operan

- Example 2: A + B * C



A + (B * C)
(a + b) * c

- To evaluate infix expression, the following rules were applied:
  1. Precedence rules.
  2. Association rules (associate from *left to right*)
  3. Parentheses rules.

Prefix and Postfix Expressions

- Alternatives to infix expression
- Prefix : Operator appears before its operand.
- Example:

```
+ a b
+ a * b c
* + a b c
```

- Postfix : Operator appears after its operand.
- Example:

```
a b +
a b c * +
a b + c *
```

Infix, prefix and postfix

Infix	Prefix	Postfix
a + b	+ a b	a b +
a + (b * c)	+ a * b c	a b c * +
(a + b) * c	* + a b c	a b + c *

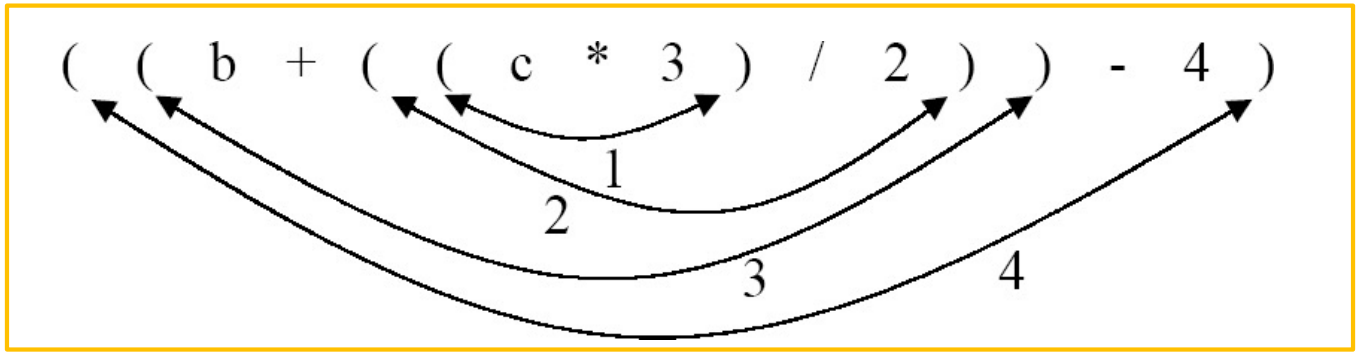
The advantage of using prefix and postfix is that we don't need to use precedence rules, associative rules and parentheses when evaluating an expression.

Converting Infix to Prefix

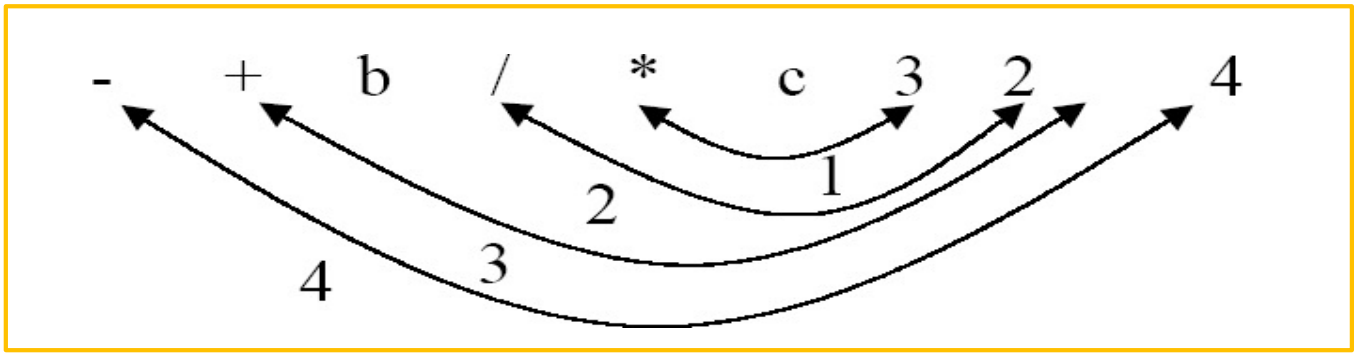
Steps to convert infix expression to prefix:

b + c * 3 / 2 - 4

STEP 1 : Determine the precedence.



STEP 2



Another Example:

1. a + b = (a + b)
   = + a b

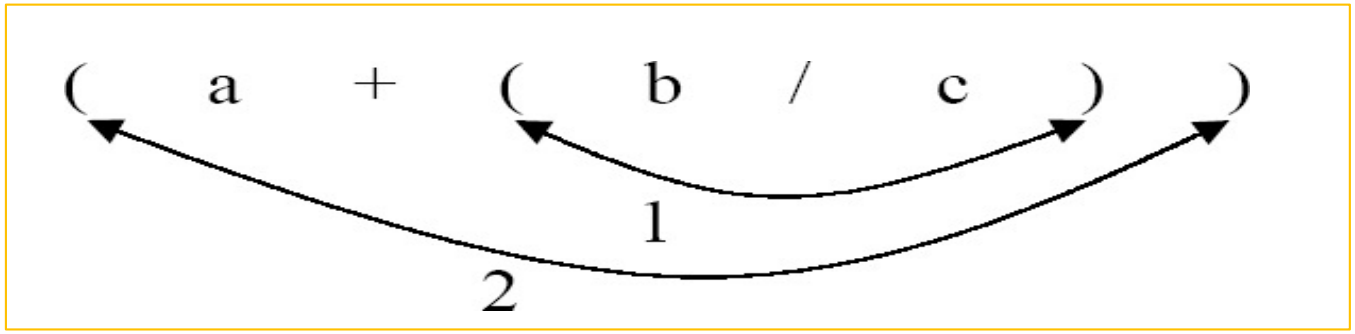
2. a + (b * c) = (a + (b * c))
   = + a * b c

Converting Infix to Postfix

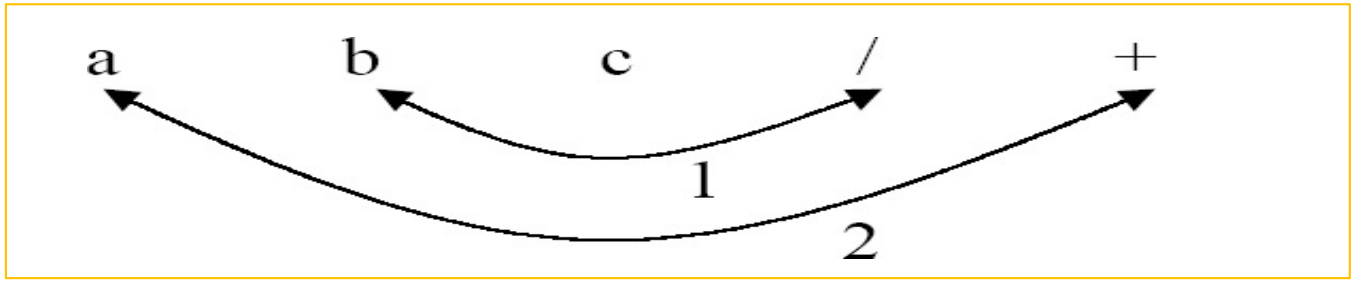
Steps to convert infix expression to postfix:

a + b / c

STEP 1



STEP 2



Converting Infix to Postfix

1. $a + b = (a + b)$
   $= a\ b\ +$

2. $a + b * c = (a + (b * c))$
   $= a\ b\ c\ *\ +$

3. $a + b * (c - d) / (p - r)$
   $= a + (b * (c - d)) / (p - r)$
   $= (a + ((b * (c - d)) / (p - r)))$
   $= a\ b\ c\ d\ -\ *\ p\ r\ -\ /\ +$

Evaluating Postfix Expression

- Postfix expression can be evaluated easily using stack.
- Stack operations, such as push(), pop() and isEmpty() will be used to solve this problem.
- Steps to evaluate postfix expression :

1. Convert infix to postfix expression.
2. Evaluate postfix using stack.

Converting Infix to Postfix Algorithm

```
create(s);  push(s, '#');
while (not end of infix input)
    {  ch = getch  // get next input character
       if (ch is an operan)
         add ch to postfix notation;
       if (ch = '(')
         push(s, ch)
       if (ch = ')')
       { ch = pop(s);
         while (ch != '(')
         {    add ch to postfix  notation;
              ch = pop(s);
         }
       }
```

Converting Infix to Postfix Algorithm

```
if (ch is an operator)
{    while (!isEmpty(s) &&
           (precedence(stacktop()) >= precedence(ch)))
     {   chpop = pop(s);
         add chpop to postfix notation;
      }
     push(s, ch);
     }
} // end while
while (stacktop() != '#')
{   ch = pop(s);
    add ch to postfix notation;
}// end while
```

Converting Infix to Postfix

A + B * C – D / E

infix	stack	postfix
A + B * C – D / E	#	
+ B * C – D / E	#	A
B * C – D / E	# +	A
* C – D / E	# +	A B
C – D / E	# + *	A B C
– D / E	# + *	A B C * +
D / E	# -	A B C * + D
/ E	# -	A B C * + D
E	# - /	A B C * + D E
	# - /	A B C * + D E
	#	A B C * + D E / -

Converting Infix to Postfix :

A * B – (C + D)+ E

INFIX	STACK	POSTFIX
A * B – (C + D) + E	#	
* B – (C + D) + E	#	A
B – (C + D) + E	# *	A
– (C + D) + E	# *	A B
(C + D) + E	# -	A B *
C + D) + E	# - (	A B *
+ D) + E	# - (	A B * C
D) + E	# - (+	A B * C
) + E	# - (+	A B * C D
+ E	# -	A B * C D +
E	# +	A B * C D + -
	# +	A B * C D + - E
	#	A B * C D + - E +

Steps to Evaluate Postfix Expression

1. If char read from postfix expression is an operand, push operand to stack.
2. If char read from postfix expression is an operator, pop the first 2 operand in stack and implement the expression using the following operations:
   - **pop(opr1) dan pop(opr2)**
   - **result = opr2 operator opr1**
3. Push the result of the evaluation to stack.
4. Repeat steps 1 to steps 3 until end of postfix expression

- Finally, At the end of the operation, only one value left in the stack. The value is the result of postfix evaluation.

Evaluating Postfix Expression

```
create ( s )
while ( not end of postfix notation )
{    ch = getch;
     if ( ch is operand )
          push ( ch );  // insert ch to stack
     else // if ch = operator
    {     operan1 = pop(  );
           operan2 = pop( );
           result = operan2 ch operan1;
           push ( result);
     } // end else
}// end while
pop ( result );
```

Evaluating Postfix Expression : 2 4 6 + *

Postfix	Ch	Opr	Opn1	Opn2	Result	Stack
2 4 6 + *						
4 6 + *	2					2
6 + *	4					2 4
+ *	6					2 4 6
*	+	+	6	4	10	2 10
	*	*	10	2	20	20

Evaluating Postfix Expression : 2 7 * 18 - 6 +

postfix	Ch	Opr	Opn1	Opn2	result	stack
2 7 * 18 – 6 +						
7 * 18 – 6 +	2					2
* 18 – 6 +	7					2 7
18 – 6 +	*	*	7	2	14	14
– 6 +	18					14 18
6 +	–	–	18	14	-4	-4
+	6					-4 6
	+	+	6	-4	2	2

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Thank You

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE & ALGORITHM

Chapter 09

Queue

Part 2 -Queue Implementation Linked List

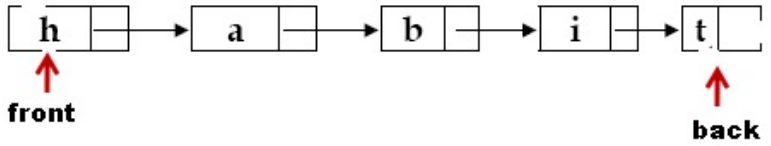
Nor Bahiah Hj Ahmad & Dayang Norhayati A.Jawawi
School of Computing

Queue Implementation Linked List

Pointer-Based Implementation

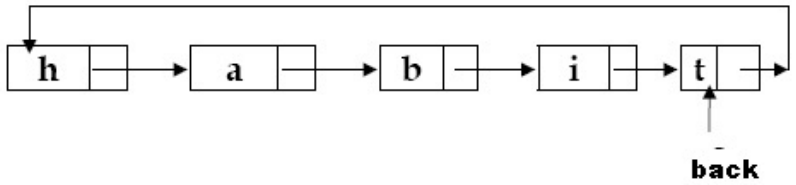
- More straightforward than array-based
- Possible options:
  - Linear linked list

    Two external pointer (Front & Back)



  - Circular linked list

    One pointer will be enough (Back)



Queue Implementation Linked List

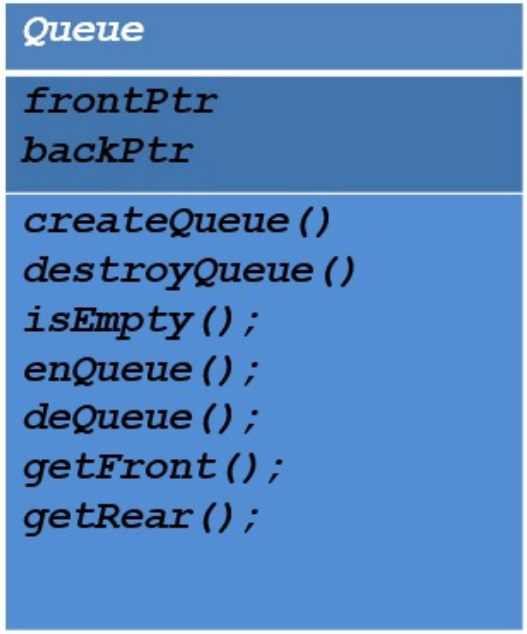
Need 2 structure

- Declaration of the node

```
struct nodeQ {
        char item;
        nodeQ * next;
}
```

- Declaration of the queue

```
class queue
{public:
        nodeQ *backPtr, * frontPtr;
        // operations for queue
};
```



Queue Implementation Linked List

createQueue()

```
backPtr = Null; frontPtr = NULL;
```

destroyQueue()

Destroy the whole nodes in the queue

```
nodeQ *temp = frontPtr;
while (temp){
frontPtr = temp->next;
delete temp; temp=frontPtr; }
```

isEmpty()

```
backPtr == Null && frontPtr == NULL
```

Insert to a linear queue

- Inserting a new node at the Back needs 3 pointer changes

  Change Next pointer in the new node
  Change the Next pointer in the Back node
  Change the External pointer

- Special case:
  - If the queue is empty

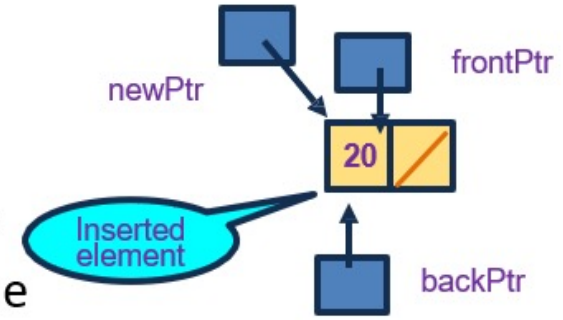
Queue Implementation Linked List

Linear linked list with 2 external pointers

1. Create a new node -> newPtr
2. Insert to an empty queue

   newPtr -> next = NULL

   `frontPtr=backPtr = newPtr`

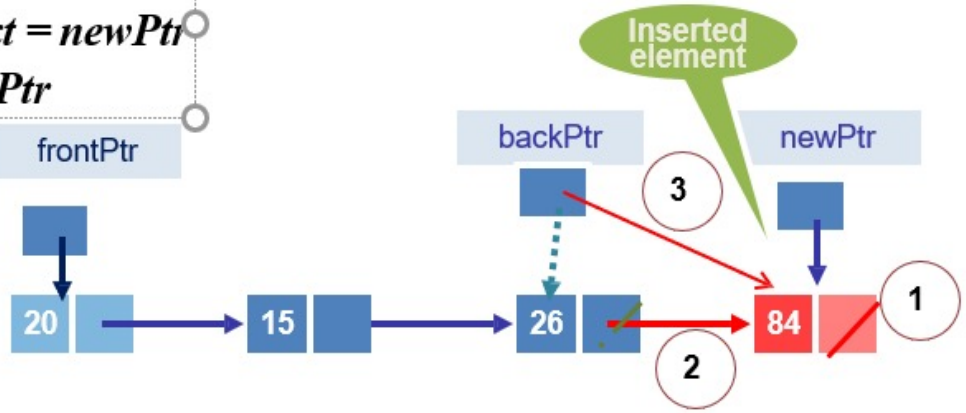


3. Insertion to a non-empty queue

1 *newPtr -> next = NULL*

2 *backPtr -> next = newPtr*

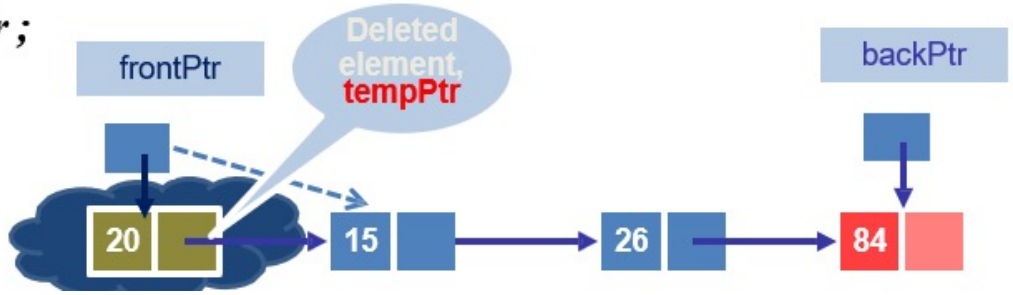
3 *backPtr = newPtr*



Delete from Linear queue

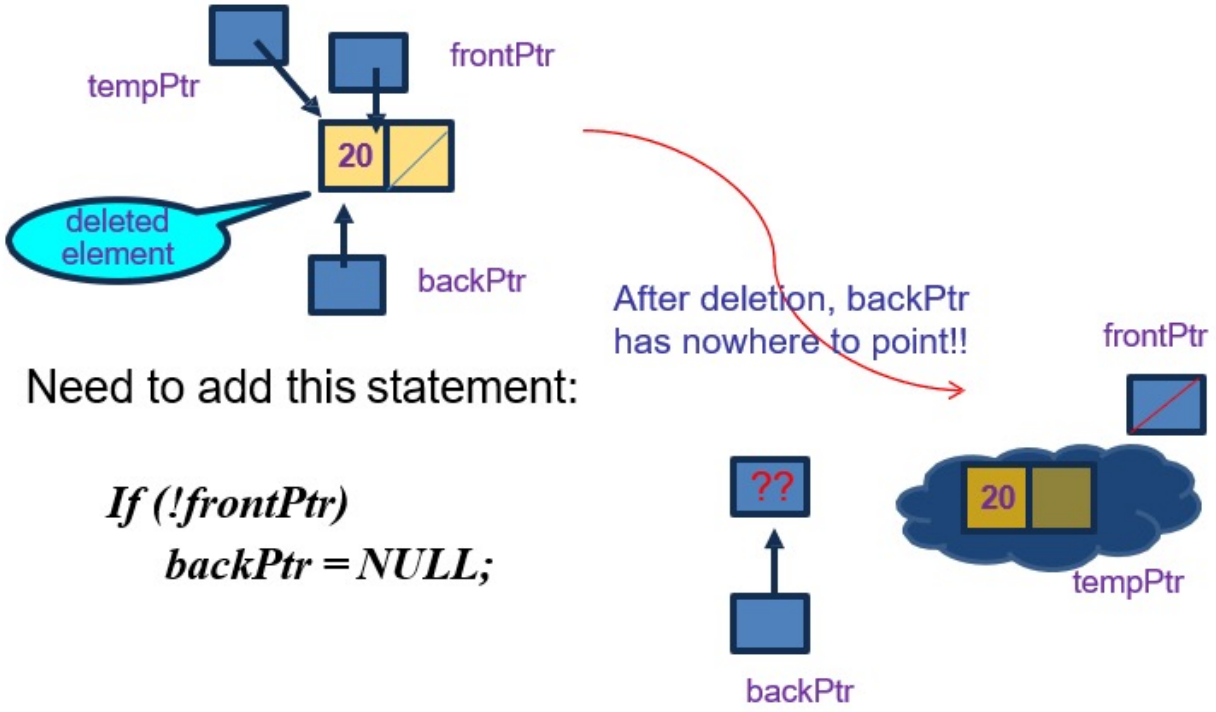
- Deletion
  - Delete from the Front
  - Only one pointer change is needed
  - Special case:
    - If the queue contains one item only
- Deletion Code

```
tempPtr = frontPtr
frontPtr = frontPtr -> next
tempPtr -> next = NULL
delete tempPtr ;
```



Delete from Linear queue

If the queue contains one item only,



Need to add this statement:

```
If (!frontPtr)
    backPtr = NULL;
```

Circular Queue Implementation

Circular linear linked list with one external pointer

– Insertion

- Into an empty queue

  NewPtr -> Next = NewPtr

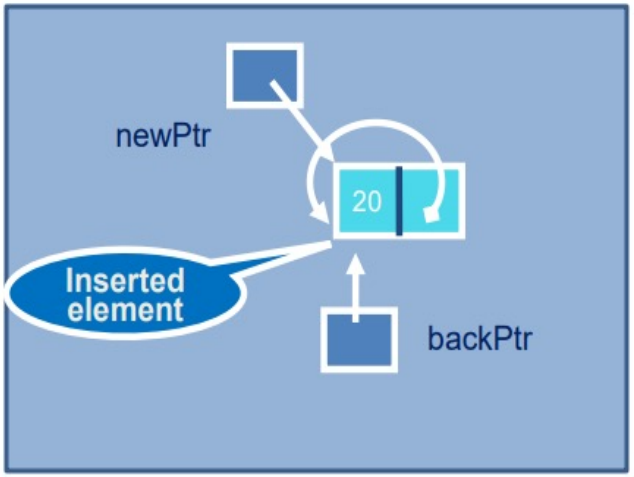
  BackPtr = NewPtr

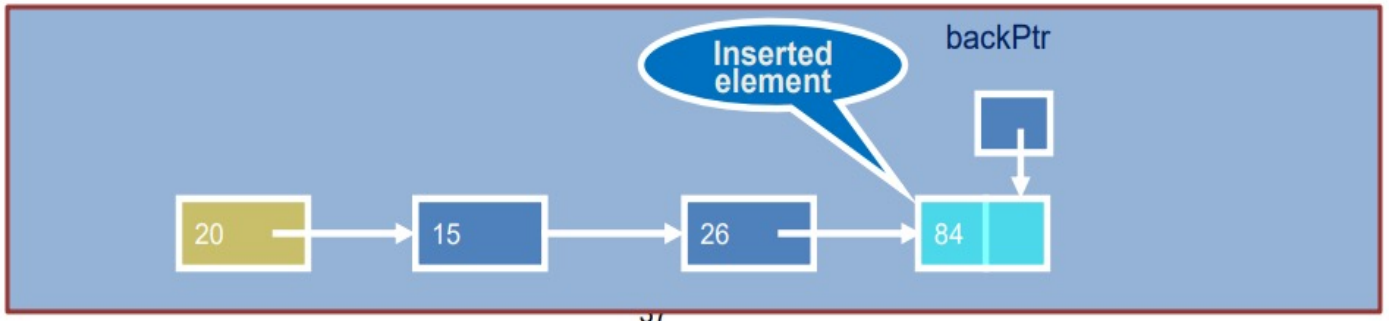
- Into a non-empty queue

  NewPtr -> Next = BackPtr-> Next

  BackPtr -> Next = NewPtr

  BackPtr = NewPtr



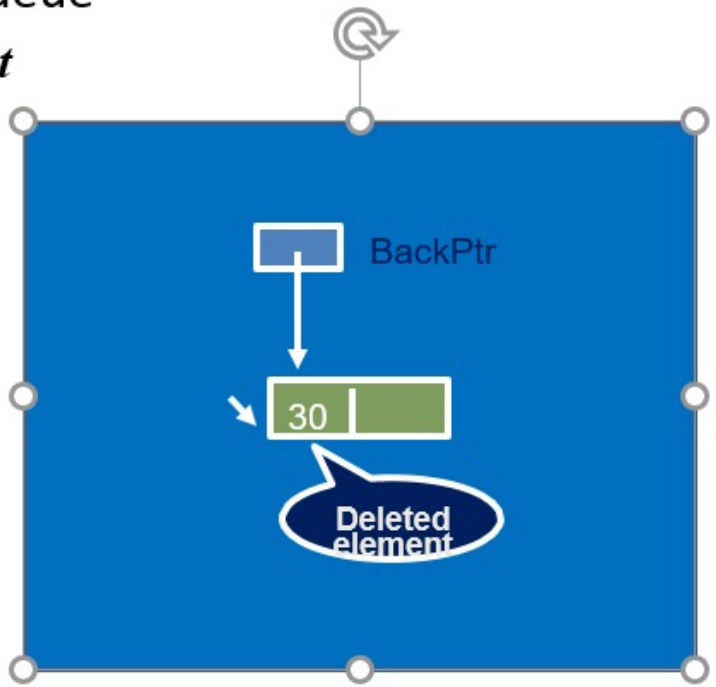


Circular Queue Implementation

Deletion

- From a one-node (one item) queue

```
deletePtr  = BackPtr -> Next
If (deletePtr = BackPtr)
        BackPtr = NULL
delete deletePtr
```



Circular Queue Implementation

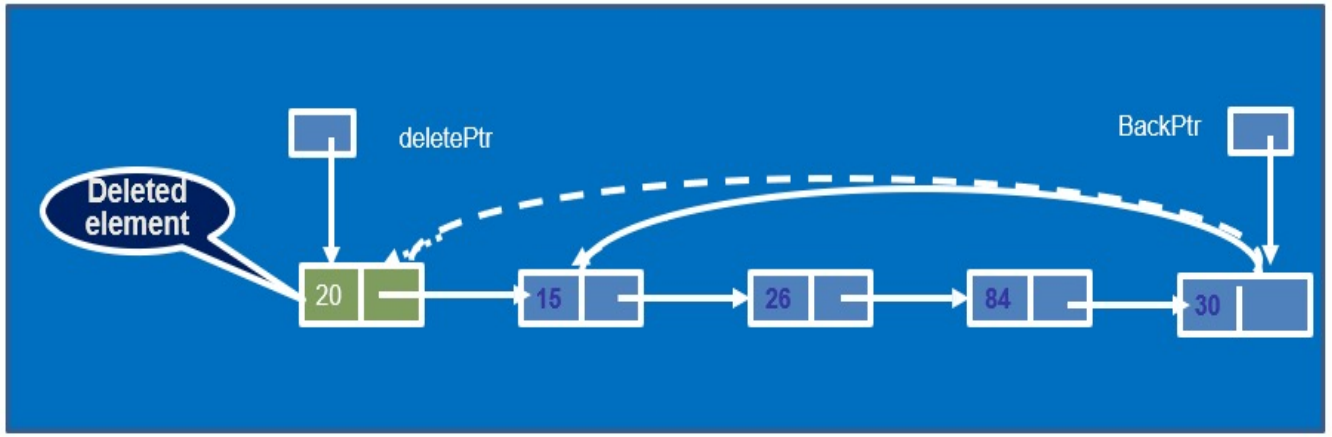
Deletion

- From a non-empty, more than one item queue

deletePtr = BackPtr -> Next

BackPtr -> Next = deletePtr -> Next

delete deletePtr



Comparing Implementations

- Fixed size versus dynamic size
  - A statically allocated array
    - Prevents the *enqueue* operation from adding an item to the queue if the array is full
  - A resizable array or a reference-based implementation
    - Does not impose this restriction on the *enqueue* operation
- Pointer-based implementations
  - A linked list implementation
    - More efficient

Comparing Implementations

- Pointer-Based
  - More complicated than ADT List
  - Most flexible No size restrictions
- Array-Based
  - No overhead of pointer manipulation
  - Prevents adding elements if the array is full

Summary of Position-Oriented ADTs

Stacks

- Operations are defined in terms of position of data items
- Position is restricted to the Top of the stack. Only one end position can be accessed
- Operations:
  - *create:*
    - Creates an empty ADT of the Stack type
  - *isEmpty:*
    - Determines whether an item exists in the ADT
  - *push:*
    - Inserts a new item into the Top position
  - *pop:*
    - Deletes an item from the Top position
  - *peek:*
    - Retrieves the item from the Top position

Summary of Position-Oriented ADTs

Queues

- Operations are defined in terms of position of data items
- Position is restricted to the front & back of the queue. Only the end positions can be accessed
- Operations:
  - *create:*
    - Creates an empty ADT of the Queue type
  - *isEmpty:*
    - Determines whether an item exists in the ADT
  - *enqueue:*
    - Inserts a new item in the Back position
  - *dequeue:*
    - Deletes an item from the Front position
  - *peek:*
    - Retrieves the item from the Front position

Summary of Position-Oriented ADTs

- Stacks and queues are very similar
  - Operations of stacks and queues can be paired off as
    - *createStack* and *createQueue*
    - Stack *isEmpty* and queue *isEmpty*
    - *push* and *enqueue*
    - *pop* and *dequeue*
    - Stack *getTop* and queue *getFront*

References

- Frank M. Carano, Janet J Prichard. "*Data Abstraction and problem solving with C++" Walls and Mirrors*. 5th edition (2007). Addision Wesley.
- Nor Bahiah et al. "*Struktur data & algoritma menggunakan C++". Penerbit UTM. 2005.*

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE & ALGORITHM

Chapter 07 -

Linked List

Nor Bahiah Hj Ahmad & Dayang Norhayati A.Jawawi
School of Computing

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Objectives

At the end of the class, students are expected to :

- Describe linear list concepts using array and linked list.
- Able to lists variations of linked list and basic operations of linked lists.
- Able to explain in detail the implementation and operations of link lists using pointers.
- Able to write program that can implement linked list concept.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Introduction

Lists Definition

- Lists is a group of objects which is organized in sequence.
- List categories: linear list and nonlinear list.
- Linear list – a list in which the data is organized in sequence, example: array, linked list, stack and queue
- Non-Linear list – a list in which the data is stored not sequence, example: tree and graph

Introduction to Linear List

- **Array and linked** lists are linear lists that doesn't have any restrictions while implementing operations such as, insertion, deletion and accessing data in the lists.
- The operations can be done in any parts of the lists, either in the front lists, in the middle or at the back of the lists.

- **Stack and queue** is a linear lists that has restrictions while implementing its operations.
  - **Stack** – to insert, delete and access data can only be done at the top of the lists.
  - **Queue** - Insert data in a queue can be done at the back of the lists while to delete data from a queue can only be done at the front list.

Introduction to Linear List

Pelajar

Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]	Durrani Nukman	Kej. Elektrik	3
[3]	Mohd Saufi	Sains Pendidikan	2
[4]	Nafisah Nordin	Sains Komputer	1
[5]	Safinatun Najah	Pengurusan Komputer	3
[6]			
[7]			

- An array named Pelajar which contains attributes nama pelajar, kursus and tahun Pelajar.
- The array is sorted and can only be accessed based on the index or subscript of the array.
- Example: to access information for a student named Mohd Saufi, we can use:

  Pelajar[3].Nama, Pelajar[3].Kursus dan Pelajar[3].Tahun.

Introduction to Linear List



- Linked lists which contain several nodes which is sorted in ascendeng order.
- Each node contains at least
  - A piece of data (any type)
  - Pointer to the next node in the list
- Need pointer variable *Head [Senarai]*: to point to the first node

Introduction to Linear List

- Basic operations for linear lists:
  - Insert new data in the lists.
  - Delete data from a lists.
  - Update data in the list.
  - Sort data in the lists and
  - Find data in the list.

Array as linear list

- Sized is fixed during array declaration.
- Data insertion is limited to array size
- In order to insert data, need to check whether the array is full or not. If the array is full, the insertion cannot be done.

Array as linear list

- Data in the array can be accessed at random using the index of the array.
- Example, if we access Nama, Kursus and Tahun for Pelajar[3] information of Pelajar[3], Mohd Saufi taking Sains Pendidikan course and in year 2 will be given.
- By random access, accessing data in an array can be done faster.

Pelajar

Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]	Durrani Nukman	Kej. Elektrik	3
[3]	Mohd Saufi	Sains Pendidikan	2
[4]	Nafisah Nordin	Sains Komputer	1
[5]	Safinatun Najah	Pengurusan Komputer	3
[6]			
[7]			

Array Implementation … the drawbacks

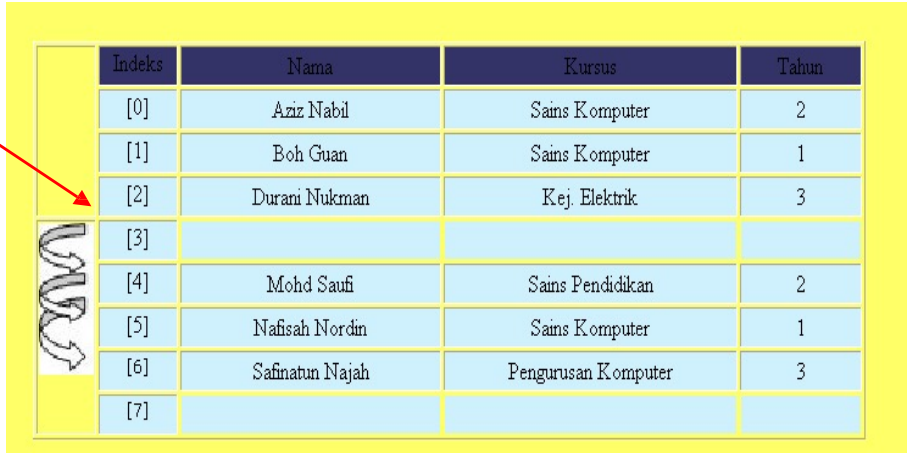
- Requires an estimate of the maximum size of the list
  - waste space
- printList and find: linear access
- findKth: constant
- insert and delete: slow
  - insert at position 0 (making a new element)
    - requires first pushing the entire array down one spot to make room
  - delete at position 0
    - requires shifting all the elements in the list up one
  - On average, half of the lists needs to be moved for either operation

Array Implementation … the drawbacks

- Need space to insert item in the middle of the list.
- Insert Fatimah Adam in between students named Durrani Nukman and Mohd Saufi.
- insert at index 3 : requires first pushing the entire array from index 3 down one spot to make room

Pelajar

Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]	Durrani Nukman	Kej. Elektrik	3
[3]	Mohd Saufi	Sains Pendidikan	2
[4]	Nafisah Nordin	Sains Komputer	1
[5]	Safinatun Najah	Pengurusan Komputer	3
[6]			
[7]			



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Array Implementation … the drawbacks

Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]	Durani Nukman	Kej. Elektrik	3
[3]	Fatimah Adam	Kej. Awam	2
[4]	Mohd Saufi	Sains Pendidikan	2
[5]	Nafisah Nordin	Sains Komputer	1
[6]	Safinatun Najah	Pengurusan Kompuer	3
[7]			

New item is inserted at index 3, after shifting the data from index 3 onwards. .

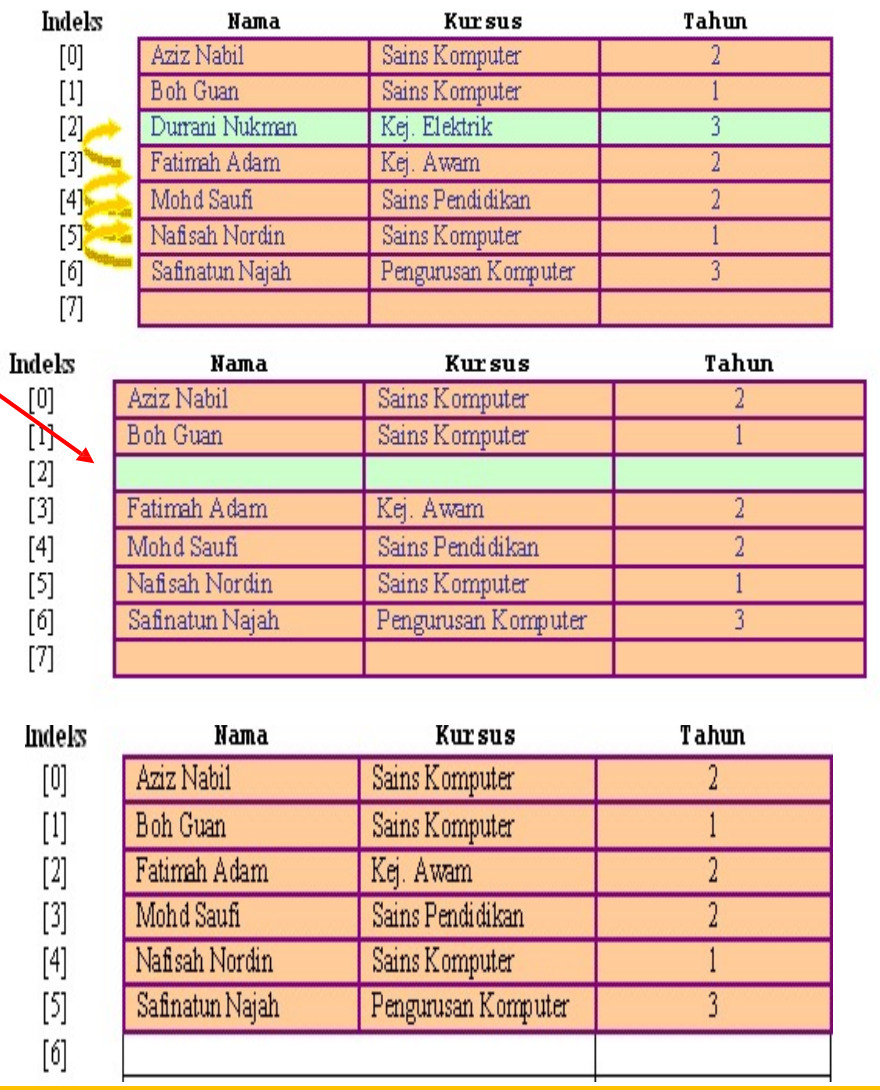
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Array Implementation … the drawbacks

To delete item in the middle of the array will leave a blank space in the middle.

It requires shifting all the elements in the list up one in order to eliminate the space..

Example: when information about Durrani Nukman is deleted, all elements below it, is shifted up.



Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]	Durrani Nukman	Kej. Elektrik	3
[3]	Fatimah Adam	Kej. Awam	2
[4]	Mohd Saufi	Sains Pendidikan	2
[5]	Nafisah Nordin	Sains Komputer	1
[6]	Safinatun Najah	Pengurusan Komputer	3
[7]			

Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]			
[3]	Fatimah Adam	Kej. Awam	2
[4]	Mohd Saufi	Sains Pendidikan	2
[5]	Nafisah Nordin	Sains Komputer	1
[6]	Safinatun Najah	Pengurusan Komputer	3
[7]			

Indeks	Nama	Kursus	Tahun
[0]	Aziz Nabil	Sains Komputer	2
[1]	Boh Guan	Sains Komputer	1
[2]	Fatimah Adam	Kej. Awam	2
[3]	Mohd Saufi	Sains Pendidikan	2
[4]	Nafisah Nordin	Sains Komputer	1
[5]	Safinatun Najah	Pengurusan Komputer	3
[6]			

Pointer Implementation (Linked List)

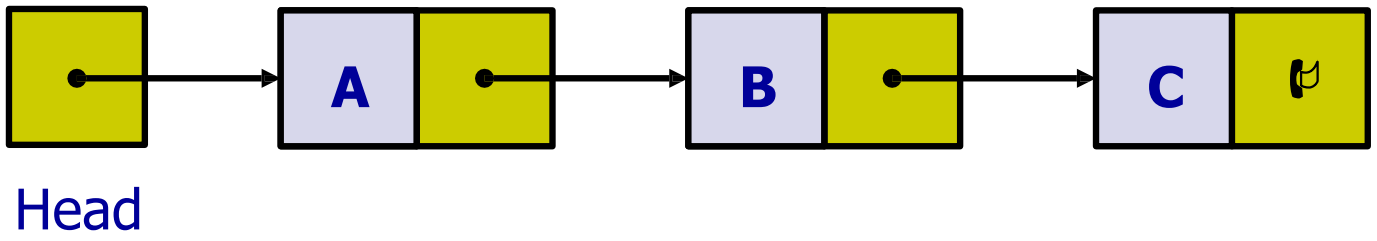
- Ensure that the list is not stored contiguously
  - use a linked list
  - a series of structures that are not necessarily adjacent in memory
- Each node contains the element and a pointer to a structure containing its successor
  - the last cell's next link points to NULL
- Compared to the array implementation,
  - the pointer implementation uses only as much space as is needed for the elements currently on the list
  - but requires space for the pointers in each cell

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

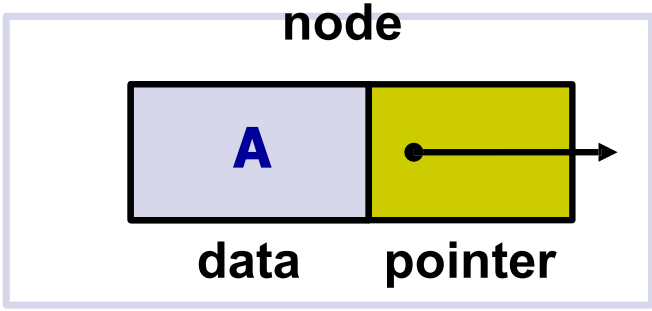
Linked list variations

- Singly linked list
- Doubly linked list
- Circular linked list
- Circular doubly linked list
- Sorted linked list
- Unsorted linked list

Singly Linked Lists



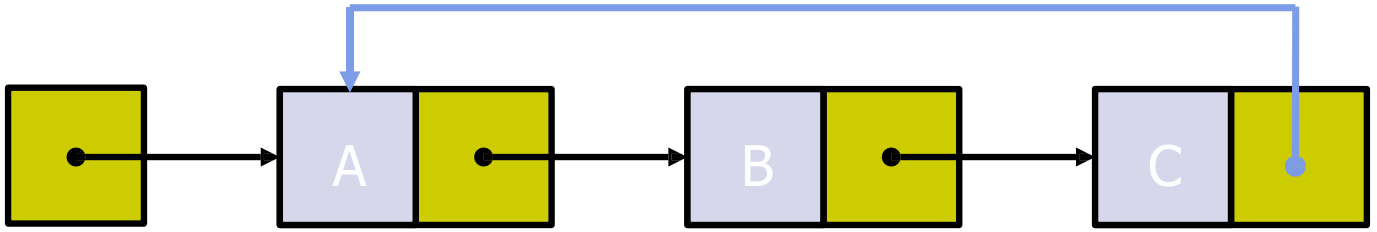
- A *linked list* is a series of connected *nodes*
- Each node contains at least
  - A piece of data (any type)
  - Pointer to the next node in the list
- *Head*: pointer to the first node
- The last node points to `NULL`



UTM UNIVERSITI TEKNOLOGI MALAYSIA

Variations of Linked Lists

- *Circular linked lists*
  - The last node points to the first node of the list

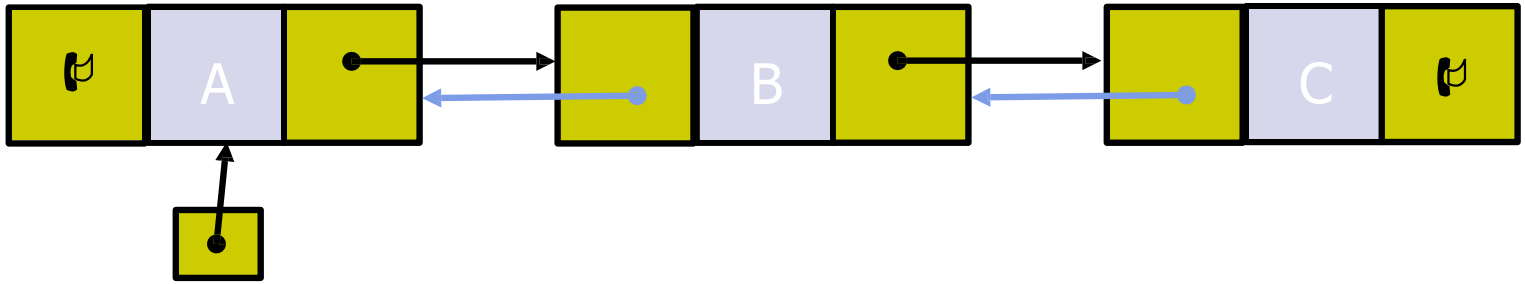


- How do we know when we have finished traversing the list? (Tip: check if the pointer of the current node is equal to the head.)

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Variations of Linked Lists

- *Doubly linked lists*
  - Each node points to not only successor but the predecessor
  - There are two `NULL:` at the first and last nodes in the list
  - Advantage: given a node, it is easy to visit its predecessor. Convenient to traverse lists backwards

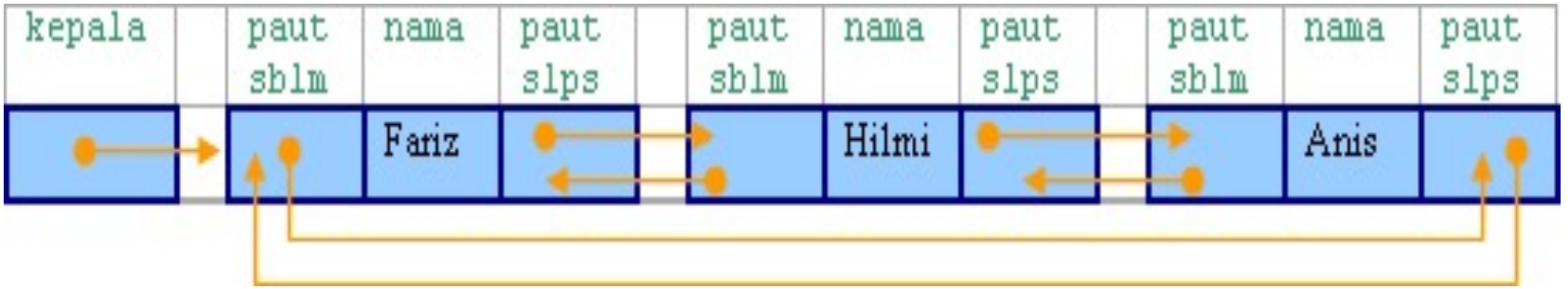


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Variations of Linked Lists

Circular doubly linked list

- No `NULL` value at the first and last nodes in the list
- Convenient to traverse lists backwards and forwards

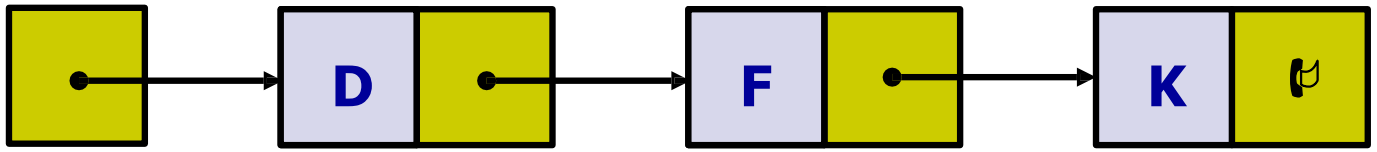


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Variations of Linked Lists

Sorted Linked list :

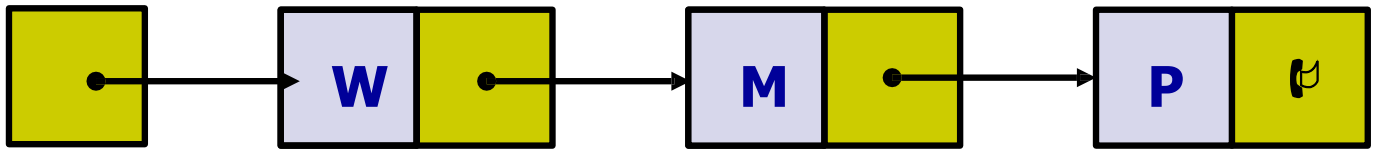
The nodes in the lists is sorted in certain order.



Head

UnSorted Linked list :

The nodes in the lists is not sorted in any order.



Head

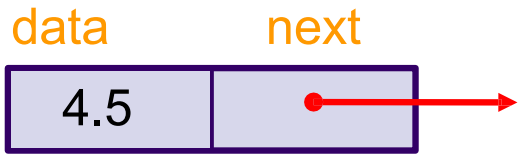
LINK LIST IMPLEMENTATION

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Linked List Implementation

- We need two classes: `Node` and `List`
- Declare `Node` class for the nodes which contains `data` and `next`: a pointer to the next node in the list

```
class Node {
public:
        double data;          // data
        Node* next;           // pointer to next node
};
```

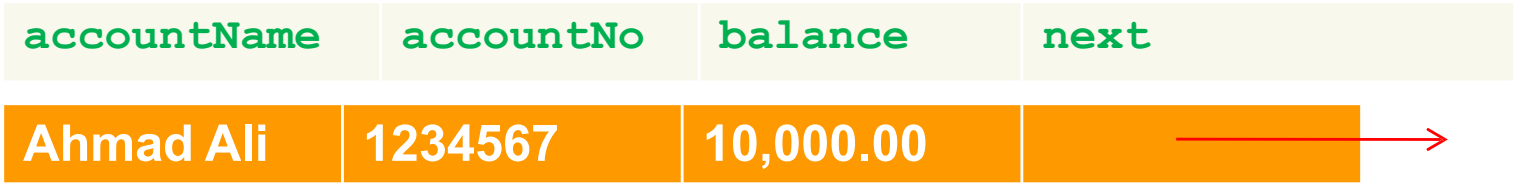


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Example :
Declaring a node for class account

Create a node for class account using struct

```
struct nodeAccount {
        char accountName[20];
        char accountNo[15];
        float balance;
        nodeAccount *next;
};
```



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Exercise – Declaring a node

bookTitle	ISBNno	price	next
Link List Guide	1234-5678	40.00	NULL

Figure above represents a node in a link list that store information for book.
The node consists of the following attributes :
bookTitle, ISBNno, price and pointer next.

Write a class/struct declaration for the node that is able to store the information.

A Simple Linked List Class

- Declare `List`, which contains
  - `head`: a pointer to the first node in the list.
    Since the list is empty initially, `head` is set to `NULL`
  - Operations on `List`

```
class List {
public:
        List(void) { head = NULL; }          // constructor
        ~List(void);                         // destructor

        bool IsEmpty() { return head == NULL; }
        Node* InsertNode(double x);
        int FindNode(double x);
        int DeleteNode(double x);
        void DisplayList(void);
private:
        Node* head;
};
```

UTM UNIVERSITI TEKNOLOGI MALAYSIA

A Simple Linked List Class

Operations of `List`

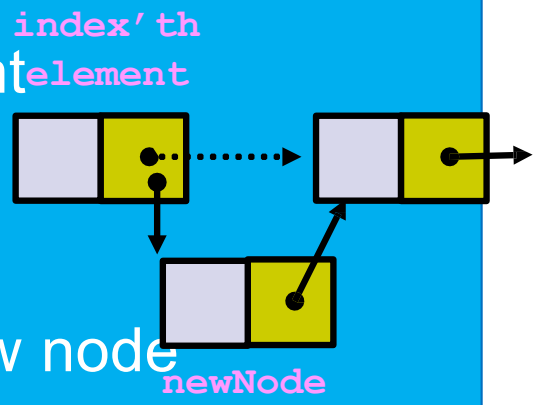
- **`IsEmpty`**: determine whether or not the list is empty
- **`InsertNode`**: insert a new node at a particular position
- **`FindNode`**: find a node with a given value
- **`DeleteNode`**: delete a node with a given value
- **`DisplayList`**: print all the nodes in the list

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Inserting a new node

X

- `Node* InsertNode(double x)`
  - Insert a node with data equal to `x`. After insertion, this function generates a sorted list : in ascending order.
  - Find the location of the value to be inserted so that the value will be in the correct order in the list. **(i.e., when `index = 0`, insert the node as the first element; when `index = 1`, insert the node after the first element, and so on)**
- Steps
  1. Locate `position to insert` the element
  2. Allocate memory for the new node
  3. Point the new node to its successor
  4. Point the new node's predecessor to the new node



UTM UNIVERSITI TEKNOLOGI MALAYSIA

Inserting a new node

- Possible cases of `InsertNode`
  1. Insert into an empty list
  2. Insert in front
  3. Insert at back
  4. Insert in middle
- But, in fact, only need to handle two cases
  - Insert as the first node (Case 1 and Case 2)
  - Insert in the middle or at the end of the list (Case 3 and Case 4)

Inserting a new node (will create a sorted list)

```
Node* List::InsertNode(double x) {
    int  currIndex  =       0;
    Node* currNode =        head;
    Node* prevNode =        NULL;
    while (currNode && x > currNode->data) {
            prevNode = currNode;
            currNode = currNode->next;
            currIndex++;
    }
    Node* newNode   =       new     Node;
    newNode->data   =       x;
    if (currIndex == 0) {
            newNode->next   =       head;
            head            =       newNode;
    }
    else {
            newNode->next   =       prevNode->next;
            prevNode->next =        newNode;
    }
    return newNode;
}
```

Try to locate the index of the node being inserted.

Inserting a new node

```
Node* List::InsertNode(double x) {

        int  currIndex  =       0;
        Node* currNode =        head;
        Node* prevNode =        NULL;
        while (currNode && x > currNode->data) {
                prevNode = currNode;
                currNode = currNode->next;
                currIndex++;
        }

        Node* newNode   =       new     Node;
        newNode->data   =       x;
        if (currIndex == 0) {
                newNode->next   =       head;
                head            =       newNode;
        }
        else {
                newNode->next   =       prevNode->next;
                prevNode->next  =       newNode;
        }
        return newNode;
}
```

Create a new node

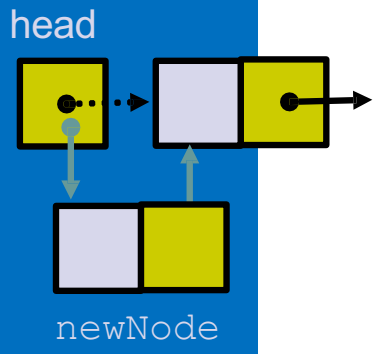
Inserting a new node

```
Node* List::InsertNode(double x) {

        int  currIndex  =       0;
        Node* currNode =        head;
        Node* prevNode =        NULL;
        while (currNode && x > currNode->data) {
                prevNode = currNode;
                currNode = currNode->next;
                currIndex++;
        }

        Node* newNode   =       new     Node;
        newNode->data   =       x;
        if (currIndex == 0) {
                newNode->next   =       head;
                head            =       newNode;
        }
        else {
                newNode->next   =       prevNode->next;
                prevNode->next  =       newNode;
        }
        return newNode;
}
```

Insert as first element



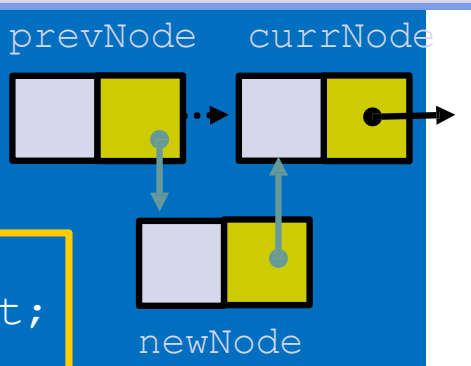
Inserting a new node

```
Node* List::InsertNode(double x) {

        int  currIndex  =       0;
        Node* currNode =        head;
        Node* prevNode =        NULL;
        while (currNode && x > currNode->data) {
                prevNode = currNode;
                currNode = currNode->next;
                currIndex++;
        }

        Node* newNode   =       new     Node;
        newNode->data   =       x;
        if (currIndex == 0) {
                newNode->next   =       head;
                head            =       newNode;
        }
        else {
                newNode->next   =       prevNode->next;
                prevNode->next =        newNode;
        }
        return newNode;
}
```

Insert after `prevNode`



Inserting a new node at certain index (will create unsorted list)

```
Node* List::InsertNode(int index, double x) {
    if (index < 0) return NULL;

    int currIndex  =      1;
    Node* currNode =      head;
    while (currNode && index > currIndex) {
          currNode        =      currNode->next;
          currIndex++;
    }
    if (index > 0 && currNode == NULL) return NULL;

    Node* newNode  =      new    Node;
    newNode->data  =      x;
    if (index == 0) {
          newNode->next   =      head;
          head            =      newNode;
    }
    else {
          newNode->next   =      currNode->next;
          currNode->next  =      newNode;
    }
    return newNode;
```

Try to locate `index`'th node. If it doesn't exist, return `NULL`.

Inserting a new node

```
Node* List::InsertNode(int index, double x) {
        if (index < 0) return NULL;

        int currIndex  =       1;
        Node* currNode =       head;
        while (currNode && index > currIndex) {
                currNode       =       currNode->next;
                currIndex++;
        }
        if (index > 0 && currNode == NULL) return NULL;

        Node* newNode  =       new     Node;
        newNode->data  =       x;
        if (index == 0) {
                newNode->next  =       head;
                head           =       newNode;
        }
        else {
                newNode->next  =       currNode->next;
                currNode->next =       newNode;
        }
        return newNode;
```

Create a new node

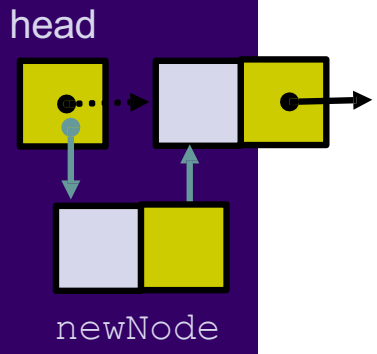
Inserting a new node

```
Node* List::InsertNode(int index, double x) {
        if (index < 0) return NULL;

        int currIndex  =       1;
        Node* currNode =       head;
        while (currNode && index > currIndex) {
                currNode       =       currNode->next;
                currIndex++;
        }
        if (index > 0 && currNode == NULL) return NULL;

        Node* newNode  =       new     Node;
        newNode->data  =       x;
        if (index == 0) {
                newNode->next  =       head;
                head           =       newNode;
        }
        else {
                newNode->next  =       currNode->next;
                currNode->next =       newNode;
        }
        return newNode;
```

Insert as first element



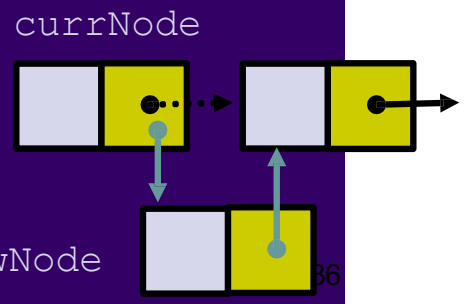
Inserting a new node

```
Node* List::InsertNode(int index, double x) {
        if (index < 0) return NULL;

        int currIndex  =       1;
        Node* currNode =       head;
        while (currNode && index > currIndex) {
                currNode       =       currNode->next;
                currIndex++;
        }
        if (index > 0 && currNode == NULL) return NULL;

        Node* newNode  =       new     Node;
        newNode->data  =       x;
        if (index == 0) {
                newNode->next  =       head;
                head           =       newNode;
        }
        else {
                newNode->next  =       currNode->next;
                currNode->next =       newNode;
        }
        return newNode;
```

Insert after currNode



Finding a node

- `int FindNode(double x)`
  - Search for a node with the value equal to `x` in the list.
  - If such a node is found, return its position. Otherwise, return 0.

```
int List::FindNode(double x) {
        Node* currNode      =      head;
        int currIndex=      1;
        while (currNode && currNode->data != x) {
                currNode      =      currNode->next;
                currIndex++;
        }
        if (currNode)
                return currIndex;
        else
                return 0;
}
```

Deleting a node

- `int DeleteNode(double x)`
  - Delete a node with the value equal to `x` from the list.
  - If such a node is found, return its position. Otherwise, return 0.
- Steps
  - Find the desirable node (similar to `FindNode`)
  - Release the memory occupied by the found node
  - Set the pointer of the predecessor of the found node to the successor of the found node
- Like `InsertNode`, there are two special cases
  - Delete first node
  - Delete the node in middle or at the end of the list

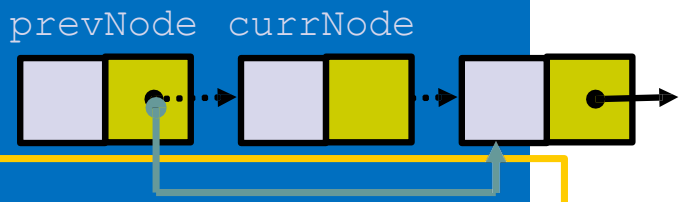
Deleting a node

```
int List::DeleteNode(double x) {
        Node* prevNode =        NULL;
        Node* currNode =        head;
        int currIndex  =        1;
        while (currNode && currNode->data != x) {
                prevNode        =       currNode;
                currNode        =       currNode->next;
                currIndex++;
        }
        if (currNode) {
                if (prevNode) {
                        prevNode->next =        currNode->next;
                        delete currNode;
                }
                else {
                        head            =       currNode->next;
                        delete currNode;
                }
                return currIndex;
        }
        return 0;
```

Try to find the node with its value equal to `x`

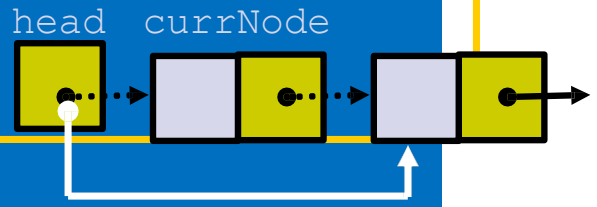
Deleting a node

```
int List::DeleteNode(double x) {
        Node* prevNode =        NULL;
        Node* currNode =        head;
        int currIndex  =        1;
        while (currNode && currNode->data != x) {
                prevNode        =       currNode;
                currNode        =       currNode->next;
                currIndex++;
        }
        if (currNode) {
                if (prevNode) {
                        prevNode->next =        currNode->next;
                        delete currNode;
                }
                else {
                        head            =       currNode->next;
                        delete currNode;
                }
                return currIndex;
        }
        return 0;
}
```



Deleting a node

```
int List::DeleteNode(double x) {
        Node* prevNode =        NULL;
        Node* currNode =        head;
        int currIndex  =        1;
        while (currNode && currNode->data != x) {
                prevNode        =       currNode;
                currNode        =       currNode->next;
                currIndex++;
        }
        if (currNode) {
                if (prevNode) {
                        prevNode->next =        currNode->next;
                        delete currNode;
                }
                else {
                        head    = currNode->next;
                        delete currNode;
                }
                return currIndex;
        }
        return 0;
}
```



UTM UNIVERSITI TEKNOLOGI MALAYSIA

Printing all the elements

- **void DisplayList(void)**
  - Print the data of all the elements
  - Print the number of the nodes in the list

```
void List::DisplayList()
{
    int num             =       0;
    Node* currNode      =       head;
    while (currNode != NULL){
        cout << currNode->data << endl;
        currNode        =       currNode->next;
        num++;
    }
    cout << "Number of nodes in the list: " << num << endl;
}
```

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Destroying the list : `~List()`

- Use the destructor to release all the memory used by the list.
- Step through the list and delete each node

```
List::~List()
{   Node* currNode = head, *nextNode = NULL;
    while (currNode != NULL)
    {    nextNode = currNode->next;
         // destroy the current node
         delete currNode;
         currNode = nextNode;
    }
    head = Null;
}
```

4

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Using List

```
int main(void)
{
        List list;
        list.InsertNode(7.0); // successful
        list.InsertNode(5.0); // successful
        list.InsertNode(6.0); // successful
        list.InsertNode(4.0); // successful
        // print all the elements
        list.DisplayList();
        if(list.FindNode(5.0)  > 0) cout << "5.0 found" << endl;
        else                        cout << "5.0 not found" << endl;
        if(list.FindNode(4.5)  > 0) cout << "4.5 found" << endl;
        else                        cout << "4.5 not found" << endl;
        list.DeleteNode(7.0);
        list.DisplayList();
        return 0;
}
```

result

```
4
5
6
7
Number of nodes in the list: 4
5.0 found
4.5 not found
4
5
6
Number of nodes in the list: 3
```

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

- Linked lists are more complex to code and manage than arrays, but they have some distinct advantages.
  - Dynamic: a linked list can easily grow and shrink in size.
    - We don't need to know how many nodes will be in the list. They are created in memory as needed.
    - In contrast, the size of a C++ array is fixed at compilation time.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Easy and fast insertions and deletions

- To insert or delete an element in an array, we need to copy to temporary variables to make room for new elements or close the gap caused by deleted elements.
- With a linked list, no need to move other nodes. Only need to reset some pointers.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Linked List - Conclusion

- Try make your own conclusion by comparing array list and linked list in term of:
  - Size of items/nodes in the list
  - Add new items/nodes at the beginning or in the middle of the list
  - Delete items at the beginning or in the middle of the list
  - Get items at nodes *n*

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Lab Exercise

- Write a complete link list program based on the implementation of the following classes and functions given in the notes.
  - Class **`Node`** and **`List`**
  - **`Constructor`** and **`Destructor`**
  - **`IsEmpty()`**, **`InsertNode()`**, **`FindNode()`** **`DeleteNode()`** and **`DisplayList()`**
  - Provide main() program to execute the list.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Thank You

innovative ● entrepreneurial ● global

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE & ALGORITHM

Chapter 09

Queue

Objectives

At the end of the lesson students are expected to be able to:

- Understand queue concepts and applications.
- Understand queue structure and operations that can be done on queue.
- Understand and know how to implement queue using array and linked list : linear array, circular array, linear link list and circular list.

Introduction to Queue

New items enter at the back, or rear, of the queue

- Items leave from the front of the queue
- First-in, first-out (FIFO) property
  - The first item inserted into a queue is the first item to leave
  - Middle elements are logically inaccessible
- Important in simulation & analyzing the behavior of complex systems

Queue Applications

Real-World Applications

– Cashier lines in any store

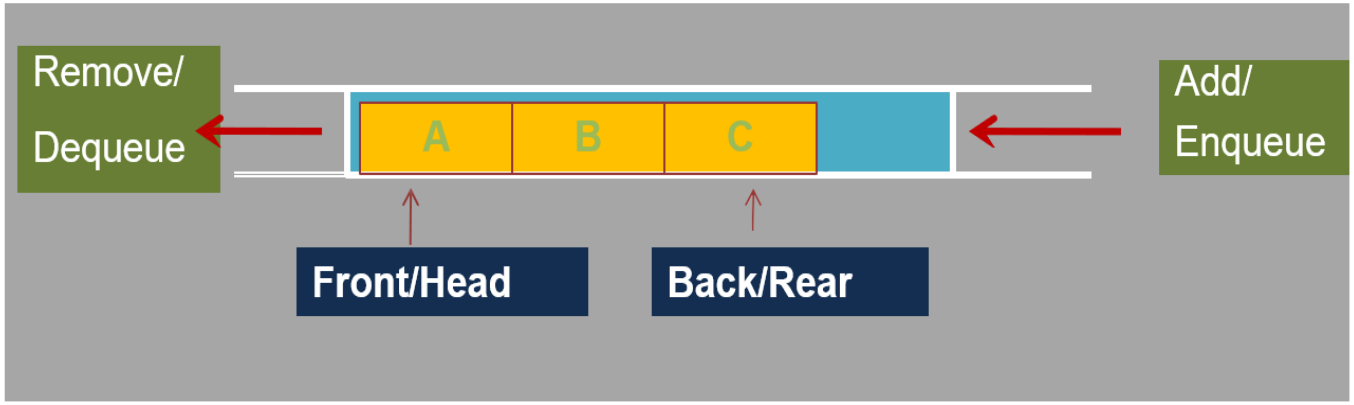
– Check out at a bookstore

– Bank / ATM

– Call an airline

- Computer Science Applications
  - Print lines of a document
  - Printer sharing between computers
  - Recognizing palindromes
  - Shared resource usage (CPU, memory access, ...)
- Simulation
  - A study to see how to reduce the wait involved in an application

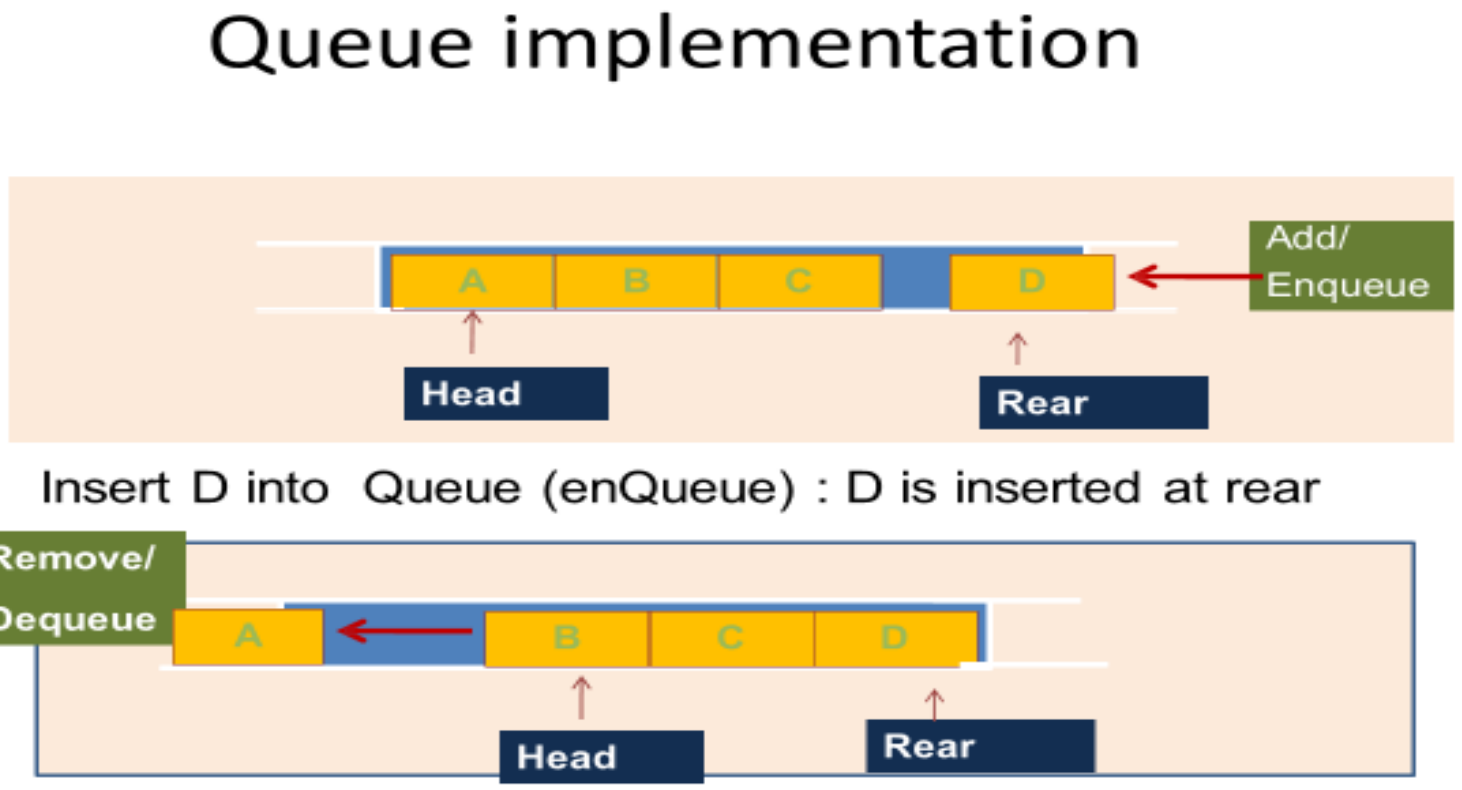
Queue Implementation



Basic Structure of a Queue:

- Data structure that hold the queue
- head
- rear

Queue Implementation



Abstract Data Type Queue

Implementation:

- Array-based (Linear or Circular)
- Pointer-based : Link list (Linear or Circular)

Abstract Data Type Queue

- ADT queue operations
  - Create an empty queue
- Destroy a queue
- Determine whether a queue is empty
- Add a new item to the queue
- Remove the item that was added earliest
- Retrieve at Front
- Retrieve at Back the item that was added earliest

Queue: Linear Array Implementation

- Number of elements in Queue are fixed during declaration.
- Need ***isFull()*** operation to determine whether a queue is full or not.
- Queue structure need at least 3 elements:

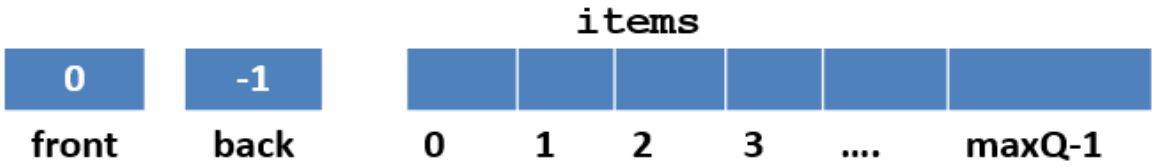
1) Element to store items in Queue
2) Element to store index at head
3) Element to store index at rear

Queue
front *rear* *items*
createQueue() *destroyQueue()* *isEmpty();* *isFull();* *enQueue();* *deQueue();* *getFront();* *getRear();*

CreateQueue() operation

- Linear Array implementation
  - *front* & *back* are indexes in the array
  - Initial condition: *front =0* & *back = -1*

```
Queue::Queue()
{ front = 0;
  back = -1;
}
```



Initial state for a queue linear array

Queue operations

- **Destroy Queue distructor : All elements in the queue will be disposed.**

```
queue::~queue()
{ delete [ ] items; }
```

- **Check whether a queue is empty**
  - Queue Empty Condition : back < front

```
bool queue::isEmpty()
{ return bool(back < front); }
```

Queue operations



Check whether a queue is Full

- Queue Full Condition : back = size -1

```
bool queue::isFull()
{ return bool(back == size - 1); }
```

- No more item can be insert into a queue, when a queue is full.

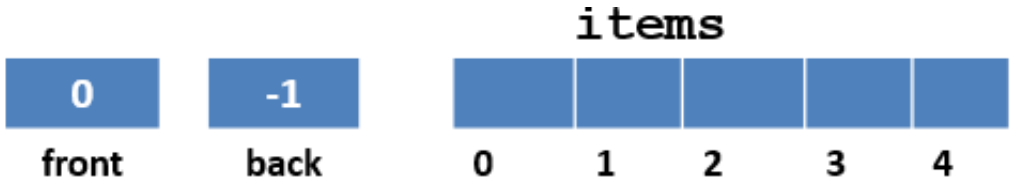
Queue operations

- Insert into a queue (enQueue)
  - Increment back
  - Insert item in `items[ back ]`

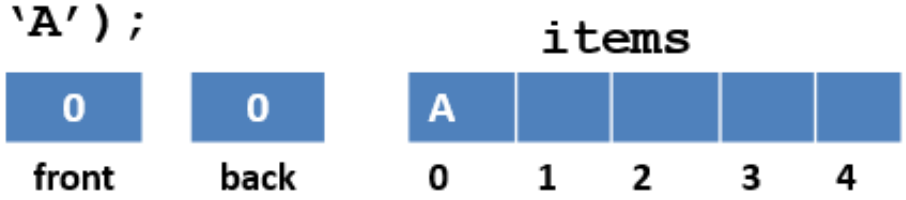
```
void queue::enQueue(char insertItem)
{ if (isFull())
      cout<< "\nCannot Insert. Queue is full!";
  else
  { //insert at back
    back++;
    items[back] = insertItem;
  } // end else if
}
```

enQueue operations for a queue with size = 5

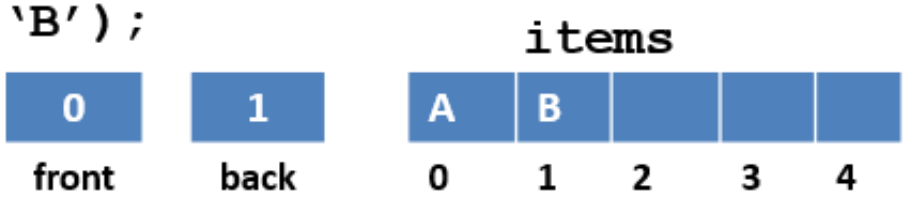
```
Queue myQueue;
```



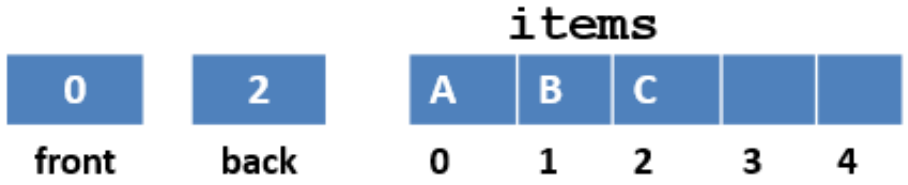

```
myQueue.enQueue('A');
```



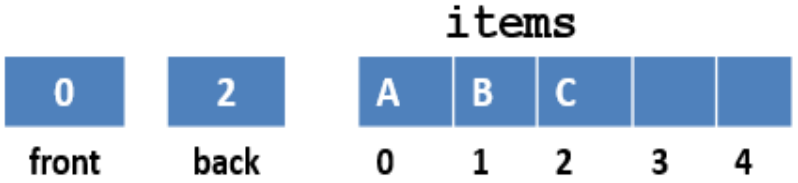

```
myQueue.enQueue('B');
```




```
myQueue.enQueue('C');
```



Queue operations



- Item at front and back can be retrieved from queue

```
char queue:: getFront() // get item at Front
{ return items[front] ; }
```

```
char queue::getRear() // get item at Back
{ return items[back] ; }
```

```
cout << myQueue.getFront(); //output is A
cout << myQueue.getRear(); // output is C
```

Queue operations

- Delete from a queue (deQueue)
  - Increment front

```
void queue::deQueue()
{ if (isEmpty())
    cout<< "\nCannot remove item. Empty Queue!";
  else
  { //retrieve item at front
    deletedItem = items[front];
    front++;
  } // end else if
}
```

deQueue operations

```
myQueue.deQueue();
```




```
myQueue.deQueue();
```




```
myQueue.deQueue();
```




```
myQueue.deQueue();
```

```
Cannot remove item.
Queue is Empty with back < front
```

Queue operations - enQueue

```
myQueue.enQueue('D');
```




```
myQueue.enQueue('E');
```




```
myQueue.enQueue('F');
```

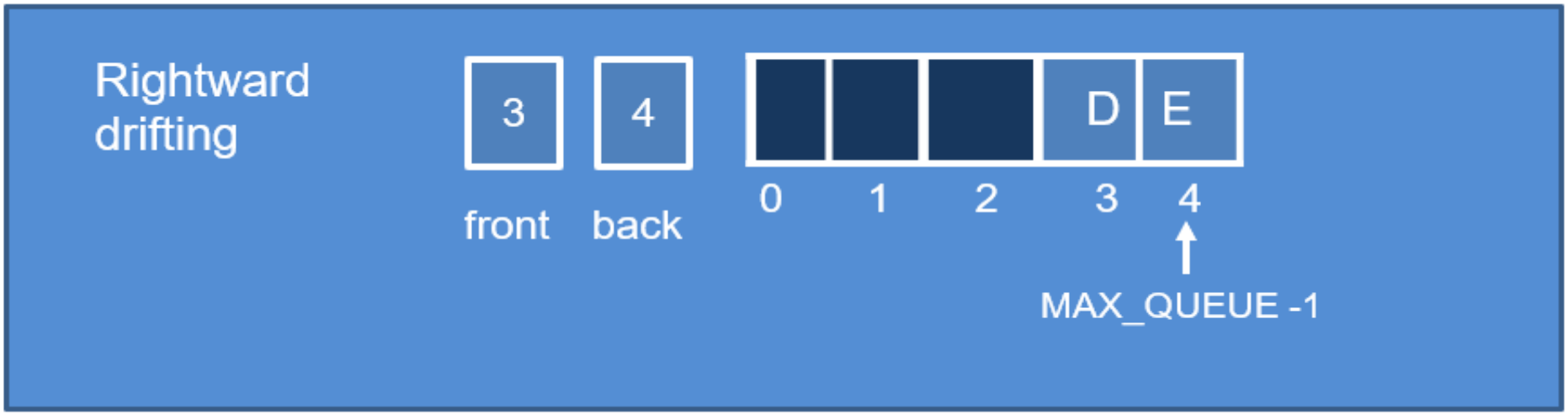



```
Cannot insert F, even though there are empty spaces
in front of the queue array.
Currently, Queue is FULL with back == size - 1.
```

Linear array implementation - drawback

Problem: Rightward-Drifting:

- After a sequence of additions & removals, items will drift towards the end of the array
- Even though, there are empty spaces in front of the queue array, enQueue operation cannot be performed on the queue, since back = max_queue – 1.



Rightward drifting solutions

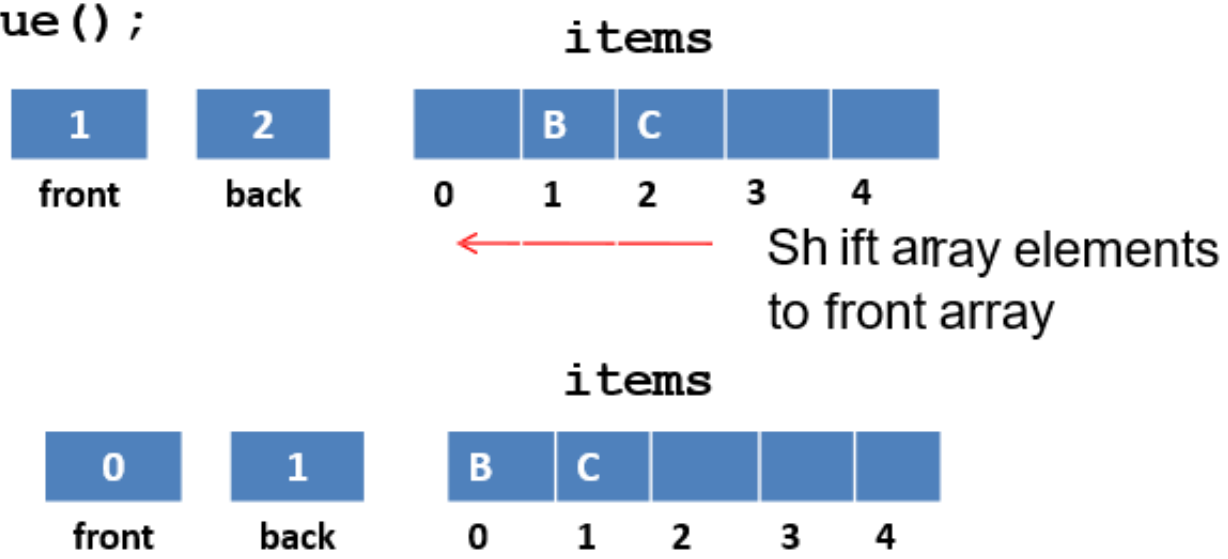
To optimize space and to solve rightward drifting:

1. Shift array elements after each deletion.

```
myQueue.deQueue();
```

deletedItem

A



However, shifting is not effective and dominates the cost of the implementation.

Queue circular array

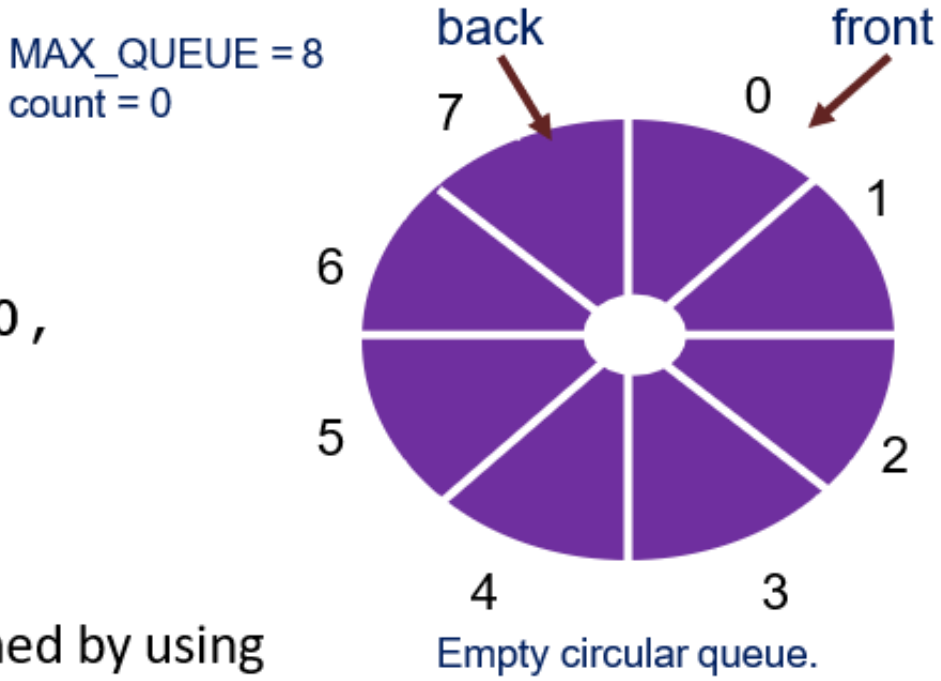
- Problem:
  - front & back no longer can be used to distinguish between queue-full & queue-empty conditions
- Solution:
  - Use a counter
  - count == 0 means empty queue
  - count == MAX_QUEUE means full queue
- Disadvantage
  - Overhead of maintaining a counter or flag

Circular Array Implementation

- Queue declarations

```
const int MAX_QUEUE = maximum-size-of-queue;
QueueItemType items [MAX_QUEUE];
int   front;
int   back;
int   count
```

- Initial condition:
  - `count = 0, front = 0,`
  - `back = MAX_QUEUE - 1`



Empty circular queue.

- The Wrap-around effect is obtained by using arithmetic (%-operator)

Circular Array Implementation

- Insertion:
  - Increment back, using modulo arithmetic
  - Insert item
  - Increment count

```
Back = (back + 1) % MAX_QUEUE;
items [back] = newItem;
++count
```

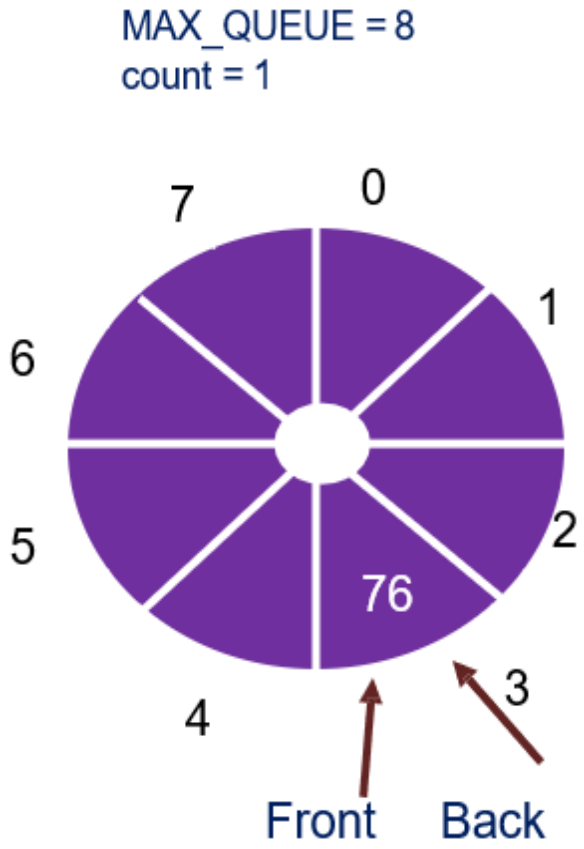
Circular Array Implementation

Deletion

Increment *front* using modulo arithmetic
Decrement *count*

```
front = ( front + 1 ) % MAX_QUEUE;
--count;
```

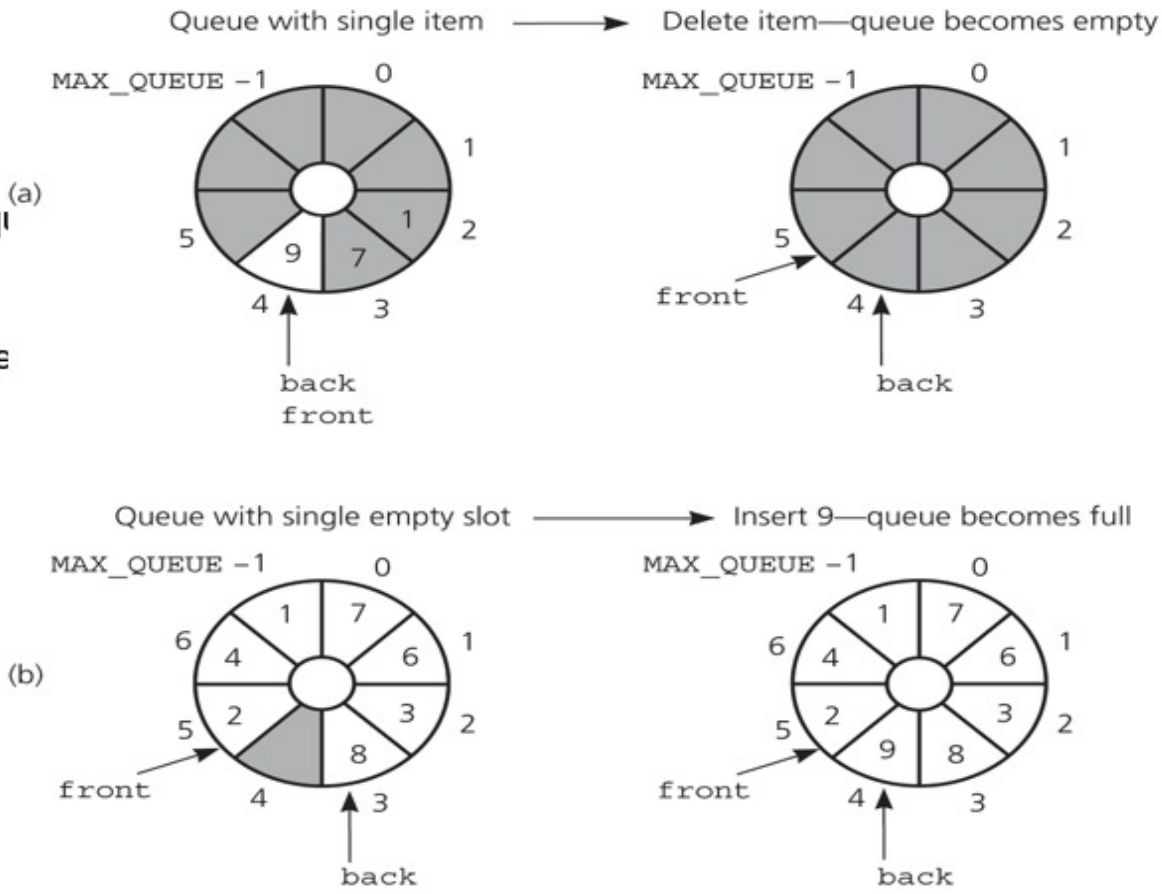
After delete 20, 45 and 51 sequentially from circular queue



Circular Array Implementation

(a) *front* passes *back* when the qu
becomes empty;

(b) *back* catches up to *front* whe
the queue becomes full





Next.....
Queue implementation Linked List

SCJ2013 Data Structure & Algorithms

Binary Search Tree

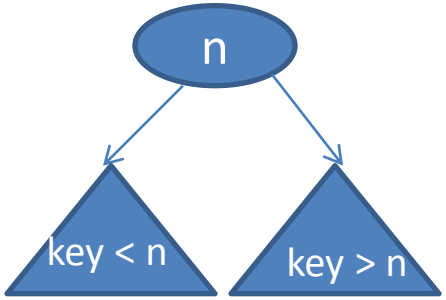
Nor Bahiah Hj Ahmad

CC BY NC SA

Binary Search Tree

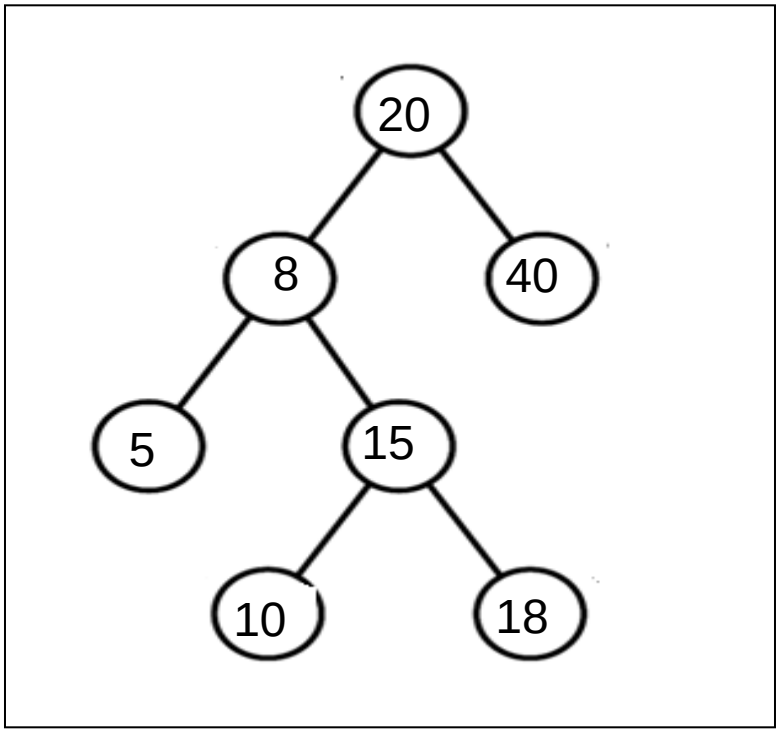
- A binary search tree has the following properties:
  - For every node n in the tree
    - Value of n is greater than all values in n's left subtree.
    - Value of n is less than all values in n's right subtree.
    - Both left subtree and right subtree are also binary search trees

Binary Search Trees

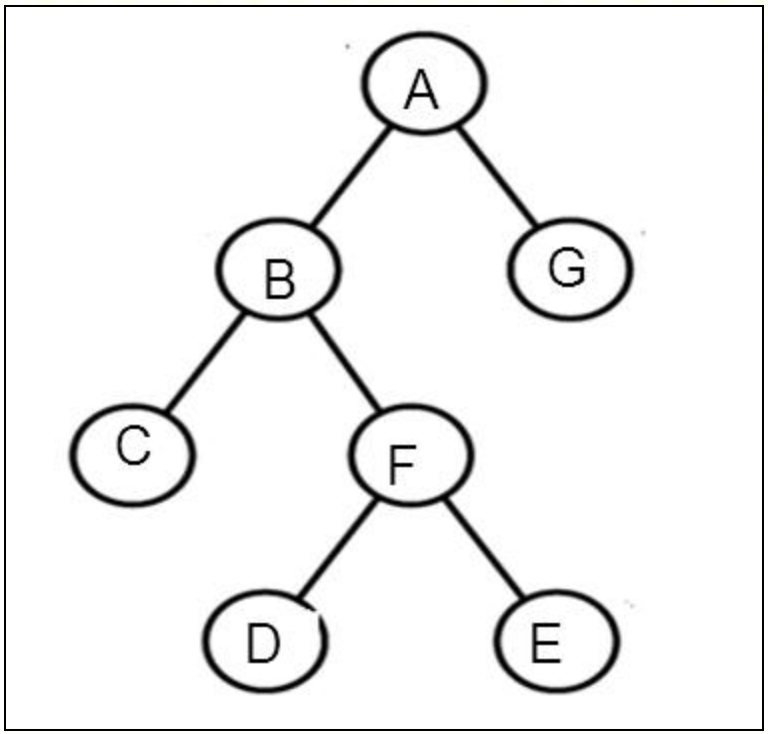


All key in the left sub-tree is less than n

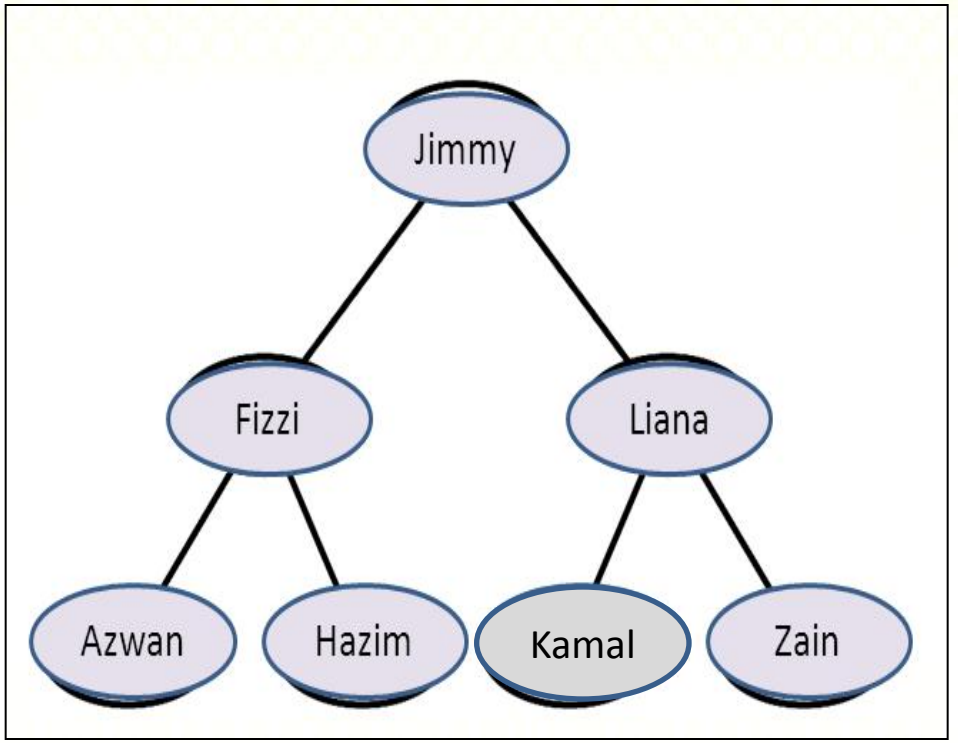
All key in the right sub-tree are greater than n



Binary Search Tree



Not a Binary Search Tree



Binary Search Tree

CC BY NC SA

Advantages of Binary Search Tree

One of the most fundamental algorithms in Computer Science and the method of choice in many applications.

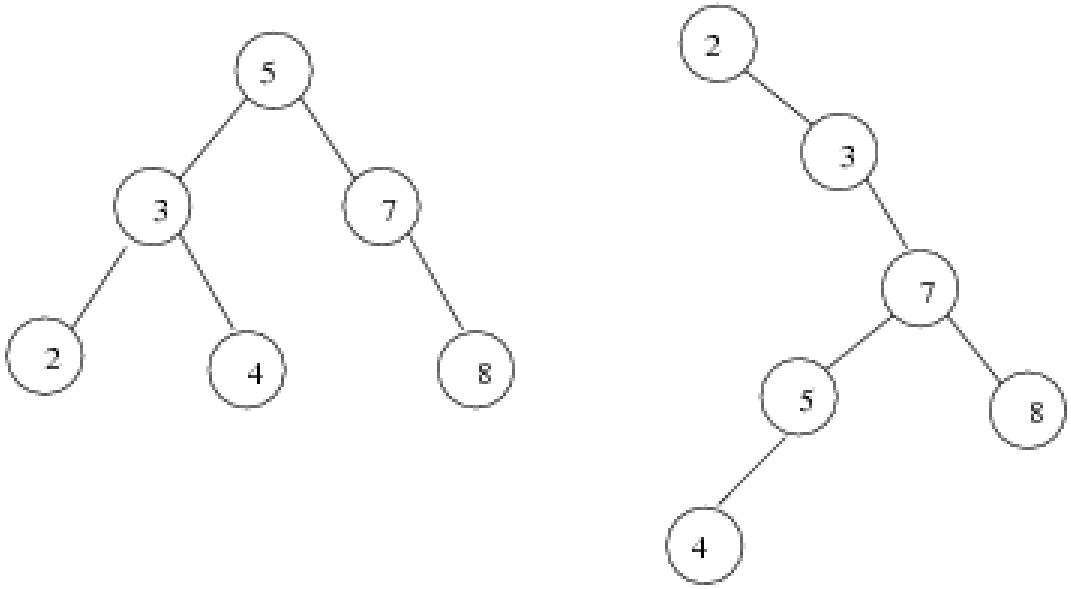
- Stores keys in the nodes in a way that searching, insertion and deletion can be done efficiently.
- Simple Implementation
- Nodes in tree are dynamic

Disadvantages of Binary Search Tree

- The shape of the tree depends on the order of insertions, and it can be degenerated.
- When inserting or searching for an element, the key of each visited node has to be compared with the key of the element to be inserted/found.
- Keys in the tree may be long and the run time may increase.

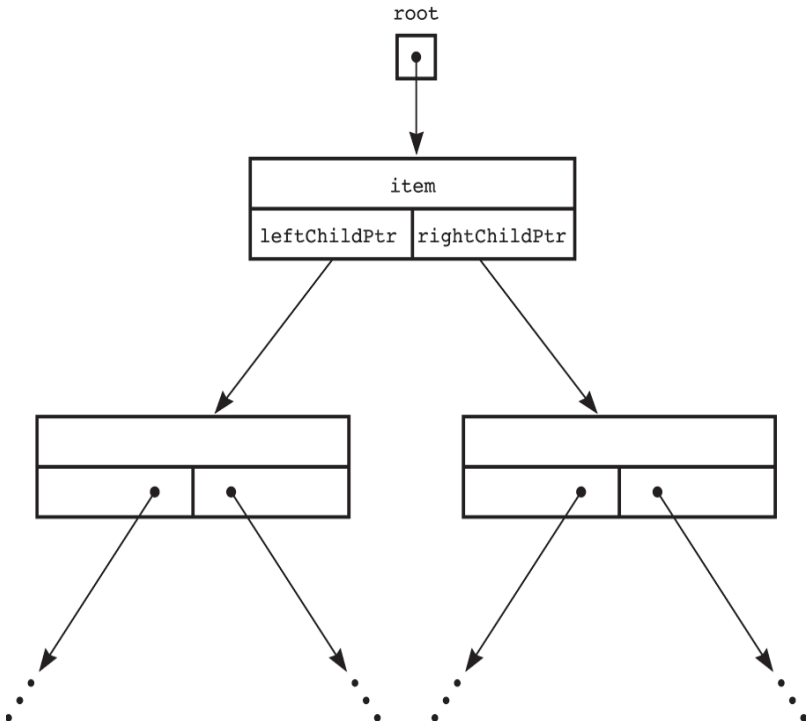
Binary search tree

Two binary search trees representing the same set:



– Average depth of a node is O(log n);
– Maximum depth of a node is O(n)

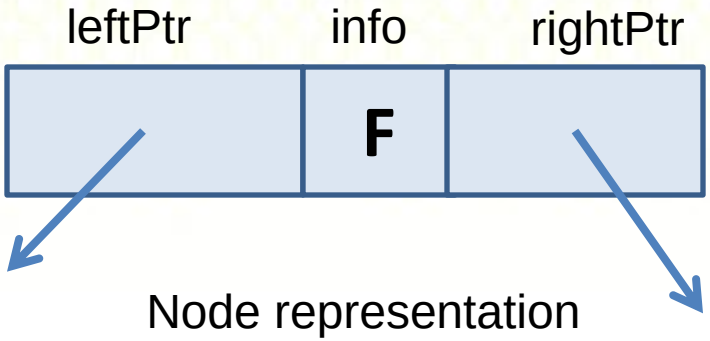
Pointer-based ADT Binary Tree



A pointer-based implementation of a binary tree

- Elements in a binary tree is represented by using nodes.
- Nodes store the information in a tree.
- Each node in the tree must contain at least 3 fields containing:
  - item
  - Pointer to left subtree
  - Pointer to right subtree
- Need 2 declaration in a tree implementation:
  1. Node declaration
  2. Tree declaration

Node Implementation



Node representation

```
typedef char ItemType;
struct TreeNode
{
    itemType info;
    TreeNode *left;
    TreeNode *right;
}
```

info, the node store char value.

left, pointer to left subtree

Right, pointer to right subtree

cc BY NC SA

Tree Implementation

```
class TreeType {
public:
  TreeType();
  ~TreeType();
  bool IsEmpty()const;
  int NumberOfNodes()const;
  void RetrieveItem(ItemType&,bool& found);
  void InsertItem(ItemType);
  void DeleteItem(ItemType);
  void PrintTree() const;
private:
  TreeNode * root;
};
```

The tree class declaration above, declare the binary serach tree using class TreeType.

The tree can be accessed using root, which is a pointer to root of the tree.

CC BY NC SA

Tree Operations

Among the tree operations in the class TreeType:

1. Initialize tree , using constructor.
2. Destroy tree, destructor.
3. Check for empty tree, IsEmpty().
4. Count number of nodes in the tree.
5. Search item in the tree
6. Insert item into a tree.
7. Delete item from tree.
8. Print all item in the tree (Inorder traversal)

cc BY NC SA

Tree Constructor

```
TreeType::TreeType()
{     root = NULL;    }
```

The constructor create an empty tree by initializing root to NULL value.

Tree Destructor

```
TreeType::~TreeType()
{   Destroy(root);   }
void Destroy(TreeNode *& tree)
{   if (tree!=NULL)
    {   Destroy(tree->left);
        Destroy(tree->right);
        delete(tree);
     }
}
```

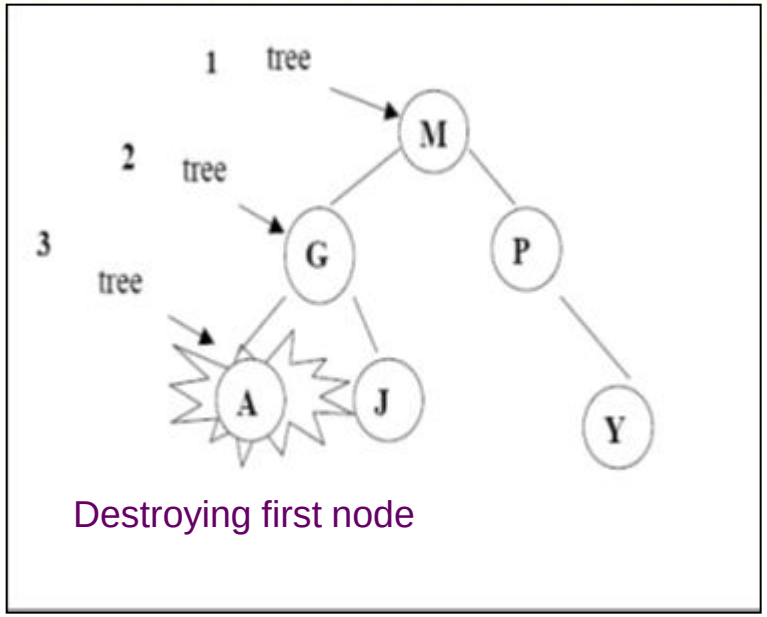
Destructor will destroy all the nodes in the tree

Function `Destroy()` is implemented recursively whereby the function will destroy all nodes in the left subtree first, followed by destroying nodes in the right subtree. Lastly, the root node will be destroyed.

cc BY NC SA

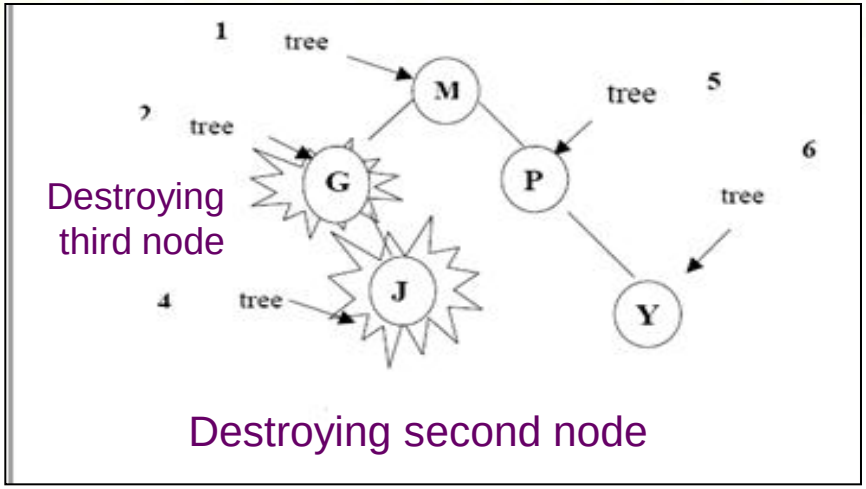
Destroying a tree using destructor

Step 1



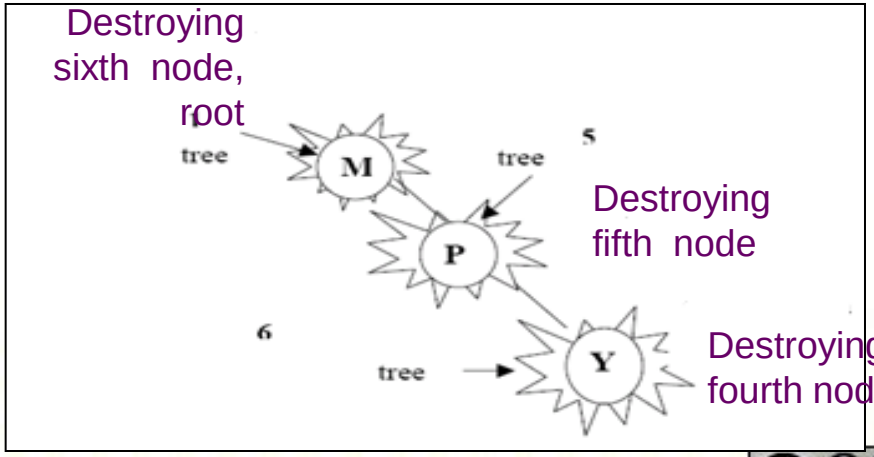
Destroying first node

Step 2



Destroying second node

Destroying sixth node, root



Step 3

CC BY NC SA

`IsEmpty()` Function

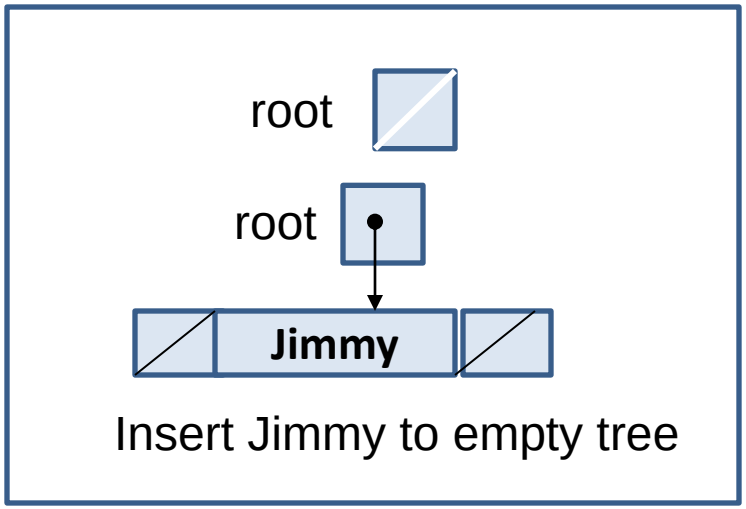
- Binary Search Tree is Empty when the root has NULL value.
- Function IsEmpty() will return True if root is NULL and will return False if the root is not NULL.

```
bool IsEmpty() const
{ if (root == NULL)
    return True; // tree is empty
  else
    return False; // tree is not empty
}
```

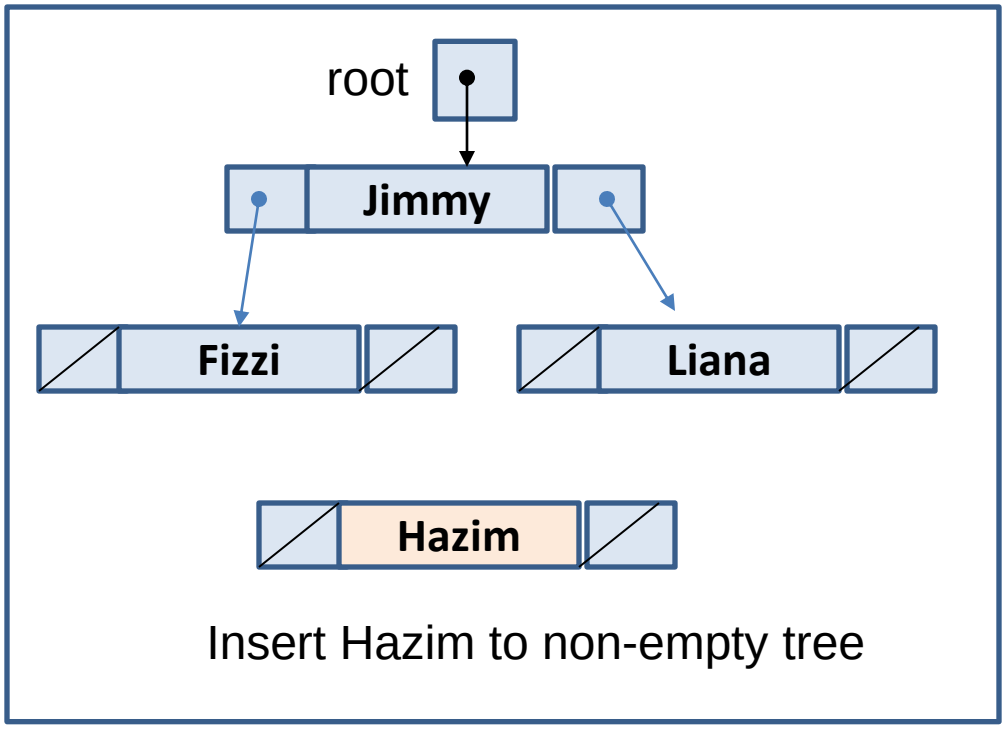
Insert Node Into a Binary Search Tree

- The insert operation will insert a node to a tree and the new node will become leaf node.
- Before the node can be inserted into a BST, the position of the new node must be determined. This is to ensure that after the insertion, the BST characteristics is still maintained.
- Steps to insert a new node in BST
  1. Find the position of the new node in the tree.
  2. Allocate new memory for the new node.
  3. Set NULL value to left and right pointer.
  4. Assign the value to be stored in the tree.

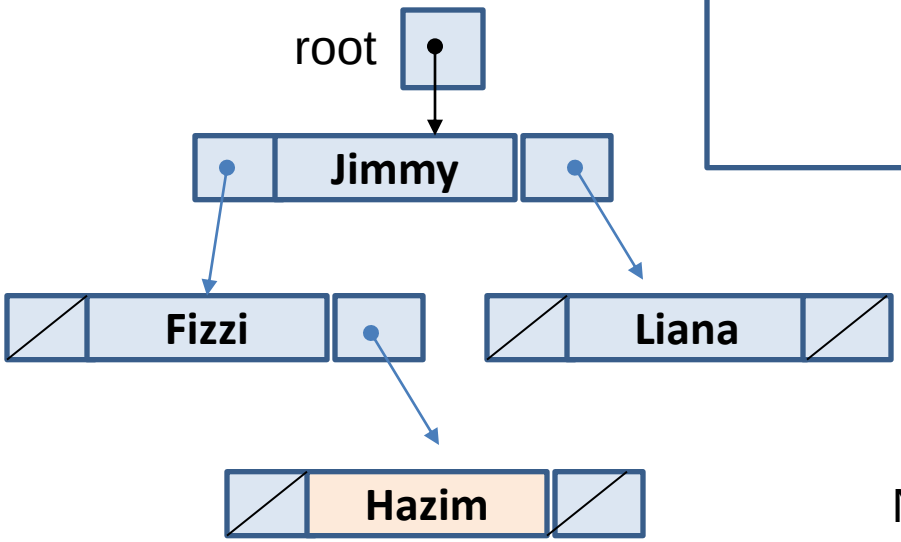
ADT Binary Search Tree: Insertion



Insert Jimmy to empty tree



Insert Hazim to non-empty tree



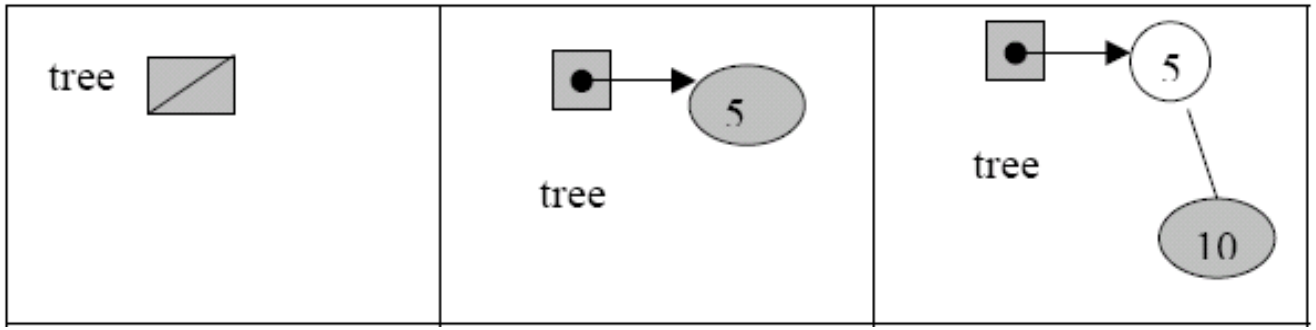
Node with value Hazim becomes a leaf.

Insert new node Implementation

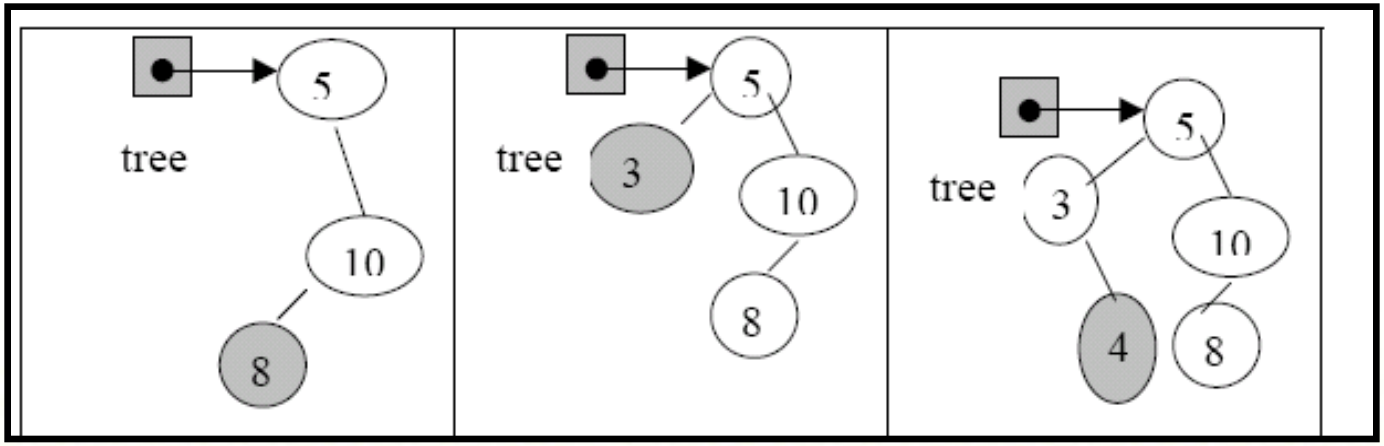
```
void TreeType::InsertItem(ItemType item)
{ Insert(root, item);}

void Insert(TreeNode*& tree, ItemType item)
{   if (tree == NULL) { // base case
          tree = new TreeNode;
          tree->right = NULL;
          tree->left = NULL;
          tree->info = item;
}
else if (item < tree->info)
        Insert(tree->left, item);
else
        Insert(tree->right, item);
}
```

Insert 5, 10, 8, 3 , 4 and 15 in a BST



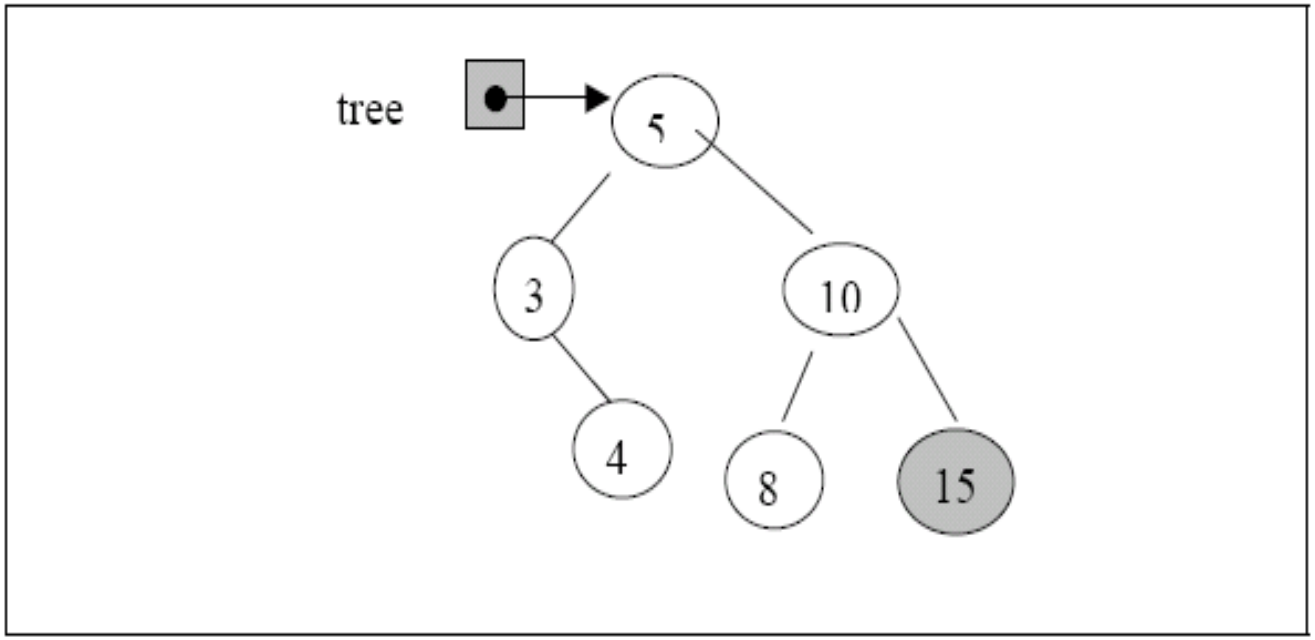
Empty tree Insert 5 Insert 10



Insert 8 Insert 3 Insert 4

CC BY NC SA

Insert 5, 10, 8, 3 , 4, 15 to a tree

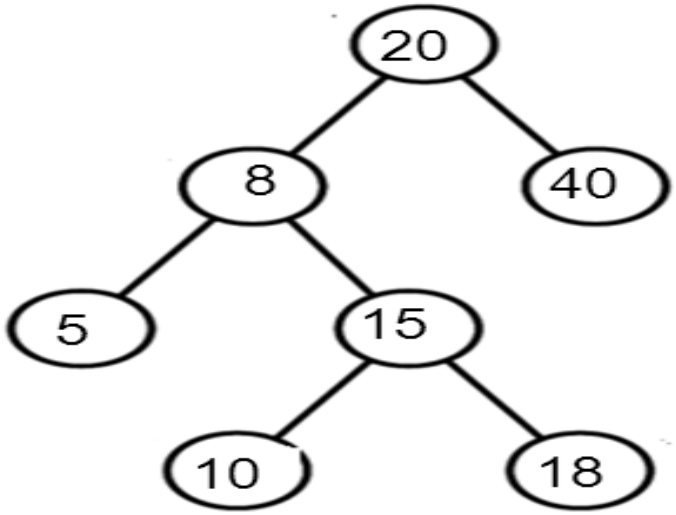


Finally, Insert the last node; 15.

Time complexity = O(height of the tree)

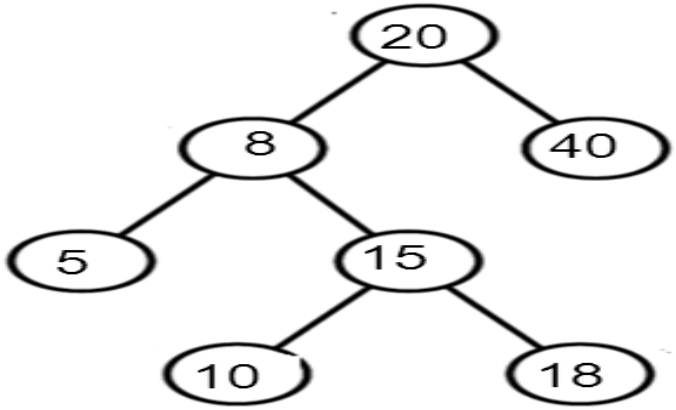
cc BY NC SA

Searching from BST



- From the figure, if we search for value 20, ***then we are done at the root.***
- If we search for a value < 20, then searching will be at the left subtree.
- If we are searching for a value > 20, then searching will be at the right subtree.

Searching from BST



Search value 10 from the tree

1. Compare 10 with value 20 (at root), go to left subtree.
2. Compare 10 with 8, go to right subtree.
3. Compare 10 with 15, go to left subtree.
4. Compare 10 with 10, value is found.

Time complexity = O(height of the tree)

= O(4)

Searching: RetrieveItem() function

```
void TreeType:: RetrieveItem
                (ItemType& item,bool& found)
{ Retrieve(root, item, found);}

void Retrieve(TreeNode* tree,
                 ItemType& item,bool& found)
{ if (tree == NULL) // base case 2
    found = false;
 else if(item < tree->info)
    Retrieve(tree->left, item, found);
 else if(item > tree->info)
    Retrieve(tree->right, item, found);
 else {  // base case 1
    found = true;
 }
```

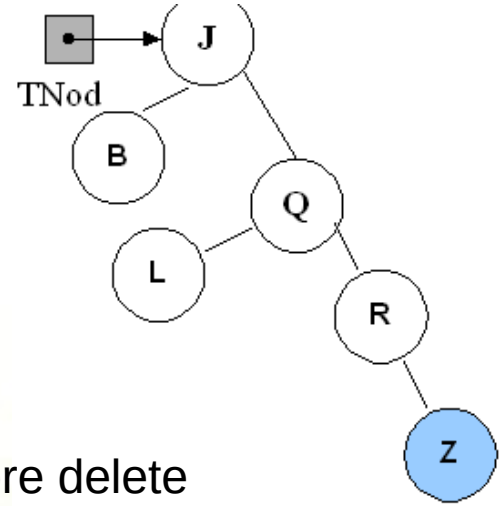
Delete a Node from a Tree

- When a node is deleted, the children of the deleted node must be taken care of to ensure that the property of the search tree is maintained.
- There are 3 possible cases to delete a node in a tree:
  1. Delete a leaf
  2. Delete a node with one child
  3. Delete a node that has two children

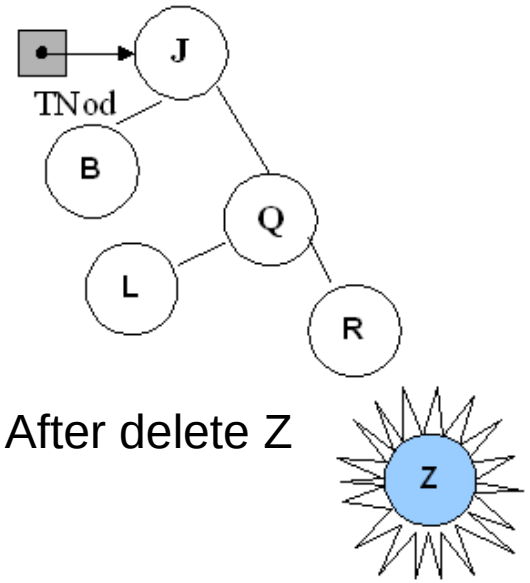
Delete a Leaf Node

The node to be deleted is a leaf

- Set the pointer in N's parent to NULL and delete it immediately
- Example : Delete leaf Node: Z



Before delete



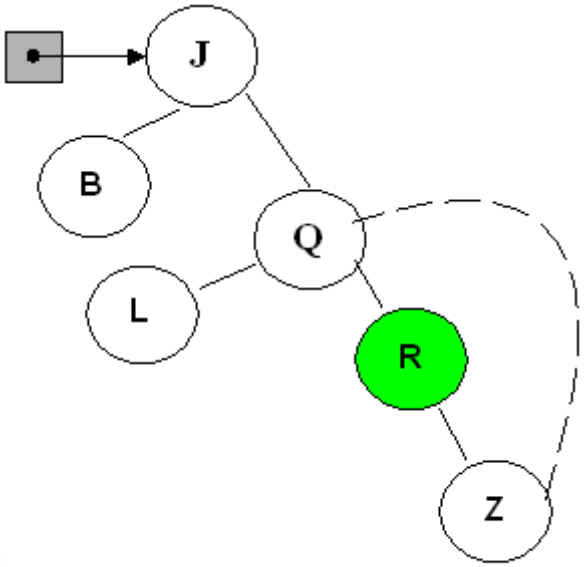
After delete Z

cc BY NC SA

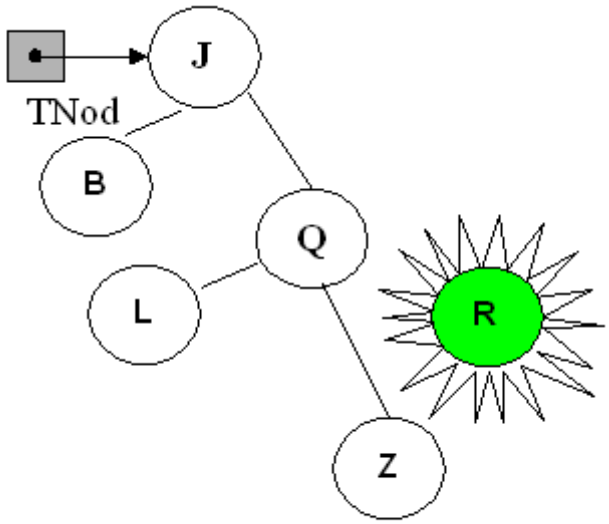
Delete a Node With One Child

Delete node R.

Adjust a pointer from the parent to bypass that node



Before delete R



After delete R

CC BY NC SA

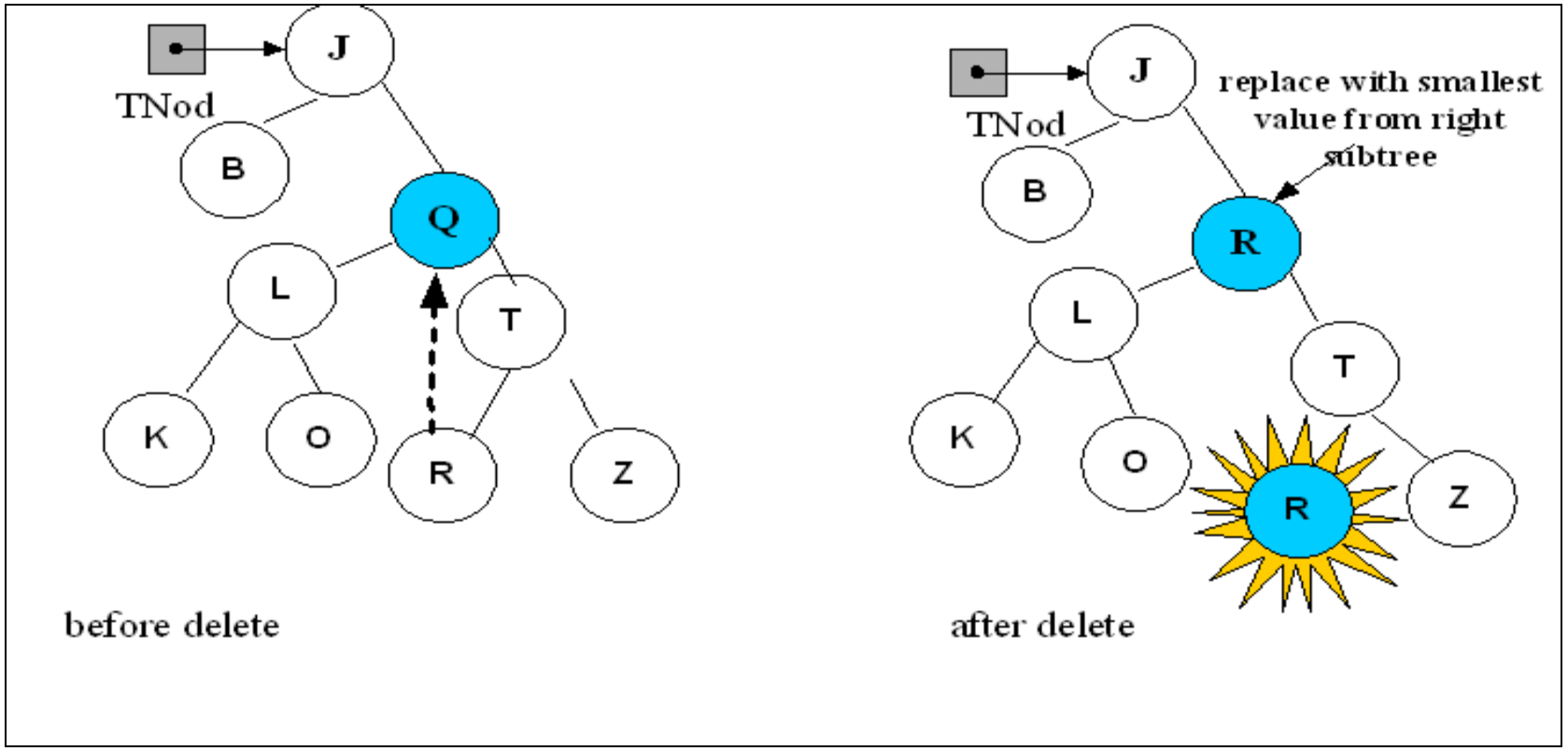
Delete a Node With Two Children

To delete a node N that has two children.

- Locate another node *M* that is easier to delete
  - M is the leftmost node in N's right subtree
  - M will have no more than one child
  - M's search key is called the inorder successor of N's search key
- Copy the item that is in M to N
- Remove the node M from the tree

Delete a Node with 2 Children

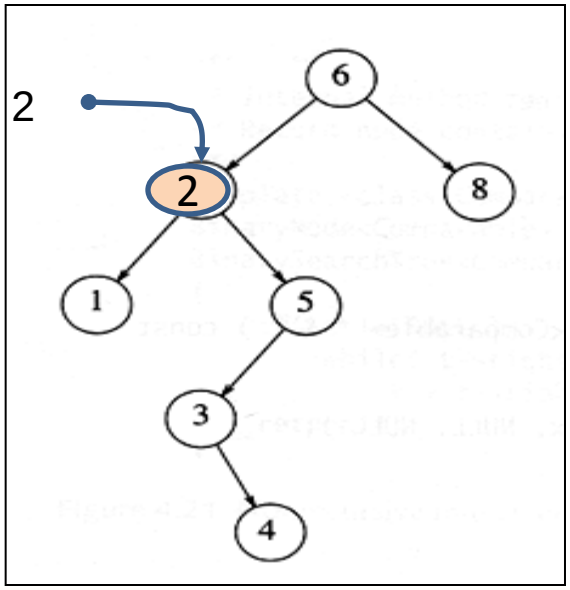
Delete Q that has 2 children

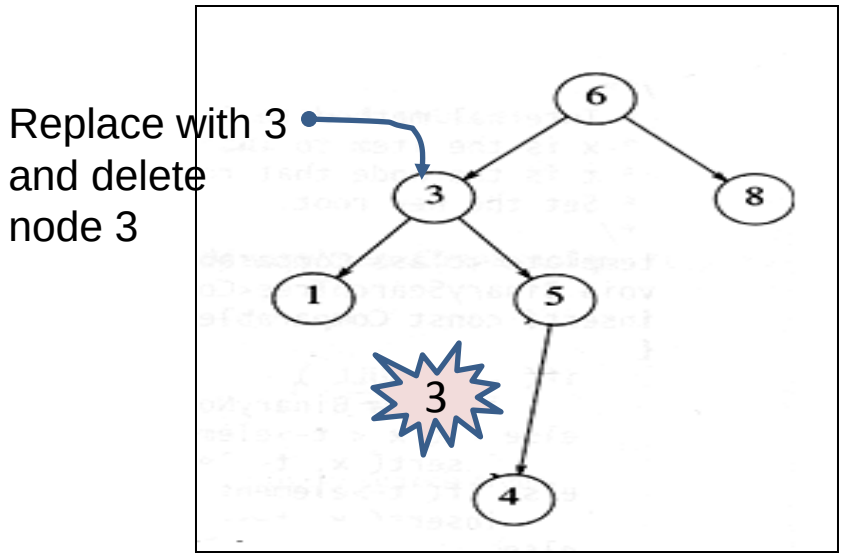


Delete a Node With 2 Children

Delete 2 with 2 children.

- Replace the key of that node with the minimum element at the right subtree.
- Delete the minimum element that has either no child or only right child because if it has a left child, that left child would be smaller and would have been chosen.





Print Values in BST

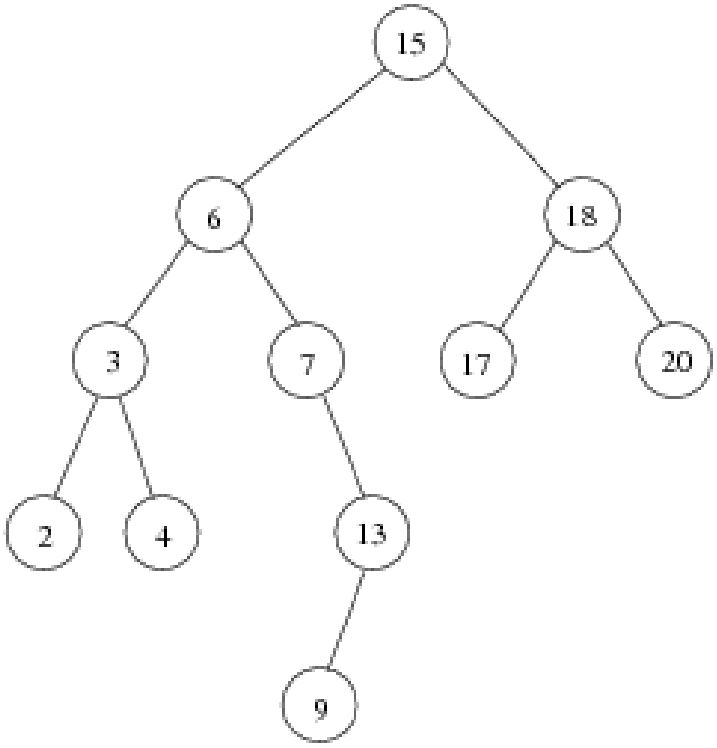
```
void TreeType::PrintTree()
{
 Print(root);
}
void Print(TreeNode* tree)
{
 if(tree != NULL) {
   Print(tree->left);
   cout << tree->info;
   Print(tree->right);
 }
```

Function PrintTree() print all values in BST using inorder traversal. Print() function will be called recursively, starting from left subtree, root and right subtree.

CC BY NC SA

Inorder traversal of BST

Print out all the keys in sorted order



Inorder: 2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20

The Efficiency of Binary Search Tree Operations

- The maximum number of comparisons required by any binary search tree operation is the number of nodes along the longest path from root to a leaf, which is the tree's height.
- The order in which insertion and deletion operations are performed on a binary search tree affects its height.
- Insertion in random order produces a binary search tree that has near-minimum height.
- Insertion in sequential order produces a binary search tree that is unbalanced and has height = O(n).

The Efficiency of Binary Search Tree Operations

Operation	Average case	Worst case
Retrieval	O(log *n*)	O(*n*)
Insertion	O(log *n*)	O(*n*)
Deletion	O(log *n*)	O(*n*)
Traversal	O(*n*)	O(*n*)

Summary and Conclusion

- Binary search trees come in many shapes. The shape of the tree determines the efficiency of its operations
- The height of a binary search tree with *n* nodes can range from a minimum of O$\log_2(n + 1)$ to a maximum of *n.*
- The efficiency of binary search tree operations:

Operation	Average case	Worst case
Retrieval	O(log *n*)	O(*n*)
Insertion	O(log *n*)	O(*n*)
Deletion	O(log *n*)	O(*n*)
Traversal	O(*n*)	O(*n*)

References

- Nor Bahiah et al. *"Struktur data & algoritma menggunakan C++". Penerbit UTM. 2005.*
- Frank M. Carano, Janet J Prichard. "*Data Abstraction and problem solving with C++" Walls and Mirrors*. 5th edition (2007). Addision Wesley.

SCJ2013 Data Structure & Algorithms

Tree

Nor Bahiah Hj Ahmad

CC BY NC SA

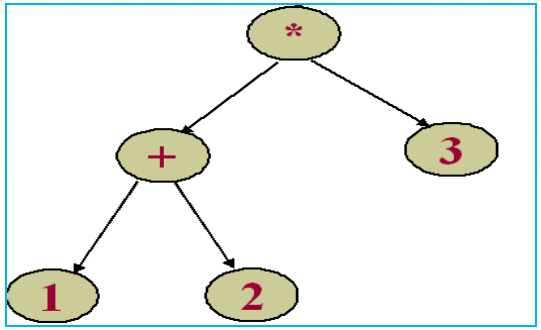
Course Objectives

At the end of the lesson students are expected to be able to:

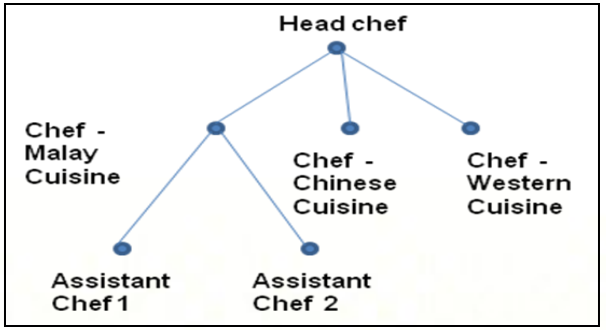
- Understand the tree concept and terms related to tree.
- Identify characteristics of general tree, binary tree and binary search tree
- Identify basic operations of a tree such as tree traversals, insert node, delete node, searching.
- Understand and know how to apply and implement tree in problem solving and in programming.

Introduction to tree - Definition

- Tree is a non-linear data structure.
- Data in a tree is stored in a hierarchy form.
- Example of tree application:
  - Represent algebraic formulas
  - Store data in hierarchy form. Ex: organization chart
  - Artificial intelligence – information is accessed based on certain decision which is stored in a tree.



algebraic formulas : (1+2)*3

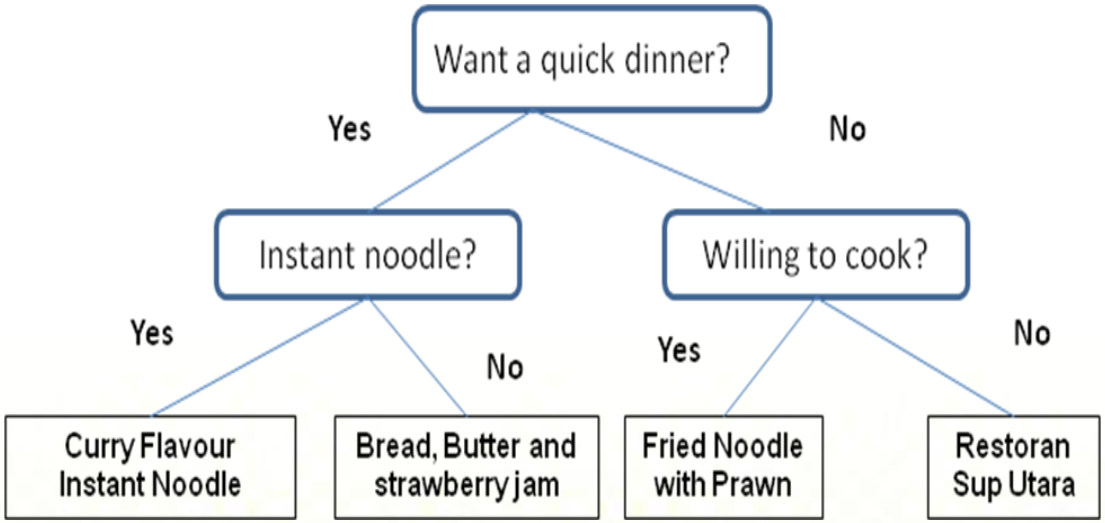


organization chart

cc BY NC SA

Introduction to tree – Decision Tree

- Binary tree associated with a decision process
- Internal nodes: questions with yes/no answer
- External nodes: decisions
- Example: dining decision tree



CC BY NC SA

Tree

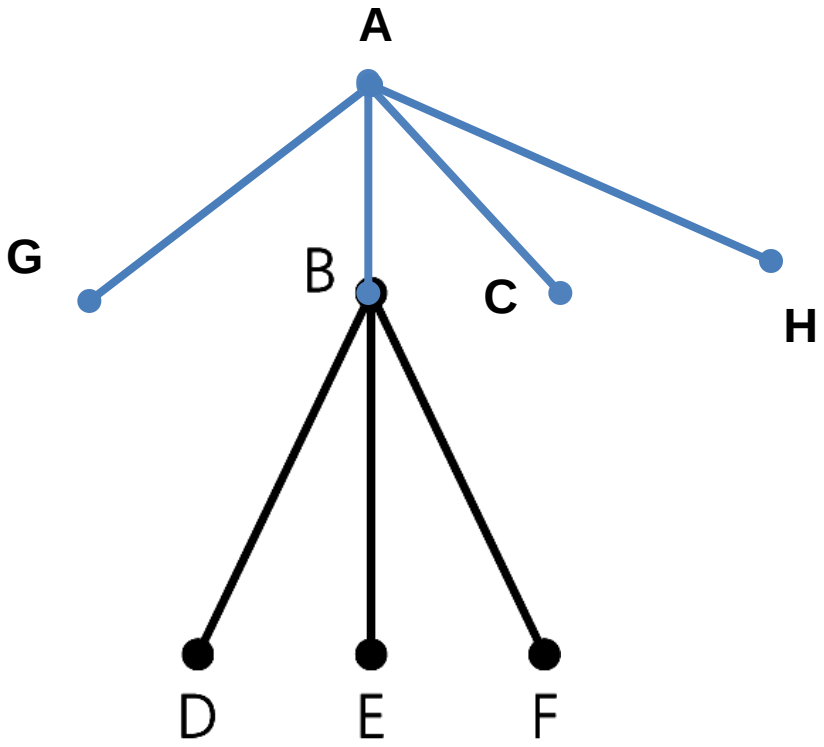
A tree is a collection of nodes and edges that connect the nodes.

- The collection can be empty.
- If not empty, a tree consists of a root, and zero or more nonempty subtrees.
- Any two vertices in a tree must have only one path between them or else its not a tree.
- Trees are hierarchical
  - Has parent-child relationship between two nodes.
  - Has ancestor-descendant relationships among nodes.

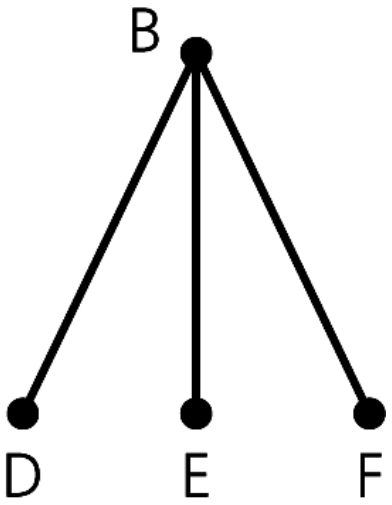
Tree terminology

- General tree
  - A general tree is a set of one or more nodes that is partitioned into :
    - The root
    - Sets that are general trees, called subtrees
  - Each node in general tree can have an unlimited children
- Subtree of a tree: Any node and its descendants

Tree Terminology



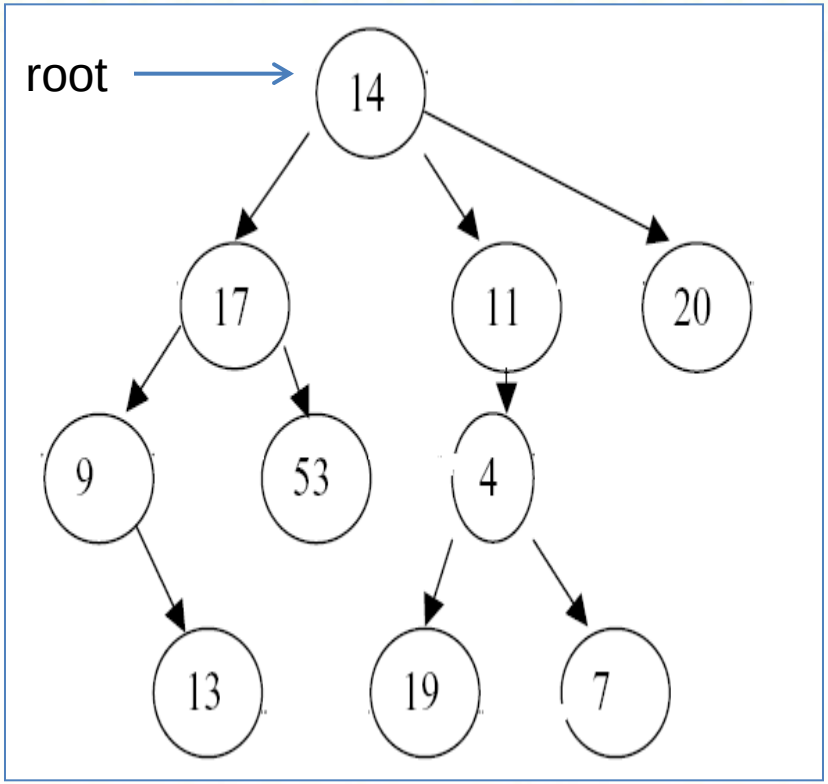
A general tree



A subtree of the tree in general tree

Tree Terminologies

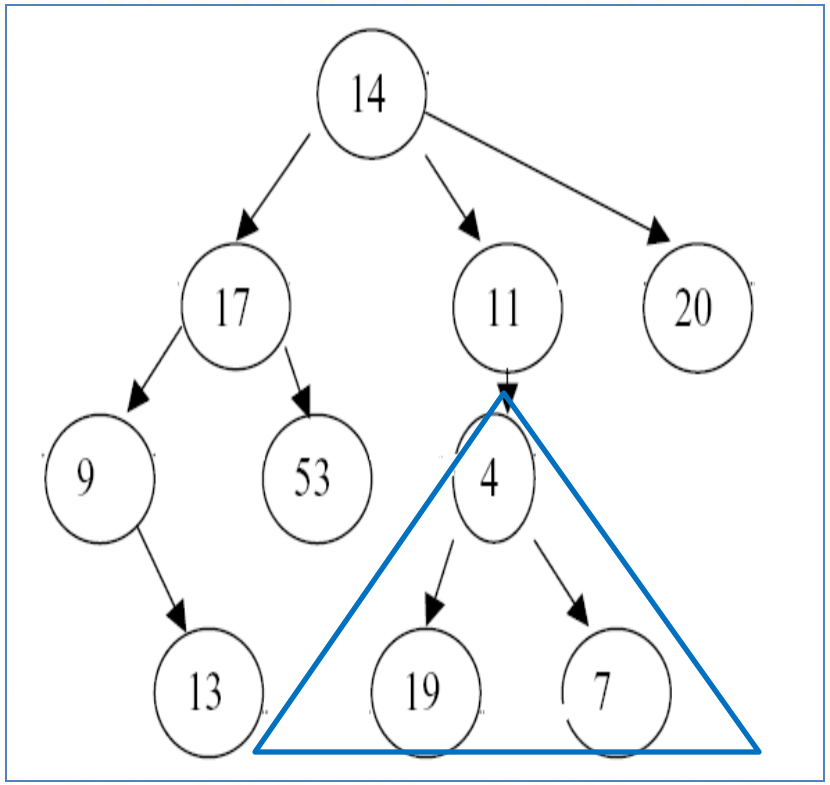
- **Root**
  - The only node in the tree with no parent
  - *A tree has only one root*
  - *Root : 14*
- ***Child* and *parent***
  - Every node except the root has one parent
  - **Parent of node *n***
    - The node directly above node *n* in the tree
    - 14 is Parent to 17, 11, 20
  - A node can have an arbitrary number of children
  - **Child of node *n***
    - A node directly below node *n* in the tree
    - 17, 11, 20 are children of 14



CC BY NC SA

Some Terminologies

- ***Leaves***
  - Nodes with no children
  - 13, 53, 19,7, 20
- ***Sibling***
  - nodes with the same parent
  - 17, 11, 20 are siblings
  - 19 and 7 are siblings
- **Subtree of node *n***
  - A tree that consists of a child (if any) of node *n* and the child's descendants

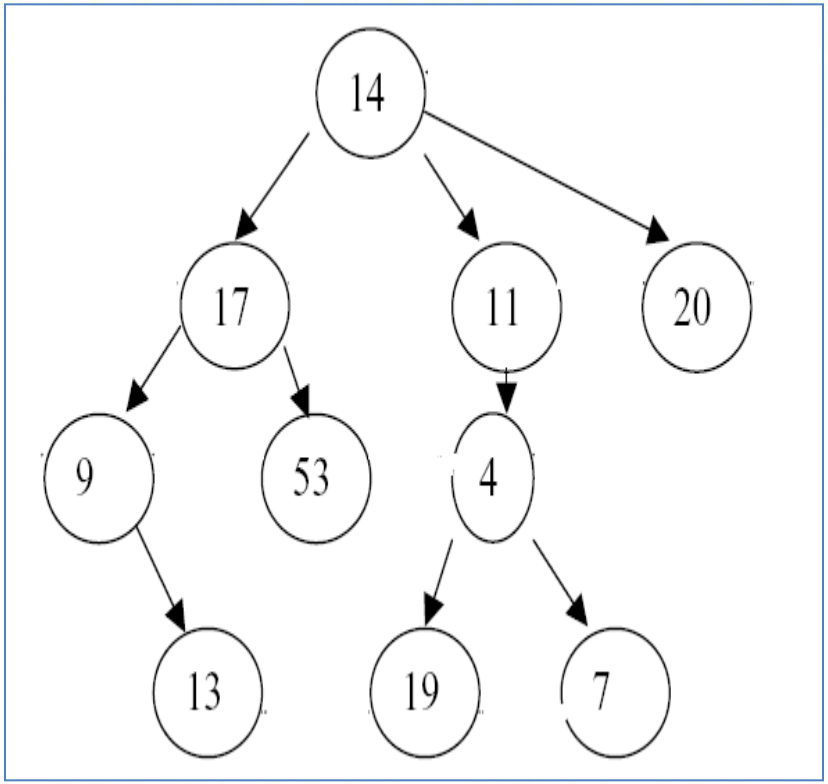


Subtree for node 11

cc BY NC SA

Some Terminologies

- **Ancestor of node *n***
  - A node on the path from the root to *n*
  - Ancesstor 13 : 9,17,14
  - Nod 14 is ancestor for all node in the tree
- **Descendant of node *n***
  - A node on a path from *n* to a leaf
  - Descendant 11: 4,19,7
  - All nodes in the tree are descendant to the root.

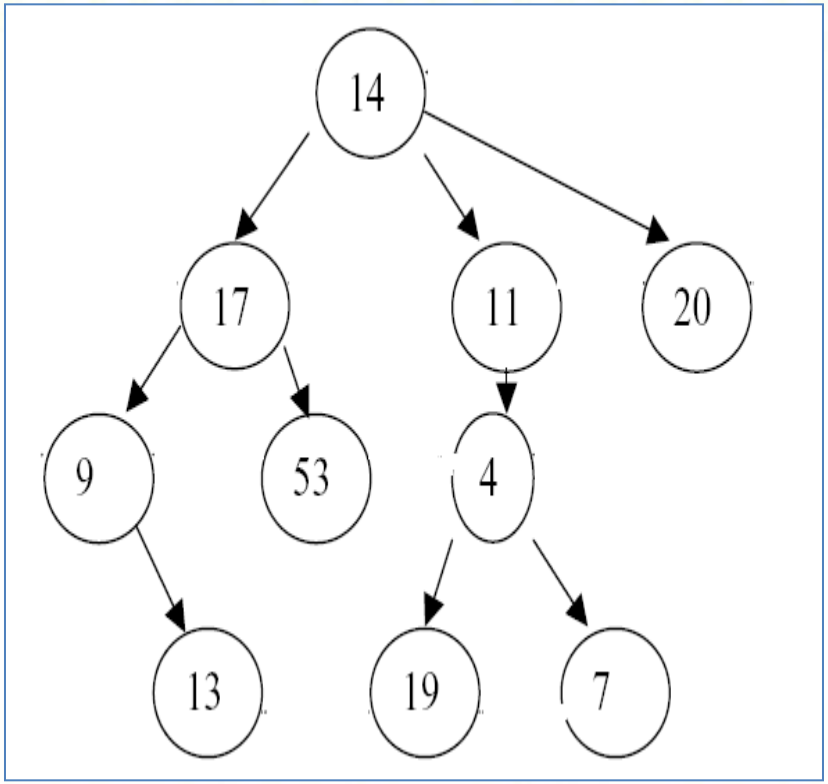


cc BY NC SA

Some Terminologies

- ***Path*** *– sequence of nodes in which each node is adjacent o the next one. Example: Path from root to 13: 14,17,9,13*

  Path from root to 19: 14,11,4,19
- ***Length***
  - number of edges on the path
  - Length of Tree : 3
- ***Depth* of a node**
  - length of the unique path from the root to that node
  - The depth of a tree is equal to the depth of the deepest leaf
  - Depth of Tree : 3
  - Depth of 4 : 2

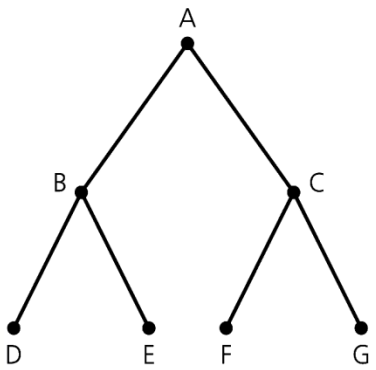


cc BY NC SA

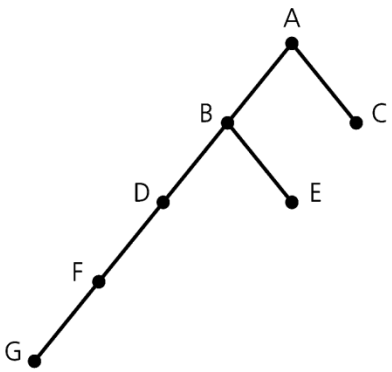
The Height of Trees

Height of a tree

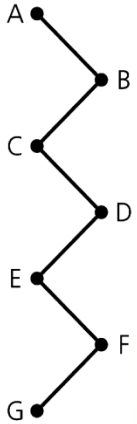
- Number of nodes along the longest path from the root to a leaf. *(Carrano,2007)*
- *If Tree is empty, its height is 0*



Height 3 (a)



Height 5 (b)



Height 7 (c)

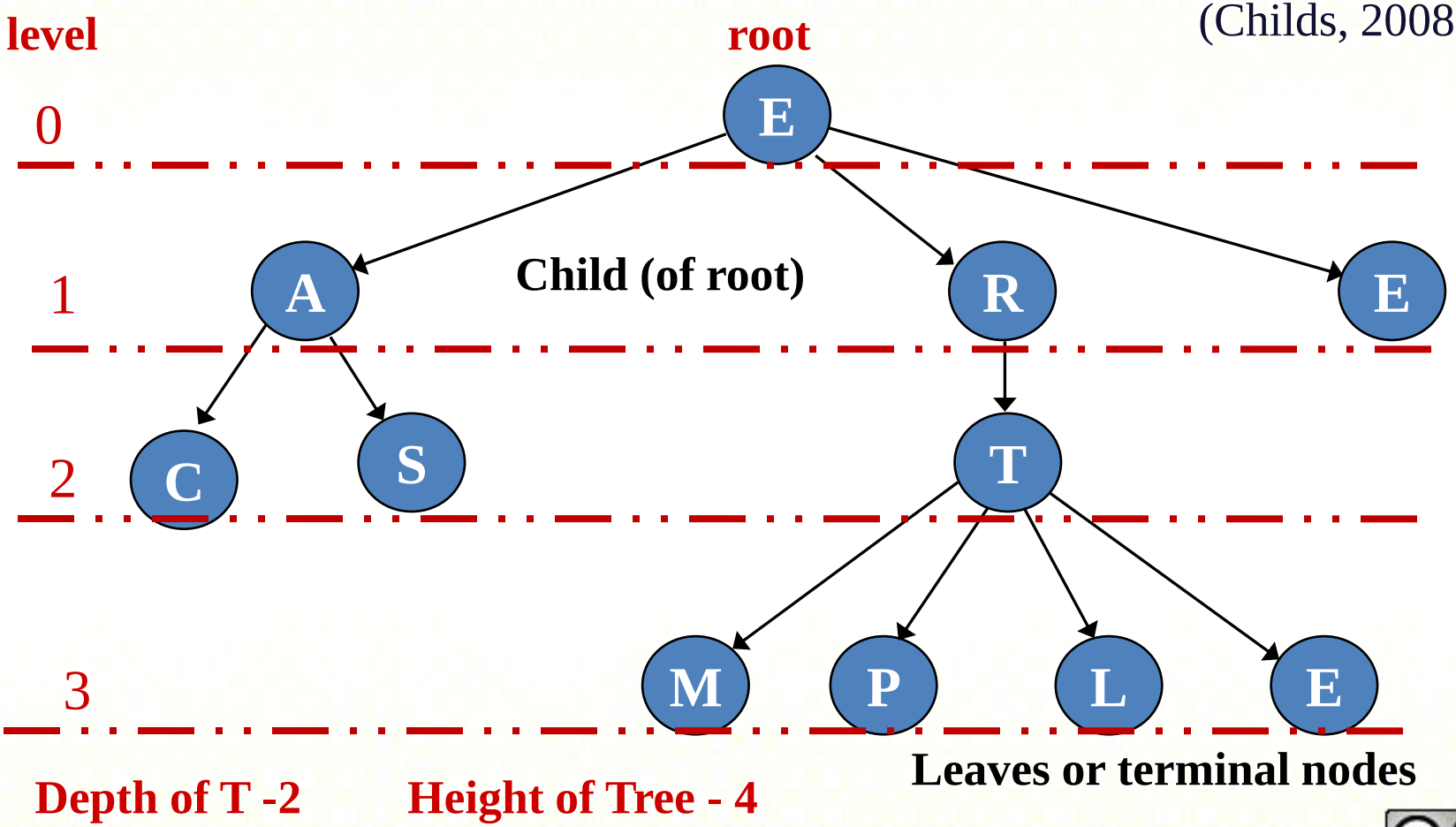
Binary trees with the same nodes but different heights

CC BY NC SA

Tree Terminologies

Level – the number of edges in the path from the root node to that node.

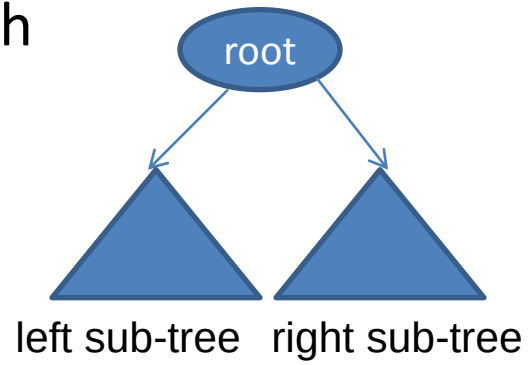
(Childs, 2008)



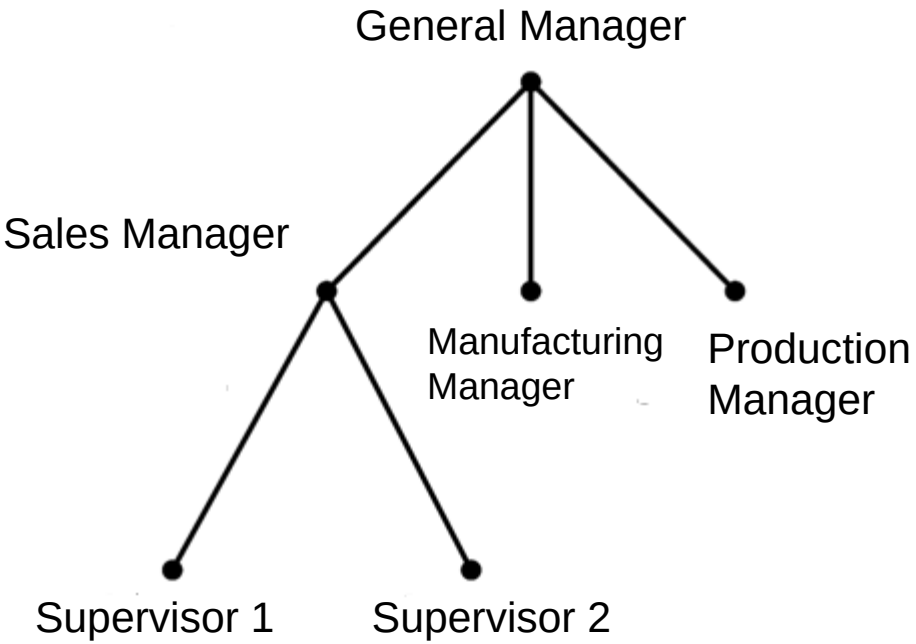
cc BY NC SA

Binary Tree Definition

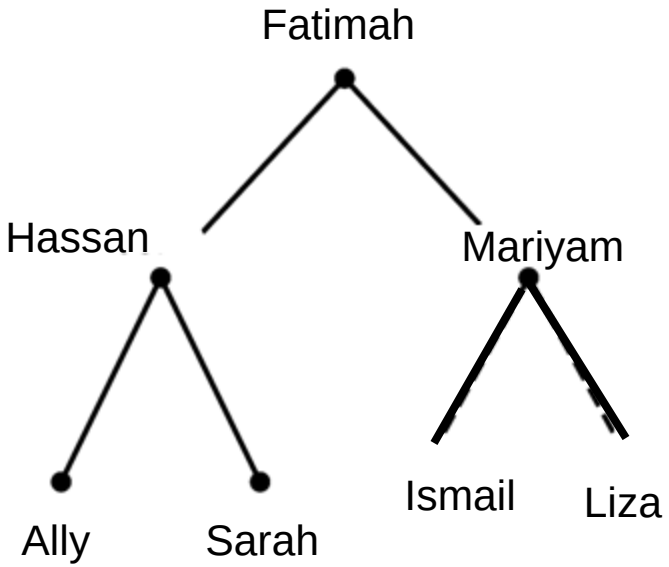
- A tree with restrictions, such that any given node can have at most two child nodes.
- A binary tree consists of a set of nodes such that either :
  - *Tree* is empty, or
  - *Tree* is partitioned into three disjoint subsets:
    - The root
    - Two possibly empty sets that are binary trees, called the left subtree of the *root* and the right subtree of the *root*



A General Tree vs A Binary Tree

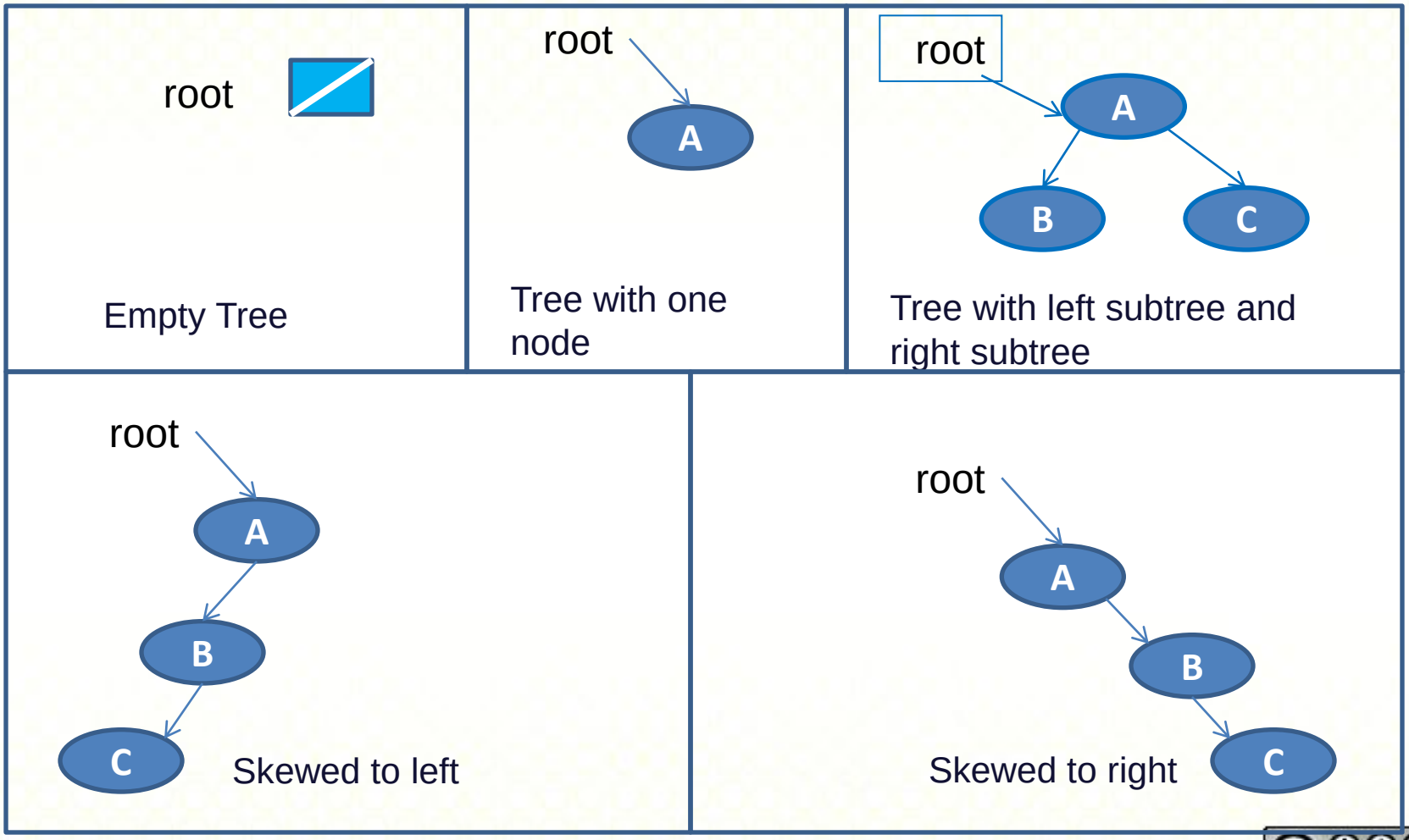


An organization chart



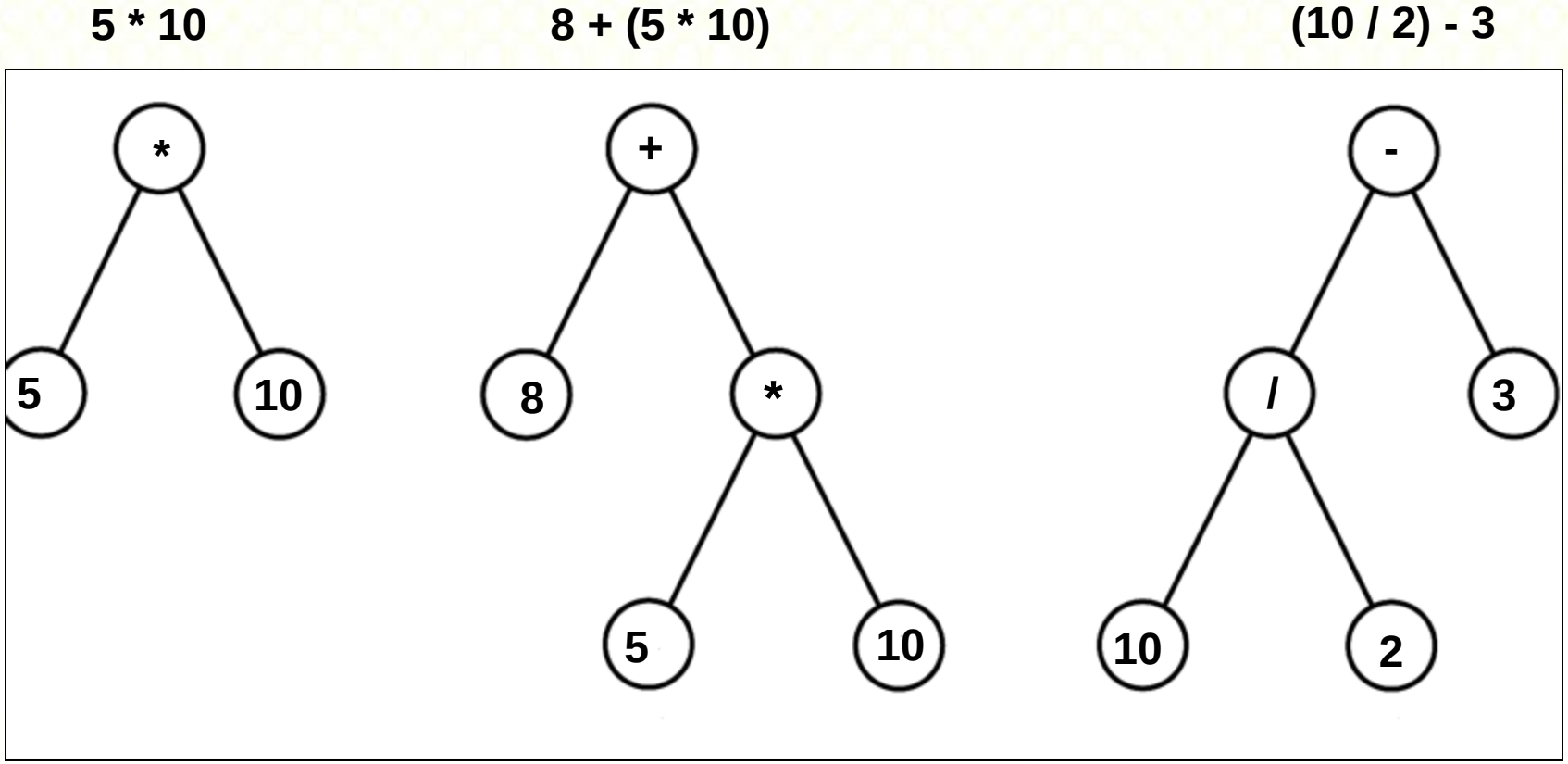
Family Tree

Collection of Binary Trees



cc BY NC SA

More Binary Trees

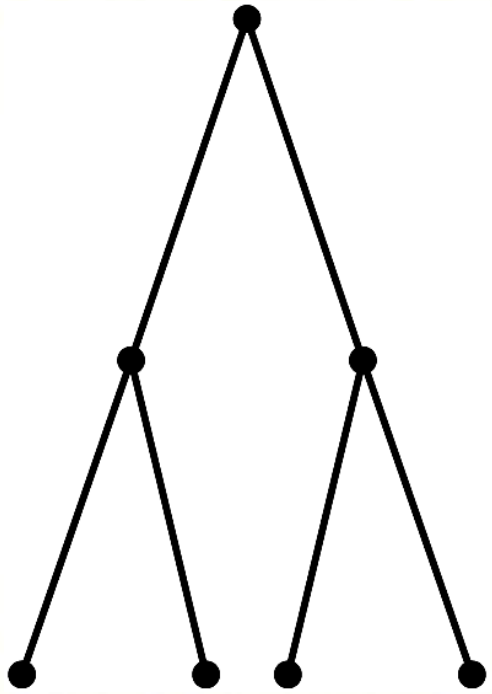


Binary trees that represent algebraic expressions.

cc BY NC SA

Full Binary Trees

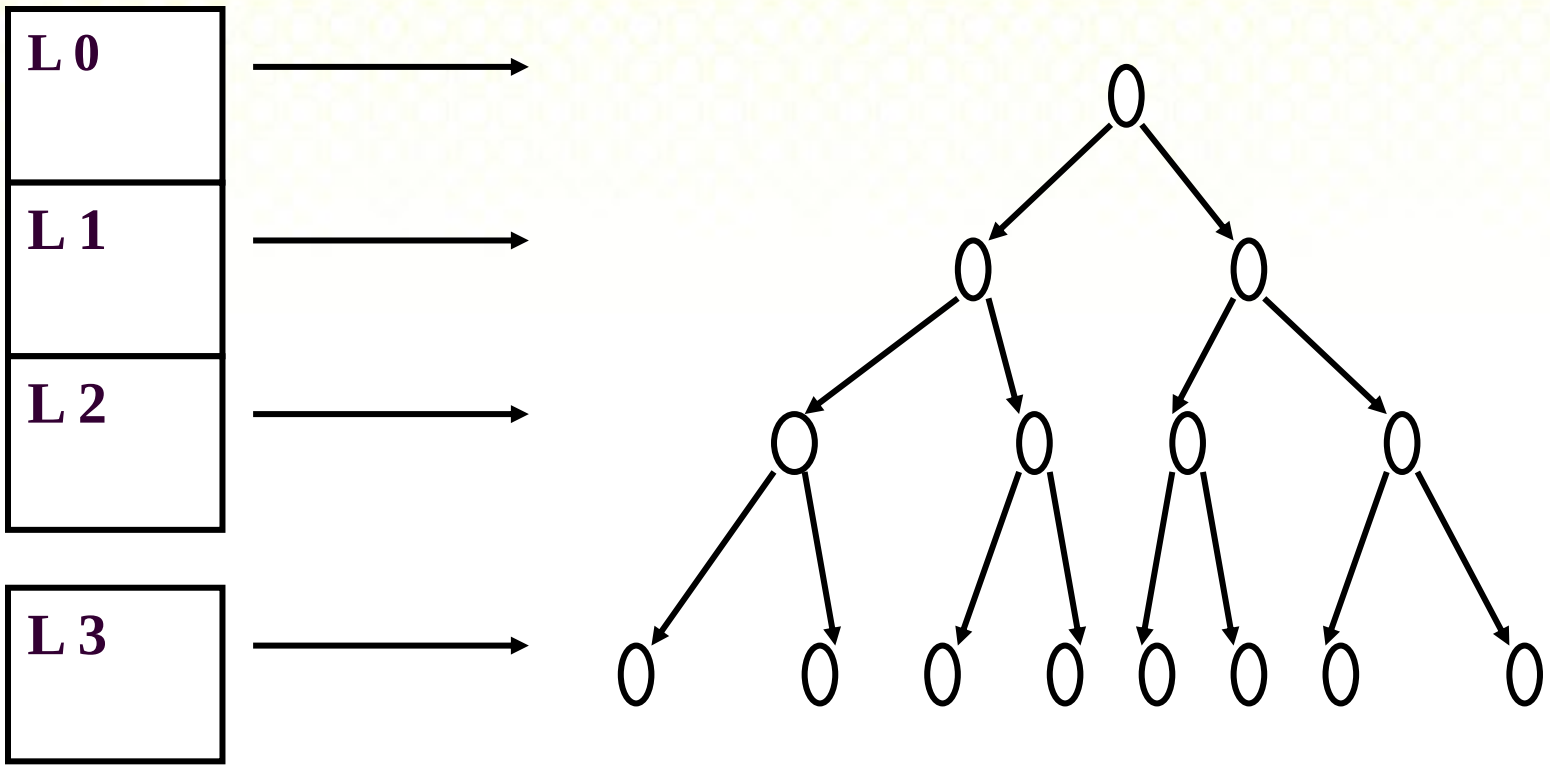
- A binary tree of height *h* is *full* if
  - Nodes at levels < *h* have two children each
- Recursive definition
  - If *T* is empty, *T* is a full binary tree of height 0
  - If *T* is not empty and has height *h* > 0, *T* is a full binary tree if its root's subtrees are both full binary trees of height *h* – 1



A full binary tree of height 3

CC BY NC SA

Full Binary Trees



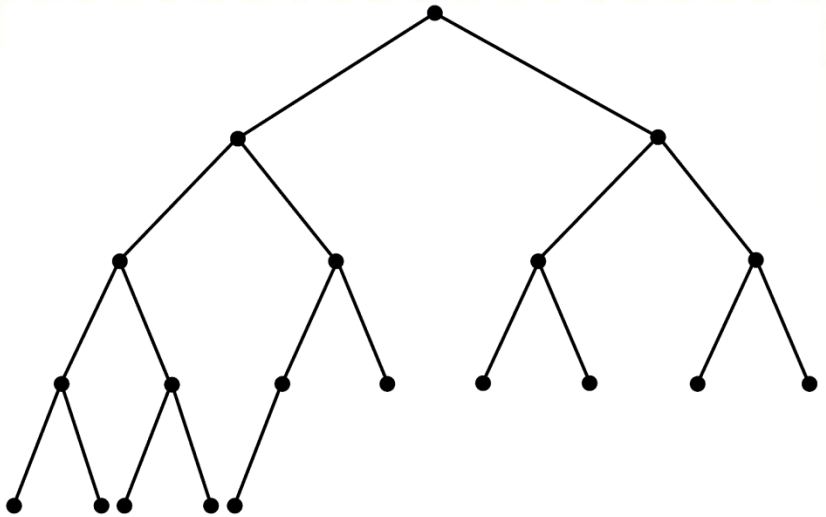
At each level the number of the nodes is doubled.

Total number of nodes: $1 + 2 + 2^2 + 2^3 = 2^4 - 1 = 15$

cc BY NC SA

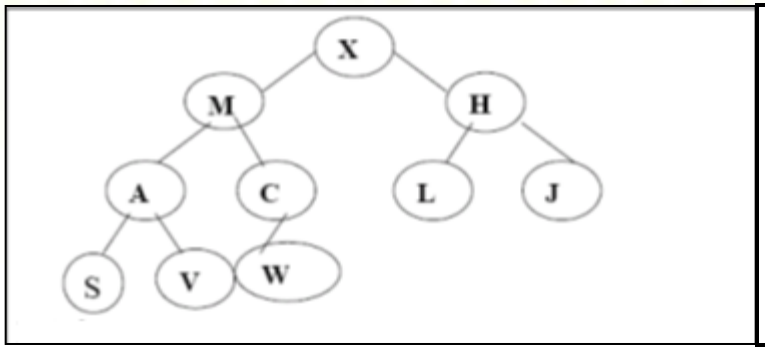
Complete Binary Trees

- A binary tree of height *h* is *complete* if
  - It is full to level *h*–1, and
  - Level *h* is filled from left to right

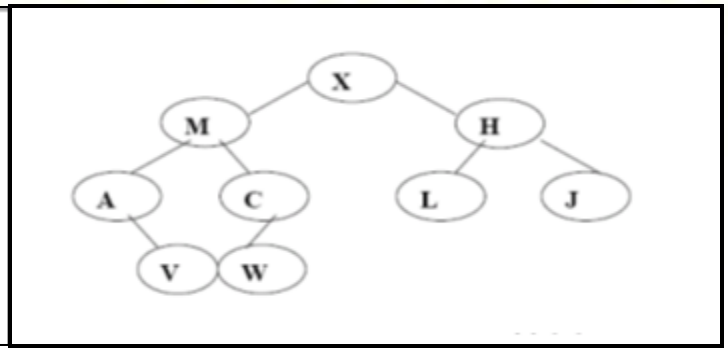


cc BY NC SA

Tree Examples

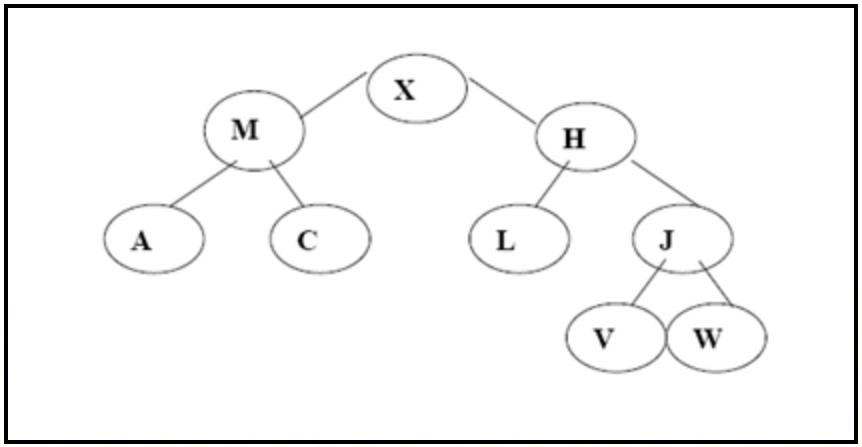


Binary tree that is complete but not full



Binary tree that is not complete and not full

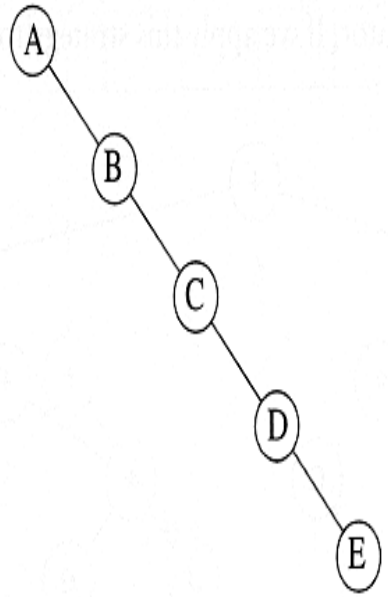
Binary tree that is not complete and not full



CC BY NC SA

Balanced Binary Trees

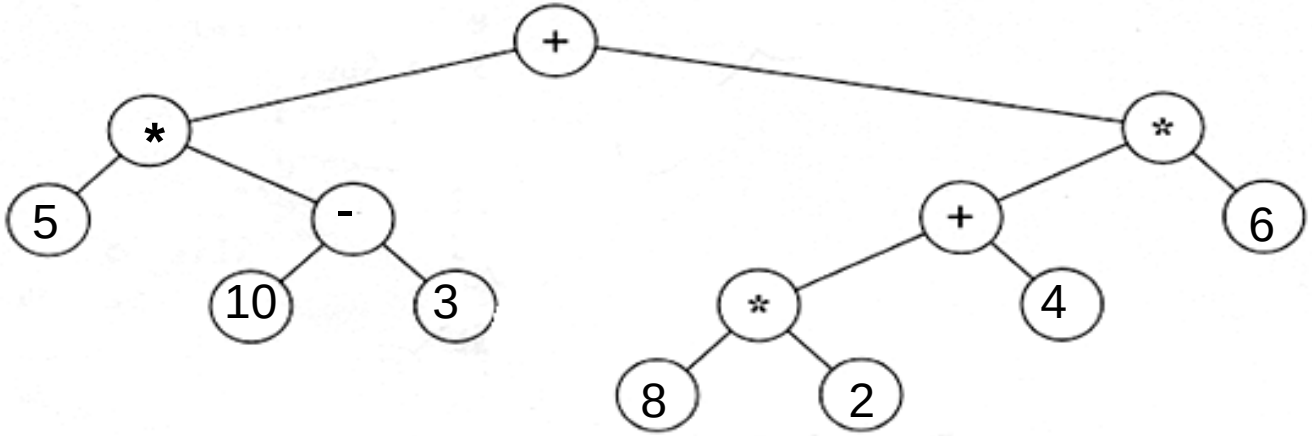
- A binary tree is balanced if the heights of any node's two sub-trees differ by no more than 1
- Complete binary trees are balanced.
- Full binary trees are complete and balanced.
- The depth of an average binary tree is considerably smaller than n, even though in the worst case, the depth can be as large as n – 1.



Unbalanced tree : skewed to the right. Depth = n-1 (4)

cc BY NC SA

Example: Expression Trees



Expression tree for ((5 * (10 – 3)) + ((8 * 2 + 4) * 6)

Expression Tree

- Leaves are operands (constants or variables)
- The other nodes (internal nodes) contain operators

cc BY NC SA

Tree traversal

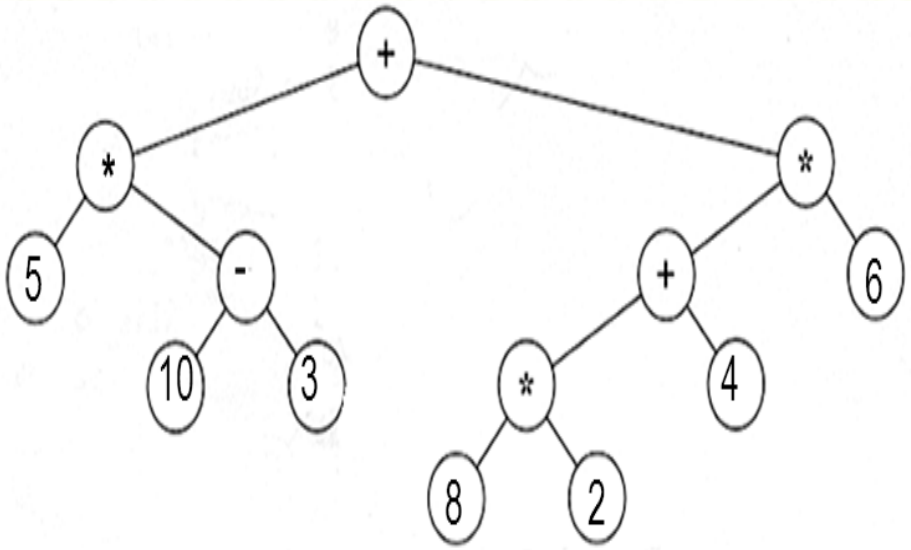
- Traverse a tree is to visit every node in a tree.
- Some operations can be done with the node during a visit.
  - For example, modify or update the data in the node
  - Used to print out the data in a tree in a certain order
- Type of Traversal
  - Inorder traversal
  - Preorder traversal
  - Postorder traversal

Pre-order traversal

Pre-order traversal

- Print the data at the root
- Recursively print out all data in the left subtree
- Recursively print out all data in the right subtree
- Give prefix expression

  +*5-103*+*8246



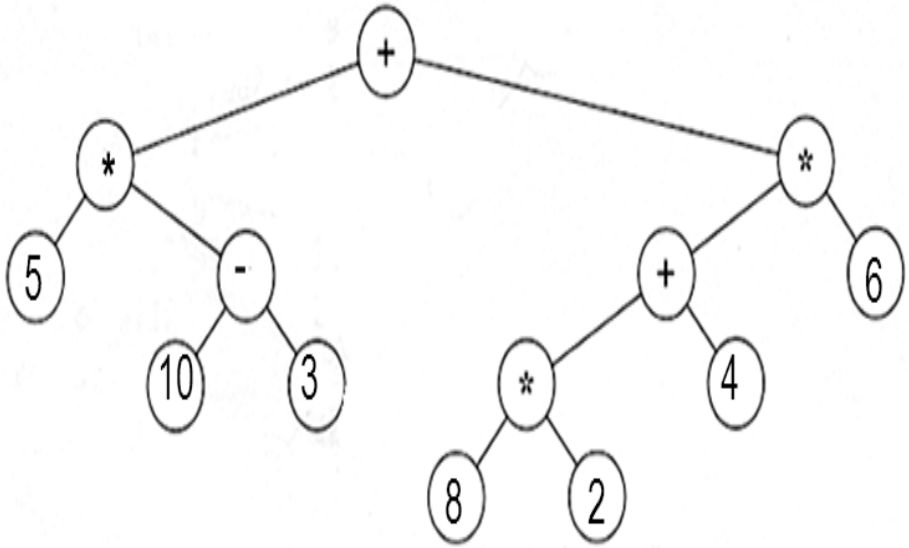
Expression tree for ((5 * (10 – 3)) + ((8 * 2 + 4) * 6)

cc BY NC SA

Postorder traversal

Postorder traversal

- Recursively print out all data in the left subtree
- Recursively print out all data in the right subtree
- Print the data at the root
- Give postfix expression
  5103-*82*4+6*+



Expression tree for ((5 * (10 – 3)) + ((8 * 2 + 4) * 6)

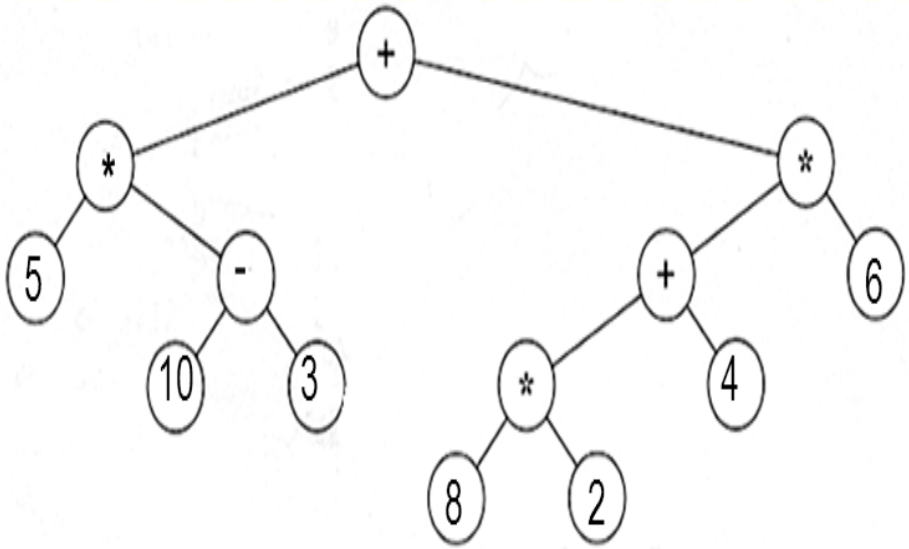
cc BY NC SA

Inorder traversal

Inorder traversal

- Recursively print out all data in the left subtree
- Print the data at the root
- Recursively print out all data in the right subtree
- Give infix expression

  5*10-3+8*2+4*6



Expression tree for ((5 * (10 – 3)) + ((8 * 2 + 4) * 6)

cc BY NC SA

Traversals of a Binary Tree

Pre-order traversal :

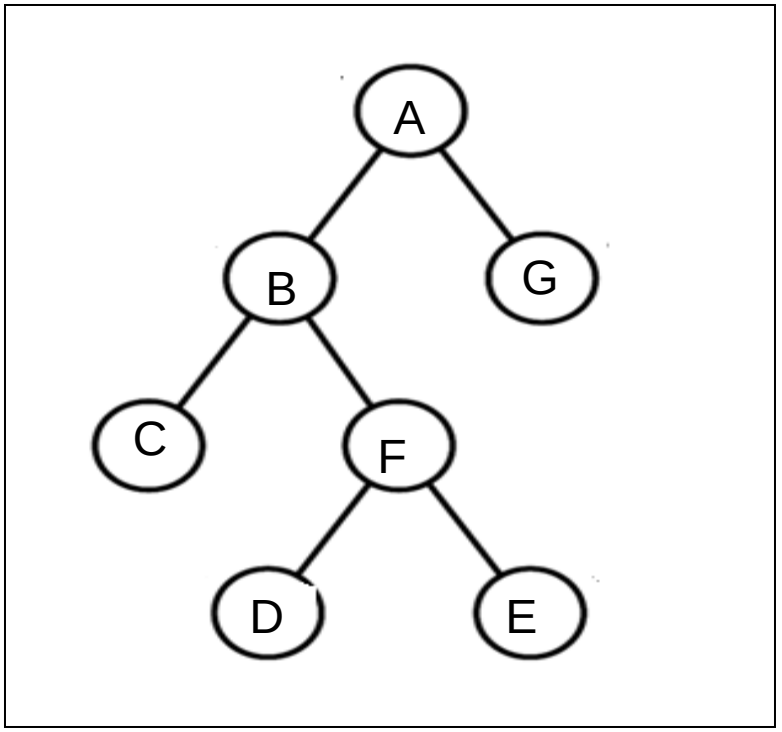
ABCFDEG

In-order traversal:

CBDFEAG

Post-order traversal:

CDEFBGA



Summary and Conclusion

- Tree provide a hierarchical organization of data with parent-child relationship.
- There are many types of tree such as general tree, binary tree and binary search tree.
- Terms related to tree : root, siblings, parent, leaf
- Traversing a tree is to visit every node in a tree either pre-order, in-order and post-order traversal.
- An in-order traversal of a binary search tree visits the tree's nodes in sorted search-key order

References

- Frank M. Carano, Janet J Prichard. "*Data Abstraction and problem solving with C++" Walls and Mirrors*. 5th edition (2007). Addision Wesley.
- Nor Bahiah et al. "*Struktur data & algoritma menggunakan C++". Penerbit UTM. 2005.*